

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP
ĐỀ TÀI: XÂY DỰNG HỆ THỐNG HỖ TRỢ ĐÁNH GIÁ CV VÀ GỢI Ý
VIỆC LÀM CHO ỨNG VIÊN DỰA TRÊN KIẾN TRÚC
MICROSERVICES VÀ TRÍ TUỆ NHÂN TẠO

Giảng viên hướng dẫn: ThS. NGUYỄN VŨ THÁI

Sinh viên thực hiện: PHẠM LỤC CHƯƠNG

Lớp : CQ.63.CNTT

Khoá : K63

TP. Hồ Chí Minh, năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP
ĐỀ TÀI: XÂY DỰNG HỆ THỐNG HỖ TRỢ ĐÁNH GIÁ CV VÀ GỢI Ý
VIỆC LÀM CHO ỨNG VIÊN DỰA TRÊN KIẾN TRÚC
MICROSERVICES VÀ TRÍ TUỆ NHÂN TẠO

Giảng viên hướng dẫn: ThS. NGUYỄN VŨ THÁI

Sinh viên thực hiện: PHẠM LỤC CHƯƠNG

Lớp : CQ.63.CNTT

Khoá : K63

TP. Hồ Chí Minh, năm 2026

NHIỆM VỤ BÁO CÁO ĐỒ ÁN TỐT NGHIỆP BỘ MÔN: CÔNG NGHỆ THÔNG TIN

-----***-----

1. Tên đề tài

Xây dựng hệ thống hỗ trợ đánh giá CV và gợi ý việc làm cho ứng viên dựa trên kiến trúc Microservices và trí tuệ nhân tạo.

2. Mục đích, yêu cầu

- Mục đích:

Xây dựng và triển khai nền tảng tuyển dụng thông minh SmartCV nhằm hỗ trợ quá trình kết nối giữa ứng viên và nhà tuyển dụng trên môi trường trực tuyến. Hệ thống ứng dụng trí tuệ nhân tạo để phân tích CV, so sánh mức độ phù hợp giữa hồ sơ ứng viên và mô tả công việc, từ đó đưa ra điểm đánh giá, các kỹ năng phù hợp, kỹ năng còn thiếu và gợi ý cải thiện. Bên cạnh đó, hệ thống được tổ chức theo kiến trúc microservices, hỗ trợ các nhóm người dùng chính gồm Ứng viên, Nhà tuyển dụng và Quản trị viên, góp phần rút ngắn thời gian sàng lọc hồ sơ, nâng cao chất lượng tuyển dụng và cải thiện trải nghiệm tìm việc.

- Yêu cầu:

- Hệ thống cho phép người dùng đăng ký, đăng nhập, xác thực email bằng OTP, quản lý thông tin cá nhân và phân quyền theo vai trò Ứng viên, Nhà tuyển dụng, Quản trị viên.
- Ứng viên có thể tạo và quản lý hồ sơ, tải CV dạng PDF, tìm kiếm việc làm, nộp đơn ứng tuyển và theo dõi trạng thái xử lý hồ sơ.
- Nhà tuyển dụng có thể đăng ký hồ sơ công ty, chờ quản trị viên phê duyệt, đăng tin tuyển dụng, quản lý danh sách ứng viên và theo dõi quy trình tuyển dụng bằng bảng Kanban ATS.

- AI Engine sử dụng mô hình ngôn ngữ lớn thông qua Spring AI và Llama3 API để phân tích CV, chấm điểm mức độ phù hợp, xác định kỹ năng khớp, kỹ năng còn thiếu và sinh gợi ý hoặc câu hỏi phỏng vấn.
- Dữ liệu hệ thống được lưu trữ, tìm kiếm và xử lý an toàn thông qua MongoDB, Redis, Elasticsearch, PostgreSQL và RabbitMQ, hệ thống có khả năng vận hành ổn định trên môi trường Docker Compose.

3. Phạm vi đề tài.

a) Nội dung nghiên cứu

- Nghiên cứu kiến trúc microservices, API Gateway, cơ chế giao tiếp giữa các service và mô hình xử lý bất đồng bộ bằng RabbitMQ trong hệ thống tuyển dụng trực tuyến.
- Nghiên cứu mô hình ngôn ngữ lớn LLM, kỹ thuật Prompt Engineering, Spring AI và Llama 3 API để xây dựng chức năng phân tích CV, đánh giá mức độ phù hợp và gợi ý cải thiện hồ sơ.
- Thiết kế cơ sở dữ liệu đa hệ gồm MongoDB cho dữ liệu nghiệp vụ, Redis cho quản lý OTP và cache, Elasticsearch cho tìm kiếm việc làm, PostgreSQL cho dịch vụ thông báo và lịch sử liên quan.
- Xây dựng các service Backend như User Service, Job Service, Application Service, AI Engine Service và Notification Service, kết hợp cơ chế xác thực JWT và phân quyền RBAC.
- Phát triển ba giao diện phần mềm riêng biệt cho ứng viên, nhà tuyển dụng và quản trị viên, hỗ trợ các luồng thao tác hoàn chỉnh từ đăng ký, tìm việc, nộp đơn, chấm điểm AI đến quản lý tuyển dụng.

b) Phạm vi đề tài

- Đề tài tập trung vào bài toán tuyển dụng trực tuyến và đánh giá CV dựa trên dữ liệu văn bản từ CV PDF và mô tả công việc.
- Phạm vi chức năng bao gồm quản lý tài khoản, xác thực OTP, hồ sơ ứng viên, hồ sơ nhà tuyển dụng, đăng và tìm kiếm việc làm, nộp đơn, chấm điểm

AI, quản lý ATS Kanban, thông báo email và thanh toán gói đăng tin ở mức mô phỏng hoặc môi trường kiểm thử.

- Hệ thống được xây dựng và kiểm thử chủ yếu trên môi trường máy cục bộ và Docker Compose, mở rộng sang triển khai deploy quy mô nhỏ.

4. Công nghệ, công cụ và ngôn ngữ lập trình

- Công nghệ: Spring Boot 3.x, Spring Cloud Gateway, Spring Security, Spring AI, Llama 3 API, RabbitMQ, React 19, TypeScript, TanStack Router, TanStack Query, Go, Echo, Docker.
- Công cụ: IntelliJ IDEA, VS Code, Postman, Swagger/OpenAPI, Orval, Docker Desktop, Git/GitHub, GitHub Actions, MongoDB Compass, DBeaver và các công cụ kiểm thử API liên quan.
- Ngôn ngữ lập trình: Java (phát triển các service Backend và AI Engine), TypeScript/JavaScript (phát triển Frontend), Go (phát triển Notification Service).
- Cơ sở dữ liệu và hạ tầng dữ liệu: MongoDB, Redis, Elasticsearch, PostgreSQL và RabbitMQ.

5. Các kết quả chính dự kiến sẽ đạt được và ứng dụng

- Hoàn thiện nền tảng SmartCV với đầy đủ các chức năng đăng ký, đăng nhập, xác thực OTP, quản lý hồ sơ ứng viên, hồ sơ nhà tuyển dụng, đăng tin và tìm kiếm việc làm.
- Tích hợp thành công AI Engine có khả năng phân tích CV, so sánh với mô tả công việc, trả về điểm phù hợp, kỹ năng phù hợp, kỹ năng còn thiếu và gợi ý cải thiện dưới dạng dữ liệu có cấu trúc.
- Xây dựng quy trình quản lý ứng viên cho nhà tuyển dụng bằng bảng Kanban ATS, hỗ trợ cập nhật trạng thái hồ sơ, sinh câu hỏi phỏng vấn bằng AI và gửi thông báo email tự động qua RabbitMQ.
- Ứng dụng thực tiễn: Hệ thống có thể hỗ trợ ứng viên cải thiện CV và lựa chọn công việc phù hợp hơn, đồng thời giúp nhà tuyển dụng giảm thời gian sàng lọc thủ công, nâng cao hiệu quả quản lý tuyển dụng và tạo nền tảng mở rộng cho các dịch vụ tuyển dụng thông minh.

6. Giáo viên và cán bộ hướng dẫn

Họ và tên : Nguyễn Vũ Thái

Điện thoại: 0762 932 115

Email: nvthai@utc2.edu.vn

Đơn vị công tác: Phân Hiệu Đại Học Giao Thông Vận Tải TP. Hồ Chí Minh

Ngày 20 tháng 06 năm 2026

Trưởng BM Công nghệ Thông tin

Đã giao nhiệm vụ TKTN

Giảng viên hướng dẫn

ThS. Trần Phong Nhã

ThS. Nguyễn Vũ Thái

LỜI CẢM ƠN

Lời đầu tiên, em xin trân trọng gửi đến Quý Thầy, Cô Bộ môn Công nghệ Thông tin – Trường Đại học Giao thông Vận tải, Phân hiệu tại Thành phố Hồ Chí Minh lời chúc sức khỏe và lời cảm ơn chân thành nhất. Trong suốt quá trình học tập và rèn luyện tại trường, Quý Thầy, Cô đã tận tình giảng dạy, truyền đạt cho em những kiến thức chuyên môn quý báu cùng với nhiều kinh nghiệm thực tiễn, tạo nên nền tảng vững chắc để em hoàn thành đồ án tốt nghiệp.

Em xin chân thành cảm ơn Ban Giám hiệu nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất và môi trường học tập, giúp em có điều kiện học tập, nghiên cứu và phát triển bản thân một cách toàn diện. Đồng thời, em xin gửi lời cảm ơn sâu sắc đến tập thể giảng viên Bộ môn Công nghệ Thông tin đã luôn đồng hành, hỗ trợ và định hướng cho em trong suốt quá trình học tập tại trường.

Đặc biệt, đề đề tài Xây dựng hệ thống hỗ trợ đánh giá CV và gợi ý việc làm cho ứng viên dựa trên kiến trúc Microservices và trí tuệ nhân tạo có thể hoàn thành, em xin bày tỏ lòng biết ơn sâu sắc nhất đến thầy ThS. Nguyễn Vũ Thái. Thầy là người đã tận tình hướng dẫn, góp ý và hỗ trợ em trong quá trình lựa chọn đề tài, xây dựng kiến trúc hệ thống, triển khai các chức năng trọng tâm và hoàn thiện nội dung báo cáo.

Trong quá trình nghiên cứu và phát triển sản phẩm đồ án, em đã gặp không ít khó khăn khi tìm hiểu kiến trúc microservices, tích hợp các service độc lập, xử lý luồng bất đồng bộ bằng RabbitMQ, triển khai AI phân tích CV và xây dựng giao diện cho nhiều nhóm người dùng khác nhau. Nhờ sự hướng dẫn tận tâm, những góp ý chi tiết và sự động viên kịp thời của thầy, em đã có thêm định hướng để từng bước giải quyết vấn đề, hoàn thiện sản phẩm và rèn luyện phương pháp làm việc khoa học hơn.

Một lần nữa, em xin gửi lời cảm ơn chân thành nhất đến thầy ThS. Nguyễn Vũ Thái cùng toàn thể Quý Thầy, Cô. Kính chúc Quý Thầy, Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái thêm nhiều thành công trong sự nghiệp trồng người.
Trân trọng cảm ơn!

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the paper.

Giảng viên hướng dẫn

vi

This image shows a full page of white paper with horizontal dotted lines, typical of primary-ruled notebook paper. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings present.

Thành phố Hồ Chí Minh, Ngày 20 tháng 06 năm 2026

vii

MỤC LỤC

NHIỆM VỤ BÁO CÁO ĐỒ ÁN TỐT NGHIỆP.....	i
LỜI CẢM ƠN.....	v
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN.....	vi
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN.....	vii
MỤC LỤC.....	viii
DANH MỤC VIẾT TẮT.....	xi
DANH MỤC HÌNH ẢNH.....	xiii
DANH MỤC BẢNG BIỂU.....	xiv
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI.....	1
1.1. Tổng quan về đề tài.....	1
1.2. Mục tiêu nghiên cứu.....	1
1.3. Đối tượng và phạm vi nghiên cứu.....	3
1.4. Phương pháp nghiên cứu.....	3
1.5. Cấu trúc báo cáo.....	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	7
2.1. Tổng quan về thị trường tuyển dụng trực tuyến và nhu cầu ứng dụng AI..	7
2.1.1. Thực trạng tuyển dụng trực tuyến tại Việt Nam.....	7
2.1.2. Vai trò của AI trong tuyển dụng.....	7
2.2. Mô hình ngôn ngữ lớn (LLM) và ứng dụng phân tích CV.....	8
2.2.1. Tổng quan về Large Language Model (LLM).....	8
2.2.2. Spring AI Framework và Llama 3 API.....	9
2.2.3. Kỹ thuật phân tích CV và đánh giá mức độ phù hợp.....	10
2.3. Kiến trúc Microservices và Message Queue.....	11
2.3.1. Kiến trúc Microservices.....	11
2.3.2. So sánh kiến trúc Microservices và Monolithic.....	11
2.3.3. API Gateway và bảo mật JWT.....	13
2.3.4. Message Queue với RabbitMQ.....	13
2.4. Cơ sở dữ liệu và tìm kiếm.....	14
2.4.1. So sánh cơ sở dữ liệu SQL và NoSQL.....	14

2.4.2. PostgreSQL - Cơ sở dữ liệu quan hệ.....	15
2.4.3. MongoDB - Document Store.....	16
2.4.4. Redis - Cache và OTP Store.....	17
2.4.5. Elasticsearch - Full-text Search.....	17
2.5. React và các công nghệ sử dụng cho Frontend.....	18
2.5.1. React 19 và TanStack Router.....	18
2.5.2. TanStack Query và quản lý Server State.....	19
2.5.3. Phân quyền RBAC trong hệ thống.....	19
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG.....	21
3.1. Khảo sát và phân tích yêu cầu hệ thống.....	21
3.1.1. Yêu cầu chức năng.....	21
3.1.2 Yêu cầu phi chức năng.....	22
3.2. Lựa chọn mô hình và kiến trúc phát triển.....	23
3.2.1. Kiến trúc phần mềm tổng thể.....	23
3.2.2. Lựa chọn công nghệ AI.....	24
3.3. Thiết kế chức năng của hệ thống.....	26
3.3.1. Biểu đồ Use Case tổng quát.....	26
3.3.2. Đặc tả các sơ đồ Use Case.....	Error! Bookmark not defined.
3.3.3. Biểu đồ tuần tự (Sequence Diagram).....	Error! Bookmark not defined.
3.3.4. Biểu đồ luồng (DFD).....	40
3.4. Biểu đồ lớp (Class Diagrams).....	44
3.4.1. Biểu đồ lớp User Service Database.....	44
3.4.2. Biểu đồ lớp Job Service Database.....	45
3.4.3. Biểu đồ lớp Application Service Database.....	46
3.4.4. Biểu đồ lớp Notification Service Database.....	47
3.4.5. Biểu đồ lớp AI Service Database....	Error! Bookmark not defined.
3.4.6. Biểu đồ lớp Tổng quát.....	48
3.5. Thiết kế cơ sở dữ liệu.....	48
3.5.1. Các Collections trong User Service.....	49
3.5.2. Các Collections trong Job Service.....	54

3.5.3. Các Collection trong Application Service.....	55
3.5.4. Các bảng trong Notification Service.....	58
3.5.5. Collection transactions_log - Lịch sử giao dịch thanh toán.....	58
3.5.6. Redis Key Schema - Cache và OTP.....	59
CHƯƠNG 4. THỰC NGHIỆM VÀ KẾT QUẢ.....	60
4.1. Môi trường phát triển và công nghệ sử dụng.....	60
4.1.1. Cấu hình phần cứng.....	60
4.1.2. Môi trường phần mềm.....	60
4.2. Thiết kế và cài đặt giao diện.....	61
4.2.1. Giao diện đăng ký, đăng nhập.....	61
4.2.2. Giao diện màn hình trang chủ.....	62
4.2.3. Giao diện màn hình cài đặt.....	63
4.2.4. Thiết kế và cài đặt giao diện quản trị người dùng.....	64
CHƯƠNG 5. KẾT LUẬN VÀ KIẾN NGHỊ.....	66
5.1. Kết quả đạt được.....	66
5.2. Hạn chế của hệ thống.....	67
5.3. Hướng phát triển.....	68
TÀI LIỆU THAM KHẢO.....	70
PHỤ LỤC.....	71

DANH MỤC VIẾT TẮT

Viết tắt	Đầy đủ	Giải thích / Mô tả
API	Application Programming Interface	Giao diện lập trình ứng dụng cho phép các hệ thống giao tiếp với nhau
DI	Dependency Injection	Cơ chế tiêm phụ thuộc – tự động cung cấp đối tượng cần thiết cho một class
IoC	Inversion of Control	Đảo ngược điều khiển – nguyên lý tách rời khởi tạo đối tượng khỏi nơi sử dụng
JVM	Java Virtual Machine	Máy ảo Java – môi trường chạy các chương trình Java
JDK	Java Development Kit	Bộ công cụ phát triển phần mềm Java
LTS	Long Term Support	Phiên bản được hỗ trợ dài hạn của phần mềm
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản – ngôn ngữ cấu trúc trang web
CSS	Cascading Style Sheets	Ngôn ngữ định kiểu cho trang web
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
REST	Representational State Transfer	Kiến trúc phần mềm dùng trong thiết kế web API
RESTful API	RESTful Application Programming Interface	API tuân theo kiến trúc REST
URI	Uniform Resource Identifier	Chuỗi xác định duy nhất tài nguyên trên web
JSON	JavaScript Object Notation	Định dạng dữ liệu nhẹ, dễ đọc, thường dùng trong API
XML	eXtensible Markup Language	Ngôn ngữ đánh dấu mở rộng dùng để lưu trữ và truyền dữ liệu

UI	User Interface	Giao diện người dùng
UX	User Experience	Trải nghiệm người dùng
MVC	Model-View-Controller	Mô hình thiết kế phần mềm phân chia chức năng thành 3 phần: Model, View, Controller
AOP	Aspect-Oriented Programming	Lập trình hướng khía cạnh – tách logic nghiệp vụ và logic phụ (như logging, bảo mật)
JDBC	Java Database Connectivity	API Java để kết nối và làm việc với cơ sở dữ liệu
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ đối tượng lập trình với cơ sở dữ liệu quan hệ
JMS	Java Message Service	API nhắn tin trong Java
SSMS	SQL Server Management Studio	Công cụ quản lý và làm việc với cơ sở dữ liệu SQL Server
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc dùng trong quản lý cơ sở dữ liệu
VS Code	Visual Studio Code	Trình soạn thảo mã nguồn của Microsoft

DANH MỤC HÌNH ẢNH

Hình 3.1	Biểu đồ usecase tổng quát.....	26
Hình 3.2	Biểu đồ Use Case: Xác thực và đăng ký	27
Hình 3.3	Biểu đồ Use Case: Ứng tuyển và AI đánh giá CV.....	29
Hình 3.4	Biểu đồ Use Case: Quản lý tuyển dụng và ứng viên.....	31
Hình 3.5	Biểu đồ Use Case: Quản trị viên vận hành hệ thống.....	33
Hình 3.6	Biểu đồ luồng đăng nhập.....	40
Hình 3.7	Biểu đồ luồng Ứng tuyển việc làm và AI chấm điểm CV.....	41
Hình 3.8	Biểu đồ luồng Quản lý việc làm và hồ sơ ứng tuyển.....	42
Hình 3.9	Biểu đồ luồng Quản lý hệ thống.....	43
Hình 3.10	Biểu đồ lớp User Service Database.....	44
Hình 3.11	Biểu đồ lớp Job Service Database.....	45
Hình 3.12	Biểu đồ lớp Application Service Database.....	46
Hình 3.13	Biểu đồ lớp Notification Service Database.....	47
Hình 3.14	Biểu đồ lớp Tổng quát.....	48
Hình 4.1	Giao diện đăng ký tài khoản.....	61
Hình 4.2	Giao diện đăng nhập.....	61
Hình 4.3	Giao diện quản lý hồ sơ ứng viên.....	62
Hình 4.4	Giao diện quản lý và xem nội dung CV.....	62
Hình 4.5	Giao diện kết quả AI phân tích CV.....	63
Hình 4.6	Giao diện Cài đặt tài khoản.....	63
Hình 4.7	Giao diện cài đặt thông báo và quyền cá nhân.....	64
Hình 4.8	Giao diện Quản trị người dùng.....	64
Hình 4.9	Giao diện kiểm duyệt tin tuyển dụng.....	65

DANH MỤC BẢNG BIỂU

Bảng 3.1	Bảng đặc tả Use Case đăng ký, xác thực OTP và đăng nhập.....	27
Bảng 3.2	Bảng đặc tả Use Case ứng viên tìm việc và ứng tuyển.....	29
Bảng 3.3	Bảng đặc tả Use Case nhà tuyển dụng quản lý việc làm và hồ sơ ứng tuyển.....	31
Bảng 3.4	Bảng đặc tả Use Case quản trị viên quản lý hệ thống.....	33
Bảng 3.5	Bảng mô tả cấu trúc Collection Users.....	49
Bảng 3.6	Bảng mô tả cấu trúc Collection role.....	49
Bảng 3.7	Bảng mô tả cấu trúc Collection user_roles.....	50
Bảng 3.8	Bảng mô tả cấu trúc Collection Candidates.....	50
Bảng 3.9	Bảng mô tả cấu trúc Collection Recruiters.....	52
Bảng 3.10	Bảng mô tả cấu trúc Collection Service Packages.....	53
Bảng 3.11	Bảng mô tả cấu trúc Collection Wishlists.....	53
Bảng 3.12	Bảng mô tả cấu trúc Collection Jobs.....	54
Bảng 3.13	Bảng mô tả cấu trúc Collection Applications.....	56
Bảng 3.14	Bảng mô tả cấu trúc Collection Assessments.....	57
Bảng 3.15	Bảng mô tả cấu trúc Collection Assessment Attempts.....	57
Bảng 3.16	Bảng mô tả cấu trúc Table Notifications.....	58
Bảng 3.17	Bảng mô tả cấu trúc Table Notification FCM Tokens.....	58
Bảng 4.1	Bảng môi trường phát triển (phần mềm).....	60

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Tổng quan về đề tài

Hiện nay, khi chuyển đổi số phát triển mạnh, hoạt động tuyển dụng tại Việt Nam ngày càng chuyển dịch từ hình thức truyền thống sang các nền tảng trực tuyến. Ứng viên có thể tạo hồ sơ, tìm kiếm việc làm và nộp CV nhanh chóng; doanh nghiệp cũng dễ dàng đăng tin, tiếp cận ứng viên và quản lý quy trình tuyển chọn. Tuy nhiên, sự gia tăng số lượng hồ sơ ứng tuyển khiến việc xử lý, phân loại và đánh giá CV trở thành một bài toán quan trọng cần được tự động hóa.

Hiện nay, nhiều nền tảng tuyển dụng vẫn chủ yếu hỗ trợ đăng tin, tìm kiếm và nộp hồ sơ. Sau khi tin tuyển dụng được công bố, nhà tuyển dụng thường phải đọc và sàng lọc thủ công nhiều CV với định dạng, nội dung và mức độ hoàn thiện khác nhau. Quá trình này tốn thời gian, dễ bỏ sót ứng viên phù hợp và phụ thuộc nhiều vào kinh nghiệm của bộ phận nhân sự. Ở phía ứng viên, người dùng cũng chưa có nhiều công cụ phản hồi rõ ràng về mức độ phù hợp của CV so với yêu cầu công việc.

Từ nhu cầu đó, đề tài Xây dựng hệ thống hỗ trợ đánh giá CV và gợi ý việc làm cho ứng viên dựa trên kiến trúc Microservices và trí tuệ nhân tạo được thực hiện nhằm xây dựng nền tảng SmartCV. Hệ thống ứng dụng mô hình ngôn ngữ lớn để phân tích CV, so sánh với mô tả công việc, chấm điểm mức độ phù hợp, chỉ ra kỹ năng khớp, kỹ năng còn thiếu và đưa ra lời khuyên cải thiện cho ứng viên.

Về mặt kỹ thuật, SmartCV được thiết kế theo kiến trúc microservices gồm API Gateway, User Service, Job Service, Application Service, AI Engine Service và Notification Service. Hệ thống sử dụng Spring Boot, Spring Cloud Gateway, Spring Security, Spring AI, RabbitMQ, MongoDB, Redis, Elasticsearch và PostgreSQL. Tầng giao diện gồm ba ứng dụng React/TypeScript cho ứng viên, nhà tuyển dụng và quản trị viên, giúp phân tách rõ vai trò người dùng và thuận tiện mở rộng hệ thống.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu, xây dựng và triển khai một nền tảng tuyển dụng thông minh có khả năng hỗ trợ đánh giá CV, gợi ý mức độ phù hợp giữa ứng viên và tin tuyển dụng, đồng thời cung cấp công cụ quản lý quy trình tuyển dụng cho doanh nghiệp. Hệ thống cần đảm bảo thao tác thuận tiện, dữ liệu được quản

lý an toàn, chức năng được phân quyền rõ ràng và kiến trúc phần mềm có khả năng mở rộng.

Các mục tiêu nghiên cứu cụ thể của đề tài bao gồm:

Nghiên cứu và ứng dụng trí tuệ nhân tạo trong phân tích CV: Tìm hiểu mô hình ngôn ngữ lớn, Prompt Engineering, structured output và phương pháp tích hợp AI vào ứng dụng doanh nghiệp. Hệ thống sử dụng Spring AI kết hợp Llama 3 API để phân tích CV và trả về kết quả có cấu trúc gồm điểm phù hợp, kỹ năng khớp, kỹ năng còn thiếu, nhận xét và gợi ý cải thiện. Kết quả này được lưu cùng hồ sơ ứng tuyển để phục vụ quá trình theo dõi và đánh giá về sau.

Xây dựng Backend theo kiến trúc microservices: Thiết kế các service độc lập gồm User Service, Job Service, Application Service, AI Engine Service và Notification Service. Các service giao tiếp đồng bộ qua REST API và bất đồng bộ qua RabbitMQ. API Gateway đảm nhiệm định tuyến, xác thực JWT tập trung và chuyển tiếp thông tin người dùng đến các service phía sau.

Thiết kế hệ thống lưu trữ dữ liệu phù hợp: Sử dụng MongoDB cho dữ liệu nghiệp vụ, Redis cho OTP và cache, Elasticsearch cho tìm kiếm việc làm full-text, PostgreSQL cho dữ liệu thông báo khi cần tính nhất quán quan hệ. Cách lựa chọn này giúp mỗi loại dữ liệu được xử lý bằng công nghệ phù hợp với đặc điểm truy vấn và lưu trữ, đồng thời hỗ trợ mở rộng từng thành phần độc lập khi số lượng người dùng và tin tuyển dụng tăng lên.

Xây dựng giao diện người dùng theo từng vai trò: Phát triển web-candidate cho ứng viên, web-recruiter cho nhà tuyển dụng và web-admin cho quản trị viên. Các giao diện hỗ trợ đăng ký, xác thực OTP, quản lý CV, tìm kiếm việc làm, nộp đơn, đăng tin tuyển dụng, duyệt hồ sơ công ty và theo dõi trạng thái hệ thống.

Cài đặt quy trình ATS Kanban: Hệ thống hỗ trợ nhà tuyển dụng quản lý hồ sơ ứng tuyển theo các trạng thái như Qualified, Under Review, Interview Scheduled, Offer Sent, Accepted và Rejected. Mỗi hồ sơ hiển thị thông tin ứng viên, điểm AI và trạng thái xử lý, giúp theo dõi pipeline tuyển dụng trực quan.

Kiểm thử và đánh giá hệ thống: Thực hiện kiểm thử chức năng từng service, kiểm thử tích hợp giữa Frontend, API Gateway, microservices, RabbitMQ và cơ sở dữ liệu. Đánh giá luồng AI scoring, luồng gửi thông báo bất đồng bộ, hiệu năng tìm kiếm bằng Elasticsearch và mức độ đáp ứng các tình huống sử dụng thực tế.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Đối tượng nghiên cứu của đề tài là hệ thống nền tảng tuyển dụng thông minh SmartCV có tích hợp trí tuệ nhân tạo để phân tích CV, đánh giá mức độ phù hợp giữa ứng viên và tin tuyển dụng, đồng thời hỗ trợ quản lý quy trình tuyển dụng theo mô hình ATS. Đề tài tập trung vào kiến trúc microservices, tích hợp AI vào Backend, thiết kế dữ liệu đa hệ và xây dựng giao diện theo vai trò.

Phạm vi nghiên cứu:

- Phạm vi người dùng: Hệ thống tập trung vào ba nhóm người dùng gồm ứng viên, nhà tuyển dụng và quản trị viên. Ứng viên quản lý CV, tìm kiếm việc làm và nộp đơn; nhà tuyển dụng đăng ký công ty, đăng tin và quản lý ứng viên qua ATS Kanban; quản trị viên duyệt hồ sơ công ty, quản lý người dùng và theo dõi hệ thống.
- Phạm vi trí tuệ nhân tạo: Đề tài tập trung phân tích CV dạng PDF/ đã trích xuất được văn bản và so sánh với mô tả công việc bằng mô hình ngôn ngữ lớn. Hệ thống không xử lý chữ viết tay, không đi sâu vào nhận dạng ảnh chuyên biệt và không huấn luyện mô hình AI mới từ đầu.
- Phạm vi chức năng hệ thống: Hệ thống bao gồm quản lý tài khoản, xác thực OTP, đăng nhập JWT, quản lý hồ sơ ứng viên, hồ sơ công ty, đăng và tìm kiếm việc làm, nộp đơn ứng tuyển, chấm điểm AI, quản lý hồ sơ Kanban ATS, sinh câu hỏi phỏng vấn, thanh toán gói đăng tin và gửi email tự động.
- Môi trường triển khai: Hệ thống được xây dựng và kiểm thử trên môi trường máy chủ cục bộ và Docker Compose. Phạm vi triển khai chưa bao gồm tối ưu hạ tầng cloud production quy mô lớn hoặc tích hợp chính thức với cổng thanh toán thương mại ngoài môi trường thử nghiệm.

1.4. Phương pháp nghiên cứu

- Phương pháp nghiên cứu lý thuyết:

Nghiên cứu kiến trúc microservices: Tìm hiểu nguyên lý chia nhỏ hệ thống thành các service độc lập, mô hình API Gateway, giao tiếp giữa các service và sự khác biệt so với kiến trúc monolith. Từ đó xác định cách phân tách các service trong SmartCV theo từng miền nghiệp vụ, đảm bảo mỗi service có phạm vi trách nhiệm rõ ràng.

Nghiên cứu trí tuệ nhân tạo và mô hình ngôn ngữ lớn: Tìm hiểu Large Language Model, Prompt Engineering, structured output và cách xây dựng prompt phân tích CV. Đồng thời nghiên cứu Spring AI như một lớp trừu tượng giúp Spring Boot giao tiếp với Llama 3 API.

Nghiên cứu bảo mật và phân quyền hệ thống: Tìm hiểu Spring Security, JWT, refresh token, BCrypt password hashing và RBAC. Các kiến thức này được áp dụng để xây dựng cơ chế xác thực stateless, bảo vệ API và kiểm soát quyền truy cập theo vai trò Candidate, Recruiter và Admin.

Nghiên cứu hệ sinh thái dữ liệu và xử lý bất đồng bộ: Tìm hiểu MongoDB, Redis, Elasticsearch và RabbitMQ. Việc nghiên cứu giúp lựa chọn đúng công cụ cho dữ liệu nghiệp vụ linh hoạt, dữ liệu tạm thời có TTL, dữ liệu tìm kiếm toàn văn và các luồng xử lý nền.

- Phương pháp thực nghiệm:

Thiết kế và kiểm thử pipeline phân tích CV: Xây dựng quy trình từ lúc ứng viên nộp đơn, hệ thống lấy nội dung CV và mô tả công việc, tạo prompt gửi đến AI Engine, nhận kết quả JSON và lưu kết quả vào đơn ứng tuyển. Các mẫu CV và tin tuyển dụng khác nhau được dùng để kiểm tra độ ổn định của kết quả.

Xây dựng và tích hợp từng service: Phát triển các module theo từng nhóm chức năng, sau đó kết nối qua API Gateway và RabbitMQ. Quá trình thực nghiệm tập trung vào các luồng đăng ký - OTP - đăng nhập, đăng tin - tìm kiếm việc làm, nộp đơn - AI scoring và cập nhật trạng thái trên Kanban ATS.

Kiểm thử cơ sở dữ liệu và tìm kiếm: Lưu trữ dữ liệu nghiệp vụ trên MongoDB, cache OTP và kết quả tìm kiếm bằng Redis, đồng bộ tin tuyển dụng sang Elasticsearch để kiểm thử tìm kiếm full-text kết hợp bộ lọc. Các tiêu chí đánh giá gồm tốc độ phản hồi, tính chính xác và khả năng phân trang.

Đánh giá giao diện và trải nghiệm người dùng: Kiểm thử ba ứng dụng Frontend theo các kịch bản end-to-end. Quá trình đánh giá tập trung vào tính dễ sử dụng, thông báo lỗi, hiển thị kết quả AI, thao tác Kanban và sự nhất quán giữa giao diện với quyền hạn người dùng.

1.5. Cấu trúc báo cáo

Báo cáo được tổ chức thành năm chương với nội dung chi tiết như sau:

Chương 1: Tổng quan về đề tài

- Bối cảnh tuyển dụng trực tuyến, lý do chọn đề tài và nhu cầu ứng dụng AI.
- Mục tiêu nghiên cứu, đối tượng nghiên cứu, phạm vi triển khai và phương pháp nghiên cứu của đề tài.
- Khái quát hệ thống SmartCV, các nhóm người dùng và ý nghĩa thực tiễn.
- Cấu trúc tổng thể của báo cáo.

Chương 2: Cơ sở lý thuyết

- Tổng quan tuyển dụng trực tuyến và vai trò của AI trong tuyển dụng.
- Mô hình ngôn ngữ lớn, Prompt Engineering, Spring AI và Llama 3 API trong bài toán phân tích CV.
- Microservices, API Gateway, JWT, RBAC và RabbitMQ.
- MongoDB, Redis, Elasticsearch và vai trò của từng công nghệ trong thiết kế dữ liệu của SmartCV.
- Frontend React, TanStack Router, TanStack Query và phân quyền giao diện.

Chương 3: Phân tích và thiết kế hệ thống

- Yêu cầu chức năng và phi chức năng của hệ thống SmartCV.
- Kiến trúc microservices và vai trò các thành phần chính.
- Thiết kế Use Case Diagram, Sequence Diagram và Activity Diagram cho các luồng nghiệp vụ trọng tâm.
- Thiết kế các collection dữ liệu chính, embedded document, transaction log và Redis key schema.
- Luồng giao tiếp giữa Frontend, Gateway, service, RabbitMQ và AI Engine.

Chương 4: Thực nghiệm và kết quả

- Môi trường phát triển, cấu hình phần cứng, môi trường phần mềm và các công nghệ được sử dụng.
- Cài đặt xác thực OTP, AI CV scoring, Elasticsearch và ATS Kanban.

- Giao diện người dùng cho ứng viên, nhà tuyển dụng và quản trị viên, kèm các luồng thao tác chính.
- Kết quả kiểm thử chức năng, tích hợp, hiệu năng và mức độ đáp ứng yêu cầu.

Chương 5: Kết luận và kiến nghị

- Tổng kết kết quả đạt được so với mục tiêu và phạm vi ban đầu.
- Đóng góp của SmartCV về AI, microservices và trải nghiệm tuyển dụng.
- Hạn chế còn tồn tại trong quá trình triển khai, kiểm thử, tích hợp AI và vận hành các service.
- Hướng phát triển: gợi ý việc làm, tối ưu AI, triển khai cloud và bổ sung tính năng tuyển dụng nâng cao.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về thị trường tuyển dụng trực tuyến và nhu cầu ứng dụng AI

2.1.1. Thực trạng tuyển dụng trực tuyến tại Việt Nam

Thị trường tuyển dụng trực tuyến phát triển nhanh. Ứng viên không còn phụ thuộc hoàn toàn vào hồ sơ giấy hoặc các buổi nộp hồ sơ trực tiếp như trước. Chỉ với một tài khoản, người tìm việc có thể tạo CV, lưu hồ sơ cá nhân, tìm kiếm vị trí phù hợp và gửi đơn đến nhiều doanh nghiệp trong thời gian ngắn.

Doanh nghiệp cũng thay đổi cách tuyển người. Tin tuyển dụng được đăng lên nền tảng trực tuyến, mạng xã hội nghề nghiệp và website công ty. Bộ phận nhân sự nhờ đó tiếp cận được lượng ứng viên lớn hơn, nhưng số lượng hồ sơ tăng nhanh lại tạo ra áp lực mới ở bước sàng lọc ban đầu.

Vấn đề xuất hiện khá rõ.

- Nhà tuyển dụng phải đọc nhiều CV có định dạng khác nhau.
- Thông tin kỹ năng, kinh nghiệm và học vấn thường không được trình bày đồng nhất.
- Ứng viên ít nhận được phản hồi cụ thể về lý do phù hợp hoặc chưa phù hợp.
- Quá trình lọc thủ công dễ bị ảnh hưởng bởi cảm tính và thời gian xử lý.

Các nền tảng như TopCV, VietnamWorks, LinkedIn hay website tuyển dụng nội bộ đã giúp kết nối nhanh hơn giữa ứng viên và doanh nghiệp. Tuy vậy, phần lớn hệ thống vẫn tập trung vào đăng tin, tìm kiếm và quản lý hồ sơ. Khâu đánh giá mức độ phù hợp giữa CV và mô tả công việc còn cần nhiều thao tác thủ công.

Đây là nhu cầu thực tế. Một hệ thống có khả năng phân tích CV tự động, so sánh với yêu cầu công việc và đưa ra điểm đánh giá có cấu trúc sẽ giúp cả hai phía tiết kiệm thời gian. Nhà tuyển dụng có thêm dữ liệu tham khảo. Ứng viên hiểu rõ hơn điểm mạnh và điểm còn thiếu trong hồ sơ.

2.1.2. Vai trò của AI trong tuyển dụng

AI được đưa vào tuyển dụng để hỗ trợ các bước có tính lặp lại. Công việc đọc CV, đối chiếu kỹ năng, phân nhóm ứng viên và tạo nhận xét ban đầu thường tốn nhiều thời gian nhưng có thể được chuẩn hóa bằng mô hình xử lý ngôn ngữ.

Vai trò của AI không phải thay thế hoàn toàn nhà tuyển dụng. Nó đóng vai trò như một lớp hỗ trợ ra quyết định. Kết quả do AI trả về cần được người có chuyên môn

kiểm tra, đặc biệt ở các vị trí yêu cầu đánh giá thái độ, văn hóa làm việc hoặc kinh nghiệm thực tế.

Trong hệ thống SmartCV, AI được dùng cho các nhóm tác vụ chính:

- Phân tích nội dung CV và rút ra thông tin về kỹ năng, kinh nghiệm, học vấn.
- So sánh CV với mô tả công việc và yêu cầu tuyển dụng.
- Tính điểm phù hợp để hỗ trợ sàng lọc ban đầu.
- Gợi ý kỹ năng còn thiếu để ứng viên cải thiện hồ sơ.
- Sinh câu hỏi phỏng vấn dựa trên CV và vị trí ứng tuyển.

Lợi ích dễ thấy là tốc độ. Khi hệ thống xử lý tự động phần đánh giá sơ bộ, nhà tuyển dụng có thể ưu tiên xem các hồ sơ phù hợp trước. Ứng viên cũng có phản hồi rõ ràng hơn thay vì chỉ biết hồ sơ đã gửi đi và chờ kết quả.

Tính công bằng cũng được cải thiện nếu hệ thống được thiết kế cẩn thận. Bộ tiêu chí đánh giá cần dựa trên kỹ năng, kinh nghiệm và yêu cầu công việc. Những yếu tố không liên quan đến năng lực cần được hạn chế trong prompt và trong quá trình hiển thị kết quả.

2.2. Mô hình ngôn ngữ lớn (LLM) và ứng dụng phân tích CV

2.2.1. Tổng quan về Large Language Model (LLM)

Large Language Model, viết tắt là LLM, là mô hình học sâu được huấn luyện trên lượng lớn dữ liệu văn bản. Mô hình có khả năng hiểu, sinh và biến đổi ngôn ngữ tự nhiên theo yêu cầu đầu vào. Điểm mạnh của LLM là khả năng nắm bắt quan hệ giữa các từ, câu và ý nghĩa trong văn bản dài.

Nền tảng quan trọng của nhiều LLM hiện đại là kiến trúc Transformer. Kiến trúc này sử dụng cơ chế Self-Attention để tính mức độ liên quan giữa các thành phần trong chuỗi đầu vào. Nhờ vậy, mô hình có thể xem xét nhiều phần của văn bản cùng lúc thay vì xử lý tuần tự đơn giản.

Cơ chế Self-Attention giúp mô hình hiểu liên hệ xa trong văn bản. Một kỹ năng được nêu ở phần đầu CV có thể liên quan đến kinh nghiệm dự án ở phần sau. Với bài toán tuyển dụng, khả năng này rất hữu ích vì thông tin trong CV thường phân tán ở nhiều mục khác nhau.

Một số dòng LLM phổ biến gồm GPT, Claude và Llama. Mỗi dòng mô hình có ưu điểm riêng về chất lượng suy luận, tốc độ phản hồi, chi phí triển khai và khả năng

chạy trên hạ tầng riêng. Với đồ án SmartCV, hướng tích hợp qua API được chọn để giảm độ phức tạp khi triển khai ban đầu.

Prompt Engineering là kỹ thuật thiết kế yêu cầu đầu vào cho LLM. Prompt càng rõ thì đầu ra càng ổn định. Với hệ thống phần mềm, prompt không chỉ yêu cầu mô hình trả lời bằng đoạn văn, mà còn cần ép kết quả theo cấu trúc dữ liệu cụ thể để Backend có thể lưu và xử lý tiếp.

Một prompt phân tích CV thường gồm các phần:

- Nội dung CV đã được trích xuất thành văn bản.
- Mô tả công việc và danh sách yêu cầu kỹ năng.
- Quy tắc chấm điểm phù hợp.
- Cấu trúc đầu ra mong muốn ở dạng JSON.
- Các ràng buộc về ngôn ngữ, độ dài và phạm vi nhận xét.

2.2.2. Spring AI Framework và Llama 3 API

Spring AI là một framework trong hệ sinh thái Spring. Framework này cung cấp lớp trừu tượng để ứng dụng Spring Boot có thể giao tiếp với nhiều nhà cung cấp mô hình AI theo cách thống nhất. Lập trình viên không cần tự xử lý toàn bộ phần gọi API, định dạng request và đọc response ở mức thấp.

Cách làm này phù hợp với SmartCV. Phần lớn Backend của hệ thống được xây dựng bằng Java Spring Boot, nên việc tích hợp AI thông qua Spring AI giúp mã nguồn nhất quán hơn. Service AI có thể được tổ chức như một microservice riêng, nhận dữ liệu từ Application Service rồi trả lại kết quả phân tích.

Llama 3 API được dùng như nguồn xử lý LLM. Khi nhận nội dung CV và mô tả công việc, AI Engine tạo prompt theo mẫu đã định nghĩa, gửi yêu cầu đến Azure Openai API, sau đó nhận về kết quả phân tích. Kết quả này được ánh xạ thành object trong Java và lưu vào MongoDB.

Luồng tích hợp cơ bản như sau:

- Application Service tạo đơn ứng tuyển và phát sự kiện cần chấm điểm CV.
- AI Engine Service nhận dữ liệu CV, mô tả công việc và thông tin vị trí.
- Spring AI gửi prompt đến Azure Openai API.
- Mô hình trả về JSON gồm điểm phù hợp, kỹ năng khớp, kỹ năng thiếu và gợi ý.
- AI Engine cập nhật kết quả phân tích vào đơn ứng tuyển.

Structured Output là phần quan trọng. Nếu AI trả về văn bản tự do, Backend khó kiểm tra và lưu trữ. Vì vậy, SmartCV yêu cầu mô hình trả về các trường rõ ràng như `match_score`, `matched_skills`, `missing_skills`, `advice` và `mock_questions_generated`. Khi cấu trúc ổn định, Frontend cũng hiển thị kết quả dễ hơn.

2.2.3. Kỹ thuật phân tích CV và đánh giá mức độ phù hợp

Phân tích CV là bài toán xử lý văn bản có nhiều bước. Dữ liệu đầu vào thường là file PDF hoặc DOCX do ứng viên tải lên. Hệ thống cần trích xuất nội dung văn bản, chuẩn hóa các khoảng trắng, loại bỏ ký tự thừa và giữ lại những phần quan trọng như kỹ năng, kinh nghiệm, dự án, học vấn và chứng chỉ.

Sau đó, hệ thống thực hiện CV-JD Matching. CV là hồ sơ ứng viên. JD là mô tả công việc. Mục tiêu của bước này là so sánh thông tin trong CV với yêu cầu tuyển dụng để ước lượng mức độ phù hợp.

Pipeline phân tích trong SmartCV gồm các bước:

- Ứng viên chọn CV và gửi đơn ứng tuyển.
- Application Service lấy nội dung CV đã parse và dữ liệu tin tuyển dụng.
- AI Engine tạo prompt từ CV, JD và bộ tiêu chí đánh giá.
- LLM phân tích kỹ năng, kinh nghiệm và khoảng thiếu.
- Kết quả được lưu vào embedded document `ai_analysis`.

Điểm phù hợp được dùng như chỉ số tham khảo. Hệ thống có thể diễn giải theo ba nhóm: từ 70 điểm trở lên là phù hợp cao, từ 50 đến 69 điểm là cần xem xét thêm, dưới 50 điểm là chưa phù hợp. Cách chia này giúp nhà tuyển dụng lọc nhanh nhưng không thay thế quyết định cuối cùng.

Một điểm cần lưu ý là CV có thể viết không đầy đủ. Có ứng viên có năng lực nhưng trình bày thiếu từ khóa. Vì vậy, kết quả AI nên đi kèm phần giải thích. Khi hệ thống chỉ ra kỹ năng khớp và kỹ năng còn thiếu, người dùng có cơ sở hiểu vì sao điểm số được đưa ra.

2.3. Kiến trúc Microservices và Message Queue

2.3.1. Kiến trúc Microservices

Microservices là kiến trúc chia hệ thống thành nhiều service nhỏ. Mỗi service chịu trách nhiệm cho một miền nghiệp vụ riêng và có thể phát triển, kiểm thử, triển khai độc lập. Cách tổ chức này khác với monolith, nơi nhiều chức năng cùng nằm trong một khối ứng dụng lớn.

Hệ thống có nhiều nhóm chức năng. Nếu gom toàn bộ vào một ứng dụng duy nhất, mã nguồn sẽ khó bảo trì khi hệ thống mở rộng. Vì vậy, đề tài chọn kiến trúc microservices để tách các phần như người dùng, tin tuyển dụng, đơn ứng tuyển, AI và thông báo.

Các service chính gồm:

- API Gateway: tiếp nhận request từ Frontend, định tuyến và kiểm tra xác thực.
- User Service: quản lý tài khoản, đăng nhập, OTP và vai trò người dùng.
- Job Service: quản lý tin tuyển dụng và đồng bộ dữ liệu tìm kiếm.
- Application Service: quản lý đơn ứng tuyển và trạng thái ATS.
- AI Engine Service: phân tích CV, chấm điểm và sinh câu hỏi phỏng vấn.
- Notification Service: gửi email OTP, email trạng thái và các thông báo hệ thống.

Nguyên tắc Database per Service được áp dụng ở mức thiết kế. Mỗi service sở hữu dữ liệu nghiệp vụ của mình và các service khác không truy cập trực tiếp vào dữ liệu đó. Khi cần trao đổi, các service dùng REST API hoặc sự kiện qua RabbitMQ.

Cách làm này giúp giảm phụ thuộc chéo. Một lỗi trong Notification Service không nên làm dừng chức năng tìm kiếm việc làm. Tương tự, AI Engine có thể xử lý chậm hơn nhưng luồng nộp đơn vẫn được ghi nhận trước, sau đó kết quả chấm điểm được cập nhật bất đồng bộ.

2.3.2. So sánh kiến trúc Microservices và Monolithic

Monolithic là kiến trúc trong đó các chức năng của hệ thống được phát triển, đóng gói và triển khai trong một ứng dụng duy nhất. Các module như quản lý người dùng, tin tuyển dụng, đơn ứng tuyển, phân tích CV và gửi thông báo cùng sử dụng chung một mã nguồn triển khai. Trong khi đó, microservices chia hệ thống thành nhiều service độc lập, mỗi service phụ trách một miền nghiệp vụ và giao tiếp với nhau thông qua API hoặc message broker.

Hai kiến trúc đều có ưu điểm và hạn chế riêng. Monolithic phù hợp với hệ thống nhỏ hoặc giai đoạn đầu vì cấu trúc đơn giản, dễ cài đặt, kiểm thử và triển khai. Tuy nhiên, khi số lượng chức năng tăng, mã nguồn có thể trở nên khó bảo trì, thời gian build và triển khai dài hơn, đồng thời việc thay đổi một module có thể ảnh hưởng đến toàn bộ ứng dụng.

Microservices có độ phức tạp triển khai cao hơn do phải quản lý nhiều service, cấu hình mạng, giám sát log, xử lý lỗi giao tiếp và đảm bảo tính nhất quán dữ liệu. Đổi lại, kiến trúc này cho phép phát triển, triển khai và mở rộng từng service độc lập. Một service gặp sự cố không nhất thiết làm dừng toàn bộ hệ thống, và từng thành phần có thể sử dụng công nghệ hoặc cơ sở dữ liệu phù hợp với đặc điểm nghiệp vụ.

Các điểm khác biệt chính giữa hai kiến trúc được thể hiện như sau:

- Tổ chức mã nguồn: Monolithic tập trung các module trong một ứng dụng; microservices tách các miền nghiệp vụ thành các service riêng.
- Triển khai: Monolithic được build và triển khai đồng thời; microservices cho phép triển khai độc lập từng service.
- Khả năng mở rộng: Monolithic thường phải mở rộng toàn bộ ứng dụng; microservices có thể ưu tiên mở rộng riêng service đang có tải cao.
- Khả năng bảo trì: Monolithic dễ quản lý ở quy mô nhỏ nhưng khó kiểm soát khi mã nguồn lớn; microservices giúp giới hạn phạm vi thay đổi trong từng service.
- Độ phức tạp vận hành: Monolithic đơn giản hơn về cấu hình và giám sát; microservices yêu cầu quản lý giao tiếp mạng, message broker, log phân tán và lỗi giữa các service.
- Khả năng cô lập sự cố: Lỗi nghiêm trọng trong monolithic có thể ảnh hưởng toàn bộ ứng dụng; với microservices, sự cố thường được giới hạn trong service liên quan nếu hệ thống có cơ chế xử lý phù hợp.

Đối với SmartCV, kiến trúc microservices được lựa chọn vì hệ thống gồm nhiều miền nghiệp vụ có đặc điểm xử lý khác nhau. User Service tập trung vào xác thực và phân quyền, Job Service xử lý tin tuyển dụng và tìm kiếm, Application Service quản lý hồ sơ ứng tuyển, AI Engine Service thực hiện tác vụ phân tích CV có thời gian xử lý lớn, còn Notification Service phụ trách gửi thông báo bất đồng bộ. Việc tách các thành phần này giúp hệ thống dễ mở rộng, cô lập sự cố và thay đổi công nghệ trong

từng service. Mặc dù chi phí triển khai và vận hành cao hơn monolithic, lựa chọn này phù hợp với mục tiêu nghiên cứu kiến trúc phân tán và khả năng mở rộng của đề tài.

2.3.3. API Gateway và bảo mật JWT

API Gateway là cửa vào của hệ thống Backend. Thay vì để Frontend gọi trực tiếp từng service, mọi request đi qua Gateway trước. Gateway chịu trách nhiệm định tuyến, kiểm tra token, giới hạn tần suất request và bổ sung thông tin người dùng cho service phía sau.

Trong SmartCV, API Gateway được xây dựng bằng Spring Cloud Gateway. Khi người dùng đăng nhập thành công, User Service cấp JWT. Client lưu token và gửi kèm trong header Authorization ở các request tiếp theo.

JWT phù hợp với cơ chế xác thực stateless. Server không cần lưu session truyền thống cho từng người dùng. Thông tin cơ bản như userId, email, vai trò và thời hạn token được mã hóa trong token, sau đó Gateway kiểm tra chữ ký để xác định request hợp lệ hay không.

Quy trình xử lý đơn giản như sau:

- Client gửi request đến API Gateway kèm Bearer Token.
- Gateway kiểm tra token còn hạn và chữ ký hợp lệ.
- Gateway lấy userId và role từ token.
- Request được chuyển tiếp đến service tương ứng.
- Service phía sau kiểm tra quyền chi tiết bằng annotation hoặc logic nghiệp vụ.

Hệ thống cũng cần refresh token. Access token nên có thời gian sống ngắn để giảm rủi ro nếu bị lộ. Refresh token có thời gian sống dài hơn và được dùng để cấp lại access token khi phiên làm việc còn hợp lệ.

2.3.4. Message Queue với RabbitMQ

RabbitMQ là message broker dùng để truyền thông điệp bất đồng bộ giữa các service. Thay vì service A gọi trực tiếp service B và chờ kết quả, service A có thể gửi một message vào queue. Service B đọc message khi sẵn sàng và xử lý sau.

Cơ chế này rất hợp với các tác vụ nền. Gửi email, chấm điểm CV bằng AI hoặc cập nhật thông báo không cần làm người dùng chờ quá lâu tại màn hình thao tác. Request chính có thể hoàn tất trước, còn các phần xử lý nặng được chuyển sang hàng đợi.

Trong hệ thống, RabbitMQ được dùng cho các luồng chính:

- cv.review.queue: kích hoạt AI Engine phân tích CV sau khi ứng viên nộp đơn.
- notification.email.queue: gửi email OTP, email thay đổi trạng thái và email thông báo kết quả.
- job.sync.queue: hỗ trợ đồng bộ tin tuyển dụng sang hệ thống tìm kiếm khi cần mở rộng.

Lợi ích lớn nhất là tách rời phụ thuộc. Application Service không cần biết chi tiết gửi email ra sao hoặc AI Engine gọi mô hình nào. Nó chỉ cần phát sự kiện đúng định dạng. Service tiêu thụ message sẽ xử lý phần còn lại.

RabbitMQ cũng giúp hệ thống ổn định hơn khi tải tăng. Nếu nhiều ứng viên nộp đơn cùng lúc, queue có thể giữ message và xử lý lần lượt. Khi cần, có thể tăng số consumer của AI Engine để nâng tốc độ xử lý mà không phải sửa toàn bộ kiến trúc.

2.4. Cơ sở dữ liệu và tìm kiếm

2.4.1. So sánh cơ sở dữ liệu SQL và NoSQL

Cơ sở dữ liệu SQL và NoSQL đều được sử dụng để lưu trữ dữ liệu, nhưng khác nhau về mô hình tổ chức, cách truy vấn và khả năng mở rộng. SQL tổ chức dữ liệu theo bảng, hàng và cột, đồng thời sử dụng lược đồ xác định trước. NoSQL hỗ trợ các mô hình linh hoạt hơn như document, key-value, column-family hoặc graph, phù hợp với dữ liệu có cấu trúc thay đổi thường xuyên.

Cơ sở dữ liệu SQL phù hợp với nghiệp vụ cần quan hệ rõ ràng, ràng buộc toàn vẹn và giao dịch có tính nhất quán cao. Các hệ quản trị như PostgreSQL hỗ trợ khóa chính, khóa ngoại, JOIN và transaction theo nguyên tắc ACID. Nhờ đó, SQL thường được lựa chọn cho dữ liệu thanh toán, lịch sử giao dịch, cấu hình hệ thống hoặc các nghiệp vụ cần đối soát chính xác.

Cơ sở dữ liệu NoSQL phù hợp với dữ liệu bán cấu trúc, khối lượng lớn hoặc yêu cầu thay đổi trường dữ liệu nhanh. MongoDB cho phép lưu document gần với cấu trúc object trong ứng dụng, giảm nhu cầu ánh xạ nhiều bảng và thuận tiện khi lưu hồ sơ ứng viên, tin tuyển dụng hoặc kết quả phân tích AI có nhiều mảng và đối tượng lồng nhau.

Một số tiêu chí so sánh chính gồm:

- Mô hình dữ liệu: SQL sử dụng bảng và quan hệ; NoSQL sử dụng document, key-value hoặc các mô hình phi quan hệ khác.
- Lược đồ dữ liệu: SQL có schema chặt chẽ; NoSQL linh hoạt hơn khi thêm hoặc thay đổi trường.
- Truy vấn: SQL mạnh về JOIN và truy vấn quan hệ; NoSQL tối ưu cho truy vấn theo cấu trúc document hoặc khóa.
- Tính nhất quán: SQL ưu tiên giao dịch ACID; NoSQL thường linh hoạt giữa tính nhất quán và khả năng mở rộng tùy hệ quản trị.
- Khả năng mở rộng: SQL thường mở rộng theo chiều dọc hoặc sử dụng cơ chế phân cụm; NoSQL thuận lợi hơn khi phân tán dữ liệu theo chiều ngang.
- Phạm vi sử dụng: SQL phù hợp với dữ liệu giao dịch và quan hệ; NoSQL phù hợp với dữ liệu linh hoạt, nội dung lớn và tốc độ phát triển nhanh.

Trong hệ thống, hai nhóm cơ sở dữ liệu được sử dụng kết hợp thay vì lựa chọn tuyệt đối một loại. PostgreSQL phục vụ dữ liệu cần tính quan hệ và nhất quán, trong khi MongoDB lưu phần lớn dữ liệu nghiệp vụ có cấu trúc linh hoạt. Cách tiếp cận này giúp hệ thống tận dụng ưu điểm của từng công nghệ theo đúng đặc điểm dữ liệu.

2.4.2. PostgreSQL - Cơ sở dữ liệu quan hệ

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ đầy đủ SQL, transaction, khóa ngoại, chỉ mục và nhiều kiểu dữ liệu mở rộng. Hệ quản trị này phù hợp với các nghiệp vụ yêu cầu dữ liệu có cấu trúc rõ ràng, quan hệ chặt chẽ và khả năng kiểm soát tính toàn vẹn.

Trong SmartCV, PostgreSQL được sử dụng cho Notification Service và các dữ liệu liên quan cần lưu lịch sử có cấu trúc. Ví dụ gồm mẫu thông báo, trạng thái gửi email, số lần thử lại, thời điểm gửi và thông tin lỗi. Việc tổ chức dữ liệu thành các bảng giúp truy vấn lịch sử, thống kê trạng thái và đối soát quá trình gửi thông báo thuận tiện hơn.

PostgreSQL hỗ trợ transaction theo nguyên tắc ACID. Khi một nghiệp vụ gồm nhiều thao tác liên quan, hệ thống có thể đảm bảo tất cả thao tác cùng thành công hoặc cùng được hoàn tác. Đặc điểm này phù hợp với dữ liệu giao dịch, quota đăng tin và các bản ghi cần độ chính xác cao.

Các ưu điểm chính của PostgreSQL trong hệ thống gồm:

- Hỗ trợ quan hệ giữa các bảng bằng khóa chính và khóa ngoại.
- Cung cấp transaction và cơ chế kiểm soát đồng thời đáng tin cậy.
- Hỗ trợ truy vấn tổng hợp, JOIN và báo cáo thống kê phức tạp.
- Có hệ thống chỉ mục đa dạng, phù hợp với dữ liệu cần tìm kiếm theo nhiều điều kiện.
- Tương thích tốt với các framework Backend và công cụ quản trị cơ sở dữ liệu.

Hạn chế của PostgreSQL là việc thay đổi schema cần được quản lý cẩn thận bằng migration. Với dữ liệu có cấu trúc biến đổi liên tục hoặc chứa nhiều object lồng nhau, mô hình quan hệ có thể tạo ra nhiều bảng và phép JOIN. Vì vậy, SmartCV chỉ sử dụng PostgreSQL cho các miền dữ liệu thực sự cần tính quan hệ và nhất quán cao.

2.4.3. MongoDB - Document Store

MongoDB là hệ quản trị cơ sở dữ liệu dạng document. Dữ liệu được lưu dưới dạng BSON, gần với cấu trúc JSON nên phù hợp với các object có nhiều trường linh hoạt. Với hệ thống tuyển dụng, hồ sơ ứng viên, hồ sơ công ty, tin tuyển dụng và kết quả AI có thể thay đổi theo từng trường hợp.

So với bảng quan hệ cố định, document store giúp lưu dữ liệu bán cấu trúc thuận tiện hơn. Ví dụ, một ứng viên có thể có nhiều kỹ năng, nhiều dự án, nhiều CV và nhiều kinh nghiệm làm việc. Những phần này có thể được lưu dưới dạng mảng hoặc embedded document.

Trong hệ thống, MongoDB phù hợp cho các collection như users, candidates, recruiters, jobs, applications và transactions_log. Các document có thể chứa thông tin nghiệp vụ chính cùng những trường mở rộng phục vụ hiển thị hoặc phân tích.

Embedded Document được dùng khi dữ liệu con gắn chặt với dữ liệu cha. Ví dụ, kết quả ai_analysis có thể nằm ngay trong document applications vì kết quả này phục vụ trực tiếp cho một đơn ứng tuyển. Khi nhà tuyển dụng mở chi tiết đơn, hệ thống lấy cả trạng thái ATS và kết quả AI trong một lần truy vấn.

MongoDB cũng hỗ trợ transaction trên nhiều document. Chức năng thanh toán gói đăng tin có thể cần vừa ghi transaction log, vừa cộng quota cho nhà tuyển dụng. Transaction giúp đảm bảo hoặc tất cả thao tác cùng thành công, hoặc dữ liệu được rollback để tránh lệch số liệu.

MongoDB hỗ trợ lập chỉ mục trên các trường đơn, trường kết hợp và dữ liệu nằm trong mảng. Trong SmartCV, chỉ mục có thể được tạo cho email, recruiter_id, candidate_user_id, job_id, status và created_at để tối ưu các truy vấn thường xuyên.

Ưu điểm của MongoDB là cấu trúc document linh hoạt và gần với dữ liệu JSON trao đổi qua API. Tuy nhiên, hệ thống vẫn cần quy định schema ở tầng ứng dụng, kiểm soát kích thước document và hạn chế nhúng dữ liệu quá lớn. Những quan hệ được truy cập độc lập hoặc thay đổi thường xuyên nên lưu bằng mã tham chiếu thay vì embedded document.

2.4.4. Redis - Cache và OTP Store

Redis là cơ sở dữ liệu key-value chạy trên bộ nhớ. Tốc độ đọc ghi rất nhanh. Redis thường được dùng cho cache, lưu session ngắn hạn, hàng đợi đơn giản hoặc dữ liệu có thời gian sống rõ ràng.

Redis được dùng trước hết cho OTP. Khi người dùng đăng ký hoặc xác thực email, hệ thống sinh mã OTP và lưu theo key gắn với email. Key có TTL ngắn, ví dụ 5 phút. Hết thời gian này, Redis tự xóa mã và người dùng phải yêu cầu mã mới.

Redis còn được dùng để cache kết quả tìm kiếm việc làm. Một số bộ lọc phổ biến như địa điểm, mức lương, ngành nghề hoặc từ khóa có thể được nhiều người dùng truy vấn lặp lại. Cache giúp giảm số lần gọi đến Elasticsearch và MongoDB.

Một số dạng key trong hệ thống:

- otp_email: lưu mã OTP theo email và có TTL ngắn.
- job_cache_filter_hash: lưu kết quả tìm kiếm theo bộ lọc đã bấm.
- rate_limit_userId: hỗ trợ giới hạn tần suất một số thao tác nhạy cảm.

Khi dùng Redis cần chú ý dữ liệu hết hạn. OTP phải xóa đúng thời điểm. Cache tìm kiếm cần được làm mới khi tin tuyển dụng thay đổi. Nếu không kiểm soát, người dùng có thể thấy dữ liệu cũ dù Backend đã cập nhật.

2.4.5. Elasticsearch - Full-text Search

Elasticsearch là công cụ tìm kiếm toàn văn dựa trên inverted index. Thay vì quét toàn bộ dữ liệu như cách truy vấn thông thường, Elasticsearch phân tích văn bản thành các token và lập chỉ mục để tìm kiếm nhanh trên khối lượng dữ liệu lớn.

Tìm kiếm việc làm không chỉ là tìm theo mã hoặc tên chính xác. Người dùng thường nhập từ khóa gần đúng như Java backend, thực tập frontend, data analyst hoặc

remote. Công cụ tìm kiếm cần hiểu nhiều trường dữ liệu như tiêu đề, mô tả, kỹ năng, địa điểm và ngành nghề.

Trong hệ thống, Job Service lưu tin tuyển dụng vào MongoDB. Sau đó, dữ liệu quan trọng được đồng bộ sang Elasticsearch index. Khi ứng viên tìm kiếm trên web-candidate, hệ thống ưu tiên truy vấn Elasticsearch để trả về kết quả nhanh và có khả năng xếp hạng theo độ phù hợp.

Các dạng truy vấn thường dùng gồm:

- Full-text query trên tiêu đề và mô tả công việc.
- Filter theo địa điểm, mức lương, loại công việc và ngành nghề.
- Sort theo ngày đăng, mức lương hoặc điểm liên quan.
- Pagination để chia kết quả thành nhiều trang.

Elasticsearch không thay thế MongoDB. MongoDB vẫn là nguồn dữ liệu nghiệp vụ chính. Elasticsearch đóng vai trò chỉ mục tìm kiếm. Khi có sai lệch, dữ liệu gốc trong MongoDB được ưu tiên và hệ thống có thể đồng bộ lại index.

2.5. React và các công nghệ sử dụng cho Frontend

2.5.1. React 19 và TanStack Router

React là thư viện xây dựng giao diện theo hướng component. Giao diện được chia thành nhiều thành phần nhỏ, mỗi thành phần quản lý phần hiển thị và logic tương ứng. Cách làm này phù hợp với hệ thống có nhiều màn hình như SmartCV.

Phiên bản React 19 tiếp tục hỗ trợ cách viết giao diện hiện đại, tối ưu render và kết hợp tốt với TypeScript. Khi dùng TypeScript, các props, dữ liệu API và trạng thái component được kiểm tra kiểu rõ hơn, giúp giảm lỗi trong quá trình phát triển.

SmartCV có ba ứng dụng Frontend riêng: web-candidate, web-recruiter và web-admin. Việc tách giao diện theo vai trò giúp mỗi nhóm người dùng có luồng thao tác gọn hơn. Ứng viên không cần thấy các chức năng duyệt công ty. Nhà tuyển dụng không cần thấy chức năng quản trị toàn hệ thống.

TanStack Router được dùng để quản lý định tuyến. Router hỗ trợ kiểu route an toàn, tham số route có type và cơ chế beforeLoad để kiểm tra quyền trước khi cho người dùng vào trang. Đây là phần quan trọng với các trang cần đăng nhập.

Ví dụ, trang quản lý ứng viên của nhà tuyển dụng chỉ nên mở khi người dùng có vai trò Recruiter. Nếu token thiếu hoặc role không hợp lệ, Frontend chuyển người dùng về trang đăng nhập hoặc trang không có quyền truy cập.

2.5.2. TanStack Query và quản lý Server State

Server State là dữ liệu nằm ở Backend nhưng được hiển thị trên Frontend. Ví dụ gồm danh sách việc làm, hồ sơ ứng tuyển, thông tin tài khoản và kết quả AI. Dữ liệu này có thể thay đổi do nhiều người dùng hoặc nhiều service cùng cập nhật.

TanStack Query giúp quản lý Server State tốt hơn cách tự viết state thủ công bằng `useState` và `useEffect`. Thư viện hỗ trợ cache dữ liệu, `refetch` khi cần, xử lý `loading`, `error` và `mutation`. Nhờ vậy, code giao diện ngắn hơn và ít lặp lại.

Trong hệ thống, TanStack Query được dùng cho các màn hình như danh sách việc làm, chi tiết tin tuyển dụng, danh sách hồ sơ ứng tuyển và bảng Kanban ATS. Khi nhà tuyển dụng cập nhật trạng thái ứng viên, `mutation` gửi request lên Backend và có thể làm mới dữ liệu sau khi thành công.

Orval được dùng để sinh client API từ tài liệu OpenAPI hoặc Swagger. Khi Backend định nghĩa endpoint rõ ràng, Orval tạo ra các hàm gọi API và hook tương ứng cho Frontend. Cách này giúp giảm lỗi sai URL, sai kiểu request hoặc sai kiểu response.

Sự kết hợp giữa TanStack Query và Orval giúp tăng tính type-safe. Frontend biết dữ liệu trả về có cấu trúc gì. Khi Backend thay đổi contract, lỗi sẽ xuất hiện sớm trong quá trình build thay vì chỉ phát hiện khi chạy ứng dụng.

2.5.3. Phân quyền RBAC trong hệ thống

RBAC là mô hình phân quyền dựa trên vai trò. Người dùng không được gán quyền rời rạc từng chức năng theo cách thủ công, mà được gán vào một vai trò. Mỗi vai trò có tập quyền tương ứng với nhiệm vụ trong hệ thống.

Hệ thống sử dụng ba vai trò chính. Candidate là ứng viên. Recruiter là nhà tuyển dụng. Admin là quản trị viên. Việc phân quyền rõ ràng giúp giảm rủi ro truy cập sai dữ liệu, đặc biệt với thông tin cá nhân, CV và hồ sơ công ty.

Các vai trò chính gồm:

- `ROLE_CANDIDATE`: tìm kiếm việc làm, quản lý CV, nộp đơn và xem trạng thái ứng tuyển.

- **ROLE_RECRUITER**: quản lý hồ sơ công ty, đăng tin tuyển dụng, xem ứng viên và cập nhật Kanban ATS.
- **ROLE_ADMIN**: duyệt hồ sơ công ty, quản lý người dùng và theo dõi hoạt động hệ thống.

Phân quyền cần được kiểm tra ở cả Frontend và Backend. Frontend ẩn menu không phù hợp để giao diện gọn hơn. Backend vẫn phải kiểm tra quyền ở Gateway và trong từng service, vì người dùng có thể gọi API trực tiếp nếu chỉ bảo vệ ở giao diện.

Nguyên tắc đặc quyền tối thiểu được áp dụng. Người dùng chỉ có quyền đúng với nhiệm vụ cần làm. Cách này giúp hệ thống an toàn hơn và dễ kiểm soát khi mở rộng thêm vai trò mới như HR Staff, Company Owner hoặc Support.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1. Khảo sát và phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Hệ thống được xây dựng để hỗ trợ quy trình tuyển dụng trực tuyến từ phía ứng viên, nhà tuyển dụng và quản trị viên. Mỗi nhóm người dùng có mục tiêu khác nhau, nên yêu cầu chức năng cần được tách rõ ngay từ bước phân tích.

Luồng nghiệp vụ chính bắt đầu từ tài khoản. Người dùng đăng ký, xác thực email bằng OTP, đăng nhập bằng JWT và sử dụng các chức năng theo vai trò được cấp. Sau đó, hệ thống xử lý các nghiệp vụ riêng như quản lý hồ sơ, đăng tin, nộp đơn, phân tích CV và gửi thông báo.

Các yêu cầu chức năng chính gồm:

- Quản lý tài khoản người dùng: cho phép đăng ký, đăng nhập, xác thực email bằng OTP, đổi mật khẩu, cập nhật thông tin cá nhân và quản lý trạng thái tài khoản.
- Quản lý hồ sơ ứng viên: lưu kỹ năng, kinh nghiệm, học vấn, danh sách CV đã tải lên và CV mặc định dùng khi nộp đơn.
- Quản lý hồ sơ nhà tuyển dụng: lưu thông tin công ty, mã số thuế, địa chỉ, logo, giấy phép kinh doanh và trạng thái phê duyệt.
- Phê duyệt công ty: nhà tuyển dụng sau khi đăng ký cần gửi hồ sơ công ty. Quản trị viên kiểm tra và chuyển trạng thái sang APPROVED hoặc REJECTED.
- Quản lý tin tuyển dụng: nhà tuyển dụng có thể tạo tin, chỉnh sửa tin, đóng tin, theo dõi thời hạn và sử dụng quota đăng tin.
- Tìm kiếm việc làm: ứng viên tìm tin tuyển dụng bằng từ khóa, địa điểm, mức lương, ngành nghề và các bộ lọc liên quan.
- Nộp đơn ứng tuyển: ứng viên chọn CV phù hợp, gửi đơn vào tin tuyển dụng và theo dõi trạng thái xử lý.
- Chấm điểm CV bằng AI: hệ thống phân tích nội dung CV so với mô tả công việc, trả về điểm phù hợp, kỹ năng khớp, kỹ năng thiếu và gợi ý cải thiện.
- Quản lý hồ sơ ứng tuyển dạng Kanban: nhà tuyển dụng theo dõi ứng viên theo từng cột trạng thái như QUALIFIED, UNDER_REVIEW, INTERVIEW_SCHEDULED, OFFER_SENT, ACCEPTED hoặc REJECTED.

- Sinh câu hỏi phỏng vấn bằng AI: hệ thống tạo câu hỏi dựa trên CV ứng viên và yêu cầu vị trí để nhà tuyển dụng tham khảo trước buổi phỏng vấn.
- Thanh toán gói đăng tin: nhà tuyển dụng mua thêm quota, hệ thống ghi nhận giao dịch và cập nhật số lượt đăng tin sau khi thanh toán thành công.
- Thông báo email tự động: gửi OTP, thông báo duyệt công ty, thông báo thay đổi trạng thái đơn ứng tuyển và các email nghiệp vụ khác.

Các chức năng trên liên kết chặt với nhau. Nếu thiếu xác thực OTP, tài khoản chưa đủ tin cậy. Nếu thiếu AI scoring, hệ thống chỉ còn giống một trang đăng tin thông thường. Nếu thiếu ATS Kanban, nhà tuyển dụng khó theo dõi nhiều hồ sơ cùng lúc.

Điểm cần giữ là tính kiểm soát. AI chỉ hỗ trợ đánh giá sơ bộ. Quyết định tuyển dụng cuối cùng vẫn thuộc về nhà tuyển dụng, vì dữ liệu CV có thể thiếu ngữ cảnh hoặc chưa phản ánh đầy đủ năng lực thực tế của ứng viên.

3.1.2 Yêu cầu phi chức năng

Bên cạnh chức năng, hệ thống cần đáp ứng các yêu cầu chất lượng để có thể vận hành ổn định. Đây là phần quan trọng vì SmartCV xử lý dữ liệu cá nhân, CV, thông tin công ty và dữ liệu giao dịch.

Các yêu cầu phi chức năng được xác định như sau:

- Bảo mật: mật khẩu được mã hóa bằng BCrypt. Các API quan trọng sử dụng JWT và phân quyền theo vai trò. Endpoint nhạy cảm cần được kiểm soát bằng Spring Security và cơ chế kiểm tra quyền.
- Hiệu năng: các API đọc dữ liệu phổ biến cần phản hồi nhanh. Redis được dùng để cache OTP và một số kết quả tìm kiếm. Elasticsearch hỗ trợ tìm kiếm toàn văn thay vì truy vấn thủ công trên cơ sở dữ liệu chính.
- Tính sẵn sàng: các service được tách độc lập. Nếu AI Engine tạm thời chậm hoặc lỗi, hệ thống vẫn có thể ghi nhận đơn ứng tuyển và cập nhật kết quả sau.
- Khả năng mở rộng: từng microservice có thể mở rộng riêng. Khi lượng tìm kiếm tăng, Job Service và Elasticsearch có thể được ưu tiên scale trước.
- Tính dễ sử dụng: hệ thống có ba giao diện riêng cho ứng viên, nhà tuyển dụng và quản trị viên. Mỗi giao diện chỉ hiển thị các chức năng phù hợp với vai trò.

- Tính nhất quán dữ liệu: các thao tác liên quan đến thanh toán, quota đăng tin và trạng thái đơn ứng tuyển cần được kiểm soát để tránh cập nhật thiếu hoặc sai lệch.

Tài liệu hóa API: các service cung cấp Swagger hoặc OpenAPI để Frontend có thể sinh client code bằng Orval và giám lỗi khi tích hợp.

Khả năng bảo trì: mã nguồn được chia theo lớp Controller, Service, Repository và DTO. Cách chia này giúp sửa đổi từng module mà ít ảnh hưởng đến phần còn lại.

Một yêu cầu thực tế khác là phản hồi lỗi phải rõ ràng. Người dùng không nên nhận thông báo kỹ thuật khó hiểu. Với các lỗi như OTP hết hạn, CV không hợp lệ, hết quota đăng tin hoặc công ty chưa được duyệt, hệ thống cần trả về thông báo dễ hiểu và đúng trạng thái.

3.2. Lựa chọn mô hình và kiến trúc phát triển

3.2.1. Kiến trúc phần mềm tổng thể

Hệ thống được thiết kế theo kiến trúc microservices. Hệ thống không gom toàn bộ chức năng vào một ứng dụng duy nhất mà chia thành nhiều service theo miền nghiệp vụ. Cách tổ chức này phù hợp với hệ thống có nhiều nhóm chức năng và nhiều loại người dùng.

Kiến trúc tổng thể gồm các lớp chính:

- Client Apps: gồm web-candidate, web-recruiter và web-admin. Ba ứng dụng được xây dựng bằng React và TypeScript, phục vụ ba nhóm người dùng khác nhau.
- API Gateway: là điểm vào chung của Backend. Gateway định tuyến request, kiểm tra JWT, áp dụng một số chính sách bảo mật và chuyển tiếp thông tin người dùng xuống service phía sau.
- Business Services: gồm User Service, Job Service và Application Service. Các service này xử lý nghiệp vụ chính như tài khoản, tin tuyển dụng và đơn ứng tuyển.
- AI Engine Service: nhận dữ liệu CV và mô tả công việc, gọi LLM thông qua Spring AI, sau đó trả kết quả chấm điểm cho hệ thống.
- Notification Service: xử lý gửi email OTP, email duyệt công ty, email thay đổi trạng thái đơn và các thông báo nền khác.

- Message Broker: RabbitMQ được dùng để truyền sự kiện giữa các service trong các tác vụ bất đồng bộ.
- Data Layer: gồm MongoDB cho dữ liệu nghiệp vụ, Redis cho cache và OTP, Elasticsearch cho tìm kiếm toàn văn.

Luồng giao tiếp trong hệ thống có hai dạng. Dạng thứ nhất là đồng bộ qua REST API, dùng cho các thao tác cần phản hồi ngay như đăng nhập, tìm việc, xem hồ sơ hoặc cập nhật thông tin. Dạng thứ hai là bất đồng bộ qua RabbitMQ, dùng cho các tác vụ không cần chặn luồng chính như gửi email và chấm điểm CV.

Cách tách service giúp hệ thống dễ phát triển theo nhóm. User Service có thể tập trung vào xác thực. Job Service tập trung vào tin tuyển dụng và tìm kiếm. Application Service quản lý đơn ứng tuyển. AI Engine có thể thay đổi mô hình hoặc prompt mà không làm ảnh hưởng trực tiếp đến Frontend.

Kiến trúc này cũng có chi phí riêng. Việc triển khai microservices đòi hỏi kiểm soát cấu hình, log, message queue và lỗi mạng giữa các service. Vì vậy, trong phạm vi đề án, hệ thống ưu tiên Docker Compose để chạy đồng bộ các thành phần trên môi trường phát triển.

3.2.2. Lựa chọn công nghệ AI

Bài toán chính của hệ thống là phân tích CV và so sánh với mô tả công việc. Đây là bài toán xử lý ngôn ngữ tự nhiên, không chỉ là so khớp từ khóa đơn giản. Một kỹ năng có thể được viết bằng nhiều cách khác nhau. Một kinh nghiệm dự án có thể ngầm thể hiện năng lực dù không nêu trực tiếp tên công nghệ.

Vì vậy, đề tài chọn hướng sử dụng LLM thông qua Spring AI và Azure Openai API. Cách tiếp cận này giúp hệ thống tận dụng khả năng hiểu ngữ cảnh của mô hình ngôn ngữ lớn mà không cần tự xây dựng tập dữ liệu huấn luyện lớn ngay từ đầu.

So với mô hình machine learning truyền thống, LLM có một số điểm phù hợp với đề án:

- Có thể phân tích văn bản tiếng Việt và tiếng Anh trong CV với cùng một pipeline.
- Không cần huấn luyện mô hình riêng từ đầu trong giai đoạn đầu triển khai.
- Có thể trả về giải thích kèm điểm số, giúp người dùng hiểu kết quả hơn.
- Dễ điều chỉnh hành vi bằng prompt và schema đầu ra.

- Tích hợp thuận tiện với Spring Boot thông qua Spring AI.

Kết quả AI được yêu cầu ở dạng có cấu trúc. Các trường quan trọng gồm `match_score`, `matched_skills`, `missing_skills`, `advice` và `mock_questions_generated`. Backend kiểm tra các trường này trước khi lưu vào MongoDB để hạn chế lỗi khi Frontend hiển thị.

Một số tiêu chí được dùng khi lựa chọn công nghệ AI:

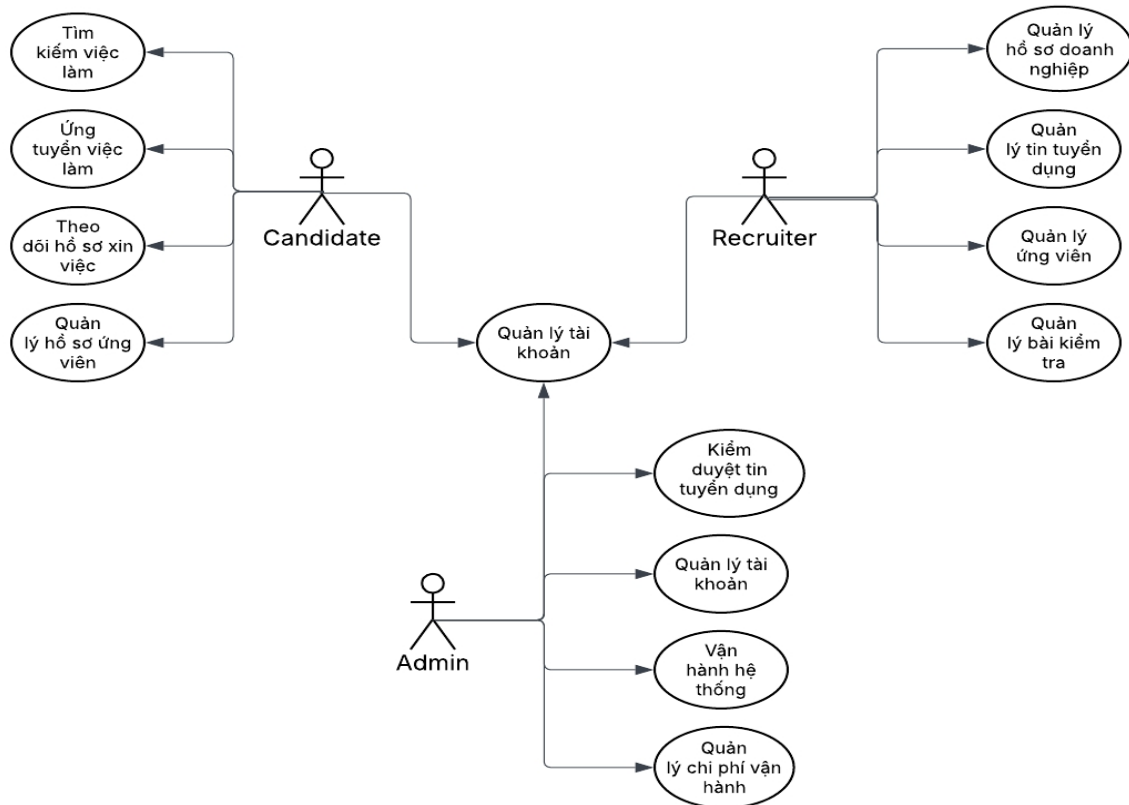
- Chất lượng phân tích: mô hình cần hiểu được CV, mô tả công việc và các kỹ năng liên quan.
- Tốc độ phản hồi: thời gian chấm điểm cần đủ nhanh để không làm gián đoạn trải nghiệm nộp đơn.
- Chi phí API: số lần gọi mô hình cần được kiểm soát, đặc biệt khi nhiều ứng viên nộp đơn cùng lúc.
- Khả năng tích hợp: service AI cần làm việc tốt với hệ sinh thái Java, RabbitMQ và MongoDB.
- Khả năng thay thế: hệ thống nên có cấu trúc để sau này đổi mô hình hoặc thêm provider khác nếu cần.

Dù LLM có nhiều ưu điểm, hệ thống vẫn cần giới hạn phạm vi sử dụng. AI không được xem là nguồn quyết định tuyệt đối. Điểm số chỉ là dữ liệu tham khảo cho bước sàng lọc ban đầu, còn nhà tuyển dụng vẫn chịu trách nhiệm đánh giá cuối cùng.

3.3. Thiết kế chức năng của hệ thống

3.3.1. Biểu đồ Use Case tổng quát

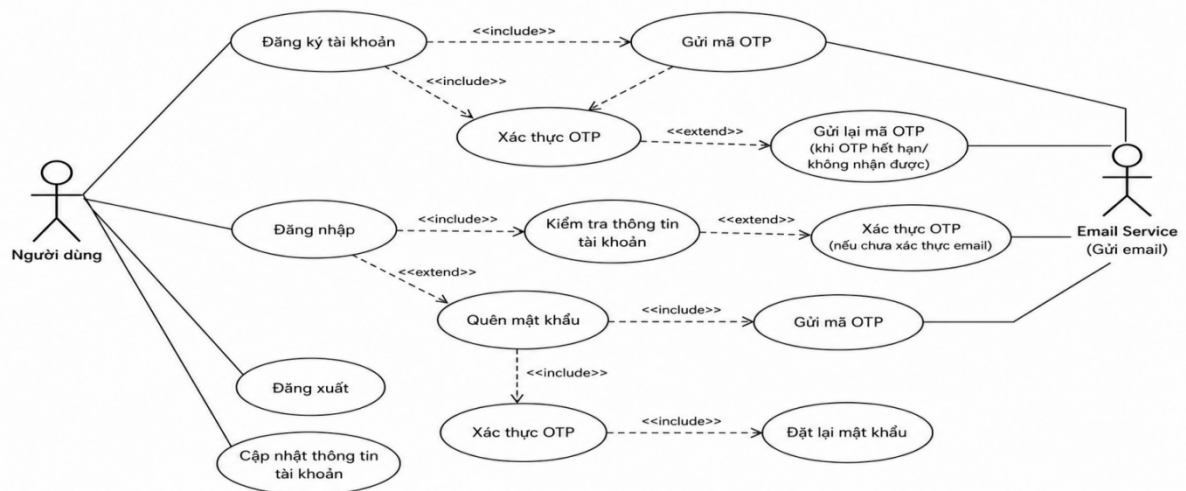
3.3.1.1. Biểu đồ Use Case tổng quát



Hình 3.1 Biểu đồ usecase tổng quát

Biểu đồ mô tả phạm vi chức năng của hệ thống và mối quan hệ giữa ba nhóm tác nhân chính gồm Ứng viên, Nhà tuyển dụng và Quản trị viên. Ứng viên thực hiện các chức năng quản lý hồ sơ, tìm kiếm và ứng tuyển việc làm; Nhà tuyển dụng quản lý tin tuyển dụng và hồ sơ ứng viên; Quản trị viên chịu trách nhiệm quản lý người dùng, kiểm duyệt nội dung và vận hành hệ thống. Ngoài ra, các dịch vụ bên ngoài như Email Service và AI Engine hỗ trợ xác thực, gửi thông báo và đánh giá CV.

3.3.1.2. Biểu đồ Use Case: Xác thực và đăng ký



Hình 3.2 Biểu đồ Use Case: Xác thực và đăng ký

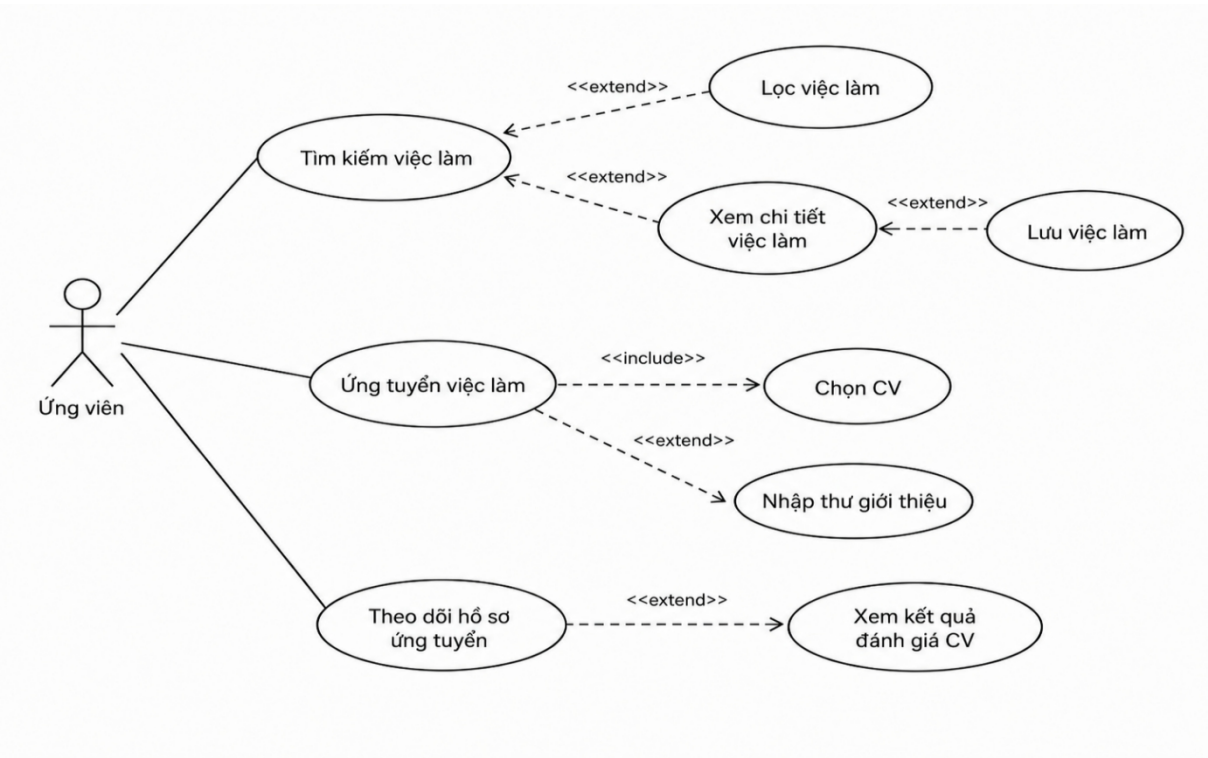
Biểu đồ thể hiện nhóm chức năng xác thực tài khoản của người dùng. Luồng đăng ký bao gồm nhập thông tin, gửi mã OTP qua Email Service và xác thực mã trước khi kích hoạt tài khoản. Khi đăng nhập, hệ thống kiểm tra thông tin tài khoản và trạng thái xác thực; trường hợp quên mật khẩu, người dùng phải xác thực OTP trước khi thiết lập mật khẩu mới. Người dùng cũng có thể đăng xuất và cập nhật thông tin tài khoản sau khi đăng nhập thành công.

Bảng 3.1 Bảng đặc tả Use Case đăng ký, xác thực OTP và đăng nhập

Mã Use Case	UC-AUTH
Tên Use Case	Đăng ký, xác thực OTP và đăng nhập
Tác nhân	Người dùng; Email Service
Mô tả	Cho phép người dùng tạo tài khoản, xác thực địa chỉ email bằng mã OTP và đăng nhập vào hệ thống bằng thông tin hợp lệ.
Điều kiện tiên quyết	Người dùng có địa chỉ email hợp lệ và có khả năng nhận email. Đối với đăng nhập, tài khoản đã được tạo và chưa bị khóa.
Điều kiện sau	Tài khoản được xác thực và kích hoạt sau khi OTP hợp lệ. Khi đăng nhập thành công, hệ thống cấp access token, refresh token và chuyển người dùng đến giao diện phù hợp với vai trò.

Luồng chính	1. Người dùng chọn chức năng đăng ký tài khoản. 2. Người dùng nhập thông tin bắt buộc và gửi yêu cầu đăng ký. 3. Hệ thống kiểm tra dữ liệu và xác nhận email chưa được sử dụng. 4. Hệ thống tạo tài khoản ở trạng thái chưa xác thực, sinh OTP và yêu cầu Email Service gửi mã. 5. Người dùng nhập OTP nhận được qua email. 6. Hệ thống kiểm tra OTP, cập nhật trạng thái xác thực và thông báo đăng ký thành công. 7. Người dùng nhập email và mật khẩu tại màn hình đăng nhập. 8. Hệ thống kiểm tra thông tin tài khoản, trạng thái xác thực và trạng thái hoạt động. 9. Hệ thống cấp token và cho phép truy cập theo vai trò.
Luồng thay thế/ngoại lệ	- A1. Email đã tồn tại: hệ thống từ chối đăng ký và yêu cầu sử dụng email khác. - A2. Dữ liệu không hợp lệ: hệ thống hiển thị lỗi tại trường tương ứng. - A3. OTP sai hoặc hết hạn: hệ thống thông báo lỗi và cho phép nhập lại hoặc gửi lại OTP. - A4. Email chưa xác thực khi đăng nhập: hệ thống chuyển người dùng đến bước xác thực OTP. - A5. Sai email hoặc mật khẩu: hệ thống từ chối đăng nhập nhưng không tiết lộ trường nào sai. - A6. Tài khoản bị khóa: hệ thống từ chối truy cập và hiển thị thông báo phù hợp.

3.3.1.3. Biểu đồ Use Case: Ứng tuyển và AI đánh giá CV



Hình 3.3 Biểu đồ Use Case: Ứng tuyển và AI đánh giá CV

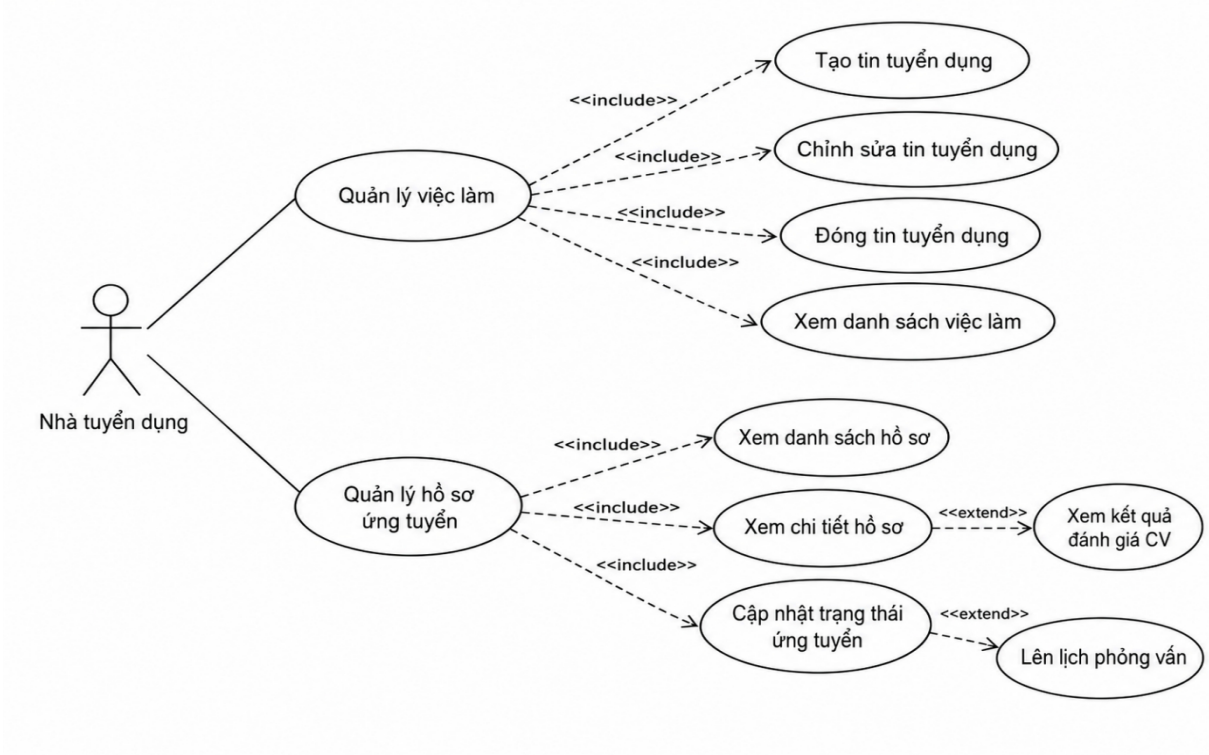
Biểu đồ mô tả các chức năng chính của ứng viên trong quá trình tìm việc và nộp hồ sơ. Ứng viên có thể tìm kiếm, lọc, xem chi tiết và lưu việc làm quan tâm. Khi ứng tuyển, người dùng chọn CV, có thể nhập thư giới thiệu và gửi hồ sơ đến nhà tuyển dụng. Sau đó, ứng viên theo dõi trạng thái hồ sơ và xem kết quả đánh giá CV do AI cung cấp.

Bảng 3.2 Bảng đặc tả Use Case ứng viên tìm việc và ứng tuyển

Mã Use Case	UC-CANDIDATE
Tên Use Case	Tìm kiếm việc làm và ứng tuyển
Tác nhân	Ứng viên
Mô tả	Cho phép ứng viên tìm kiếm, lọc và xem chi tiết việc làm; lưu việc làm; chọn CV để nộp đơn; theo dõi hồ sơ ứng tuyển và xem kết quả đánh giá CV.
Điều kiện tiên quyết	Ứng viên đã đăng nhập. Tin tuyển dụng còn hiệu lực. Ứng viên có ít nhất một CV hợp lệ khi thực hiện ứng tuyển.
Điều kiện sau	Đơn ứng tuyển được lưu, liên kết với tin tuyển dụng và CV đã chọn. Hệ thống tạo yêu cầu phân tích CV và ứng viên có thể theo dõi trạng thái xử lý.

Luồng chính	1. Ứng viên nhập từ khóa hoặc mở danh sách việc làm. 2. Hệ thống hiển thị các tin tuyển dụng phù hợp. 3. Ứng viên sử dụng bộ lọc theo địa điểm, mức lương, loại việc làm hoặc kỹ năng khi cần. 4. Ứng viên mở chi tiết một tin tuyển dụng. 5. Ứng viên chọn ứng tuyển việc làm. 6. Hệ thống hiển thị danh sách CV của ứng viên. 7. Ứng viên chọn CV và có thể nhập thư giới thiệu. 8. Hệ thống kiểm tra điều kiện ứng tuyển và lưu đơn ứng tuyển. 9. Hệ thống thông báo ứng tuyển thành công và khởi tạo quá trình đánh giá CV. 10. Ứng viên mở danh sách hồ sơ ứng tuyển để theo dõi trạng thái và xem kết quả AI khi có.
Luồng thay thế/ngoại lệ	B1. Không tìm thấy việc làm phù hợp: hệ thống hiển thị danh sách rỗng và cho phép thay đổi từ khóa hoặc bộ lọc. B2. Ứng viên chưa có CV: hệ thống yêu cầu tải CV trước khi ứng tuyển. B3. Tin đã đóng hoặc hết hạn: hệ thống không cho phép nộp đơn. B4. Ứng viên đã ứng tuyển cùng tin: hệ thống cảnh báo và không tạo đơn trùng. B5. CV không hợp lệ hoặc tải lên thất bại: hệ thống yêu cầu chọn CV khác. B6. AI chưa xử lý xong hoặc xảy ra lỗi: đơn vẫn được ghi nhận và trạng thái đánh giá được cập nhật sau.

3.3.1.4. Biểu đồ Use Case: Quản lý tuyển dụng và ứng viên



Hình 3.4 Biểu đồ Use Case: Quản lý tuyển dụng và ứng viên

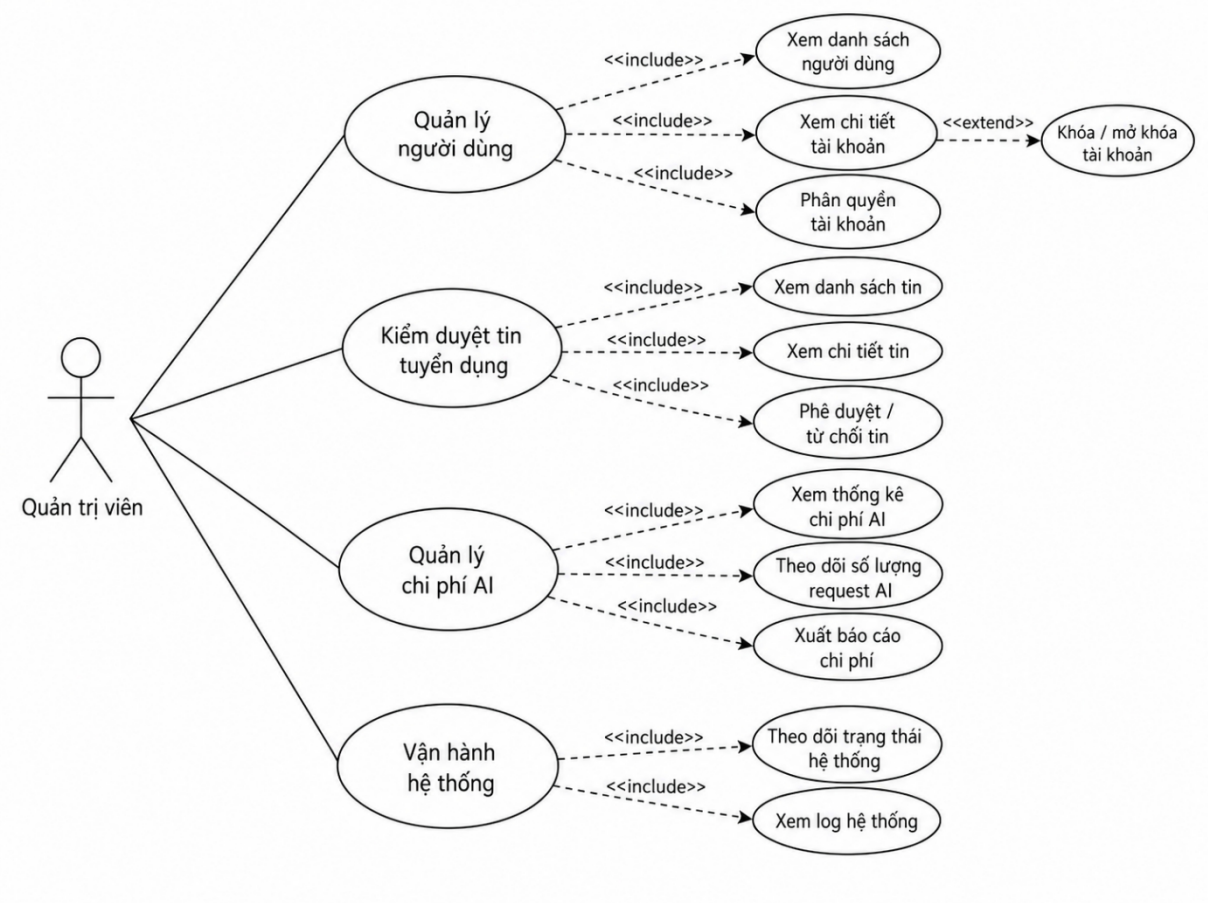
Biểu đồ mô tả hai nhóm nghiệp vụ chính của Nhà tuyển dụng. Nhóm quản lý việc làm cho phép tạo mới, chỉnh sửa, đóng và xem danh sách tin tuyển dụng. Nhóm quản lý hồ sơ ứng tuyển cho phép xem danh sách, xem chi tiết hồ sơ, theo dõi kết quả AI đánh giá CV, cập nhật trạng thái ứng tuyển và lên lịch phỏng vấn khi hồ sơ đạt yêu cầu.

Bảng 3.3 Bảng đặc tả Use Case nhà tuyển dụng quản lý việc làm và hồ sơ ứng tuyển

Mã Use Case	UC-RECRUITER
Tên Use Case	Quản lý việc làm và hồ sơ ứng tuyển
Tác nhân	Nhà tuyển dụng
Mô tả	Cho phép nhà tuyển dụng tạo, chỉnh sửa, đóng và xem danh sách tin tuyển dụng; đồng thời xem hồ sơ ứng viên, kết quả đánh giá CV, cập nhật trạng thái và lên lịch phỏng vấn.
Điều kiện tiên quyết	Nhà tuyển dụng đã đăng nhập, tài khoản đang hoạt động và hồ sơ doanh nghiệp đã được phê duyệt. Khi tạo tin mới, tài khoản còn quota đăng tin.
Điều kiện sau	Tin tuyển dụng được tạo hoặc cập nhật đúng trạng thái. Hồ sơ ứng tuyển được cập nhật trong quy trình ATS và các

	thông báo liên quan được phát sinh khi cần.
Luồng chính	1. Nhà tuyển dụng mở chức năng quản lý việc làm. 2. Hệ thống hiển thị danh sách các tin thuộc doanh nghiệp. 3. Nhà tuyển dụng tạo tin mới hoặc chọn một tin để chỉnh sửa hay đóng. 4. Hệ thống kiểm tra dữ liệu, quyền sở hữu, trạng thái doanh nghiệp và quota trước khi lưu. 5. Nhà tuyển dụng mở chức năng quản lý hồ sơ ứng tuyển của một tin. 6. Hệ thống hiển thị danh sách ứng viên đã nộp hồ sơ. 7. Nhà tuyển dụng xem chi tiết hồ sơ và kết quả đánh giá CV. 8. Nhà tuyển dụng cập nhật trạng thái ứng viên trong quy trình tuyển dụng. 9. Khi cần, nhà tuyển dụng thiết lập lịch phỏng vấn. 10. Hệ thống lưu thay đổi và gửi thông báo đến ứng viên.
Luồng thay thế/ngoại lệ	C1. Hồ sơ doanh nghiệp chưa được duyệt: hệ thống không cho phép tạo hoặc công khai tin. C2. Hết quota đăng tin: hệ thống yêu cầu mua thêm gói hoặc chờ được cấp quota. C3. Dữ liệu tin không hợp lệ: hệ thống hiển thị lỗi và không lưu. C4. Tin không thuộc nhà tuyển dụng: hệ thống từ chối truy cập. C5. Hồ sơ ứng tuyển đã thay đổi bởi người dùng khác: hệ thống yêu cầu tải lại dữ liệu trước khi cập nhật. C6. Kết quả AI chưa có: hệ thống vẫn hiển thị hồ sơ và trạng thái đang xử lý. C7. Lịch phỏng vấn không hợp lệ: hệ thống yêu cầu chọn lại thời gian.

3.3.1.5. Biểu đồ Use Case: Quản trị viên vận hành hệ thống



Hình 3.5 Biểu đồ Use Case: Quản trị viên vận hành hệ thống

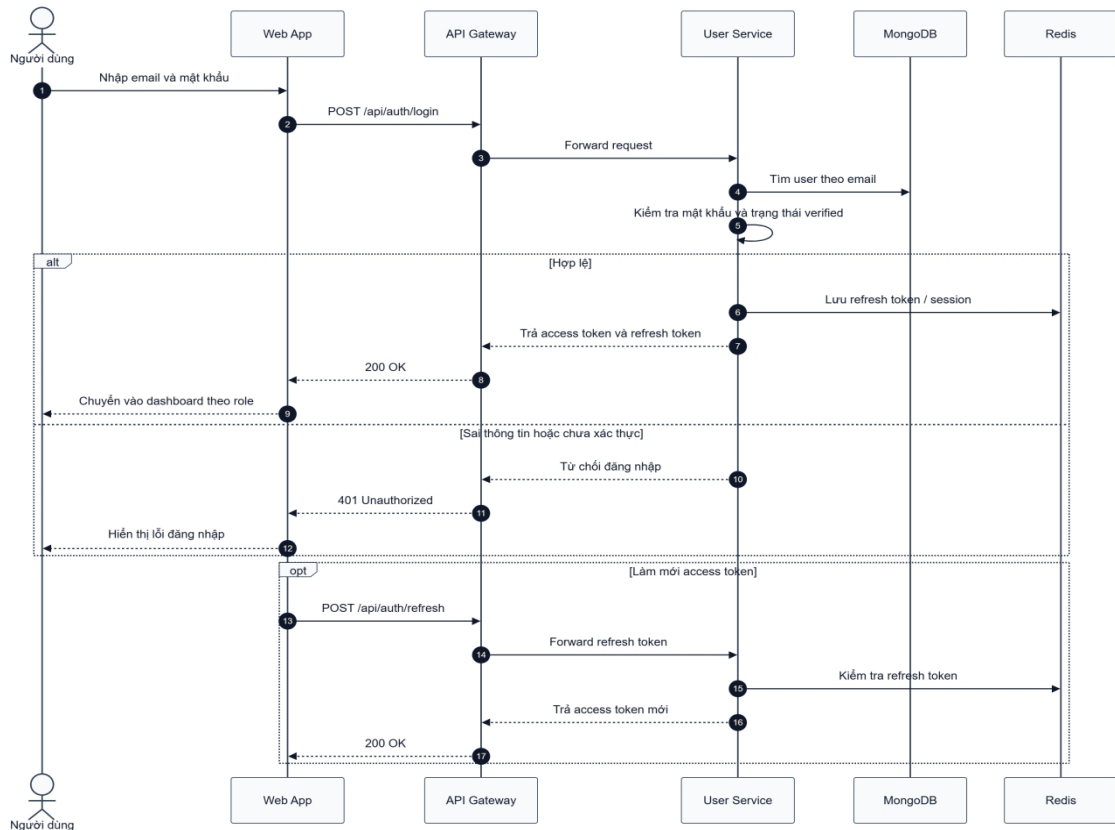
Biểu đồ thể hiện phạm vi quản trị của Quản trị viên, bao gồm quản lý người dùng, kiểm duyệt tin tuyển dụng, theo dõi chi phí AI và vận hành hệ thống. Quản trị viên có thể xem thông tin tài khoản, phân quyền, khóa hoặc mở khóa người dùng; xem và phê duyệt hoặc từ chối tin tuyển dụng; theo dõi số lượng yêu cầu AI, xuất báo cáo chi phí, kiểm tra trạng thái và xem nhật ký hệ thống.

Bảng 3.4 Bảng đặc tả Use Case quản trị viên quản lý hệ thống

Mã Use Case	UC-ADMIN
Tên Use Case	Quản lý và vận hành hệ thống
Tác nhân	Quản trị viên
Mô tả	Cho phép quản trị viên quản lý tài khoản người dùng, kiểm duyệt tin tuyển dụng, theo dõi chi phí và số lượng yêu cầu AI, xuất báo cáo, giám sát trạng thái dịch vụ và xem nhật ký hệ thống.
Điều kiện tiên	Quản trị viên đã đăng nhập bằng tài khoản có vai trò

quyết	ADMIN và tài khoản đang hoạt động.
Điều kiện sau	Thay đổi quản trị được lưu và ghi nhận nhật ký. Tin tuyển dụng hoặc tài khoản được cập nhật đúng trạng thái. Thông tin chi phí và vận hành được tổng hợp để theo dõi.
Luồng chính	1. Quản trị viên truy cập trang quản trị. 2. Hệ thống kiểm tra quyền ADMIN và hiển thị bảng điều khiển. 3. Quản trị viên xem danh sách người dùng, mở chi tiết tài khoản và thực hiện khóa, mở khóa hoặc phân quyền theo chính sách. 4. Quản trị viên mở danh sách tin chờ kiểm duyệt. 5. Quản trị viên xem chi tiết và phê duyệt hoặc từ chối tin tuyển dụng. 6. Quản trị viên mở chức năng quản lý chi phí AI để xem thống kê chi phí và số lượng request. 7. Quản trị viên xuất báo cáo chi phí khi cần. 8. Quản trị viên theo dõi trạng thái các service và xem log hệ thống. 9. Hệ thống lưu các thao tác quản trị và cập nhật dữ liệu thống kê.
Luồng thay thế/ngoại lệ	D1. Tài khoản không có quyền ADMIN: hệ thống từ chối truy cập. D2. Không tìm thấy người dùng hoặc tin tuyển dụng: hệ thống thông báo dữ liệu không tồn tại. D3. Không thể khóa tài khoản quản trị đang đăng nhập hoặc tài khoản được bảo vệ: hệ thống từ chối thao tác. D4. Từ chối tin nhưng chưa nhập lý do: hệ thống yêu cầu bổ sung lý do. D5. Dịch vụ AI hoặc hệ thống giám sát không phản hồi: hệ thống hiển thị trạng thái lỗi và ghi log. D6. Không có dữ liệu trong khoảng thời gian báo cáo: hệ thống trả về báo cáo rỗng kèm thông báo.

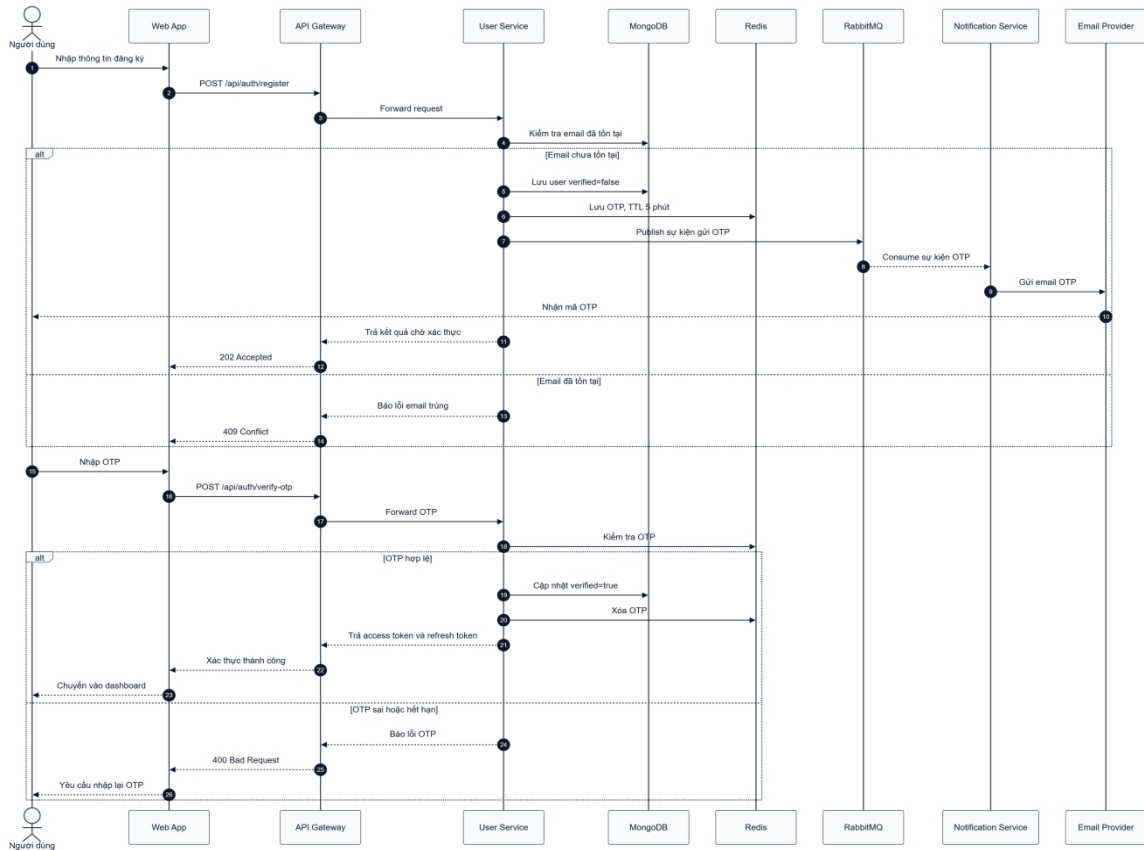
3.3.1.6. Biểu đồ tuần tự: Đăng nhập



Hình 3. 1 Biểu đồ tuần tự đăng nhập

Biểu đồ tuần tự mô tả quá trình đăng nhập từ khi người dùng nhập email và mật khẩu trên Web App. Yêu cầu được gửi qua API Gateway đến User Service để truy vấn tài khoản trong MongoDB, kiểm tra mật khẩu và trạng thái xác thực. Nếu thông tin hợp lệ, hệ thống lưu refresh token hoặc phiên làm việc vào Redis, trả access token và refresh token để Web App chuyển người dùng đến giao diện phù hợp với vai trò. Nếu thông tin không hợp lệ hoặc tài khoản chưa được xác thực, hệ thống trả lỗi 401 Unauthorized.

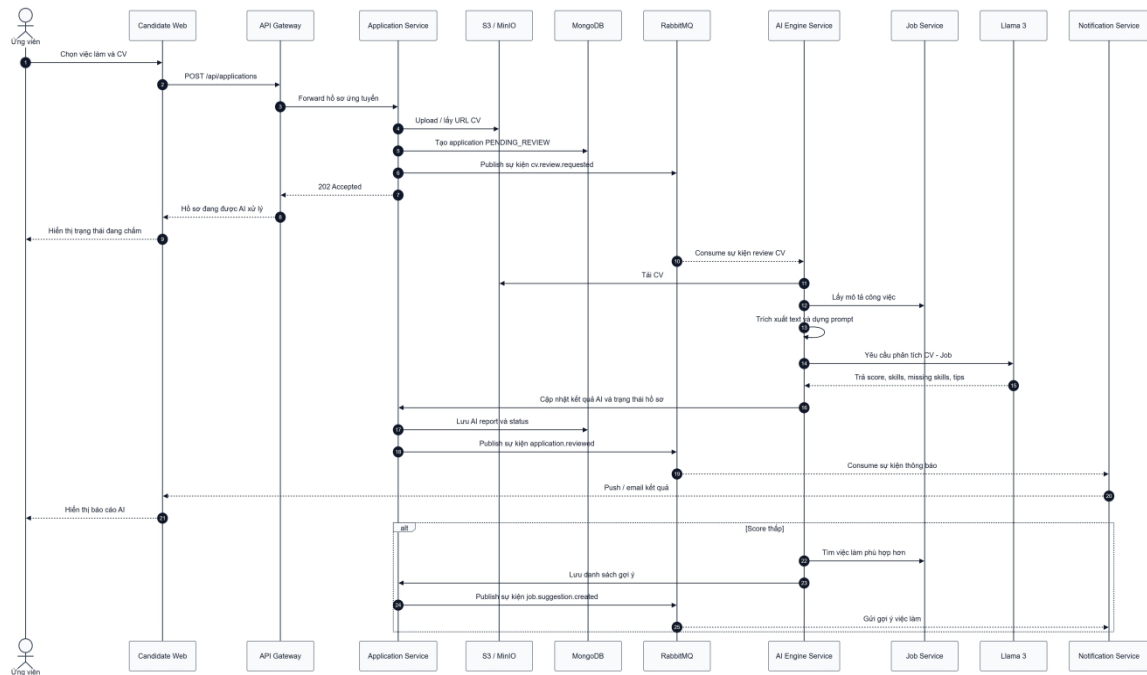
3.3.1.7. Biểu đồ tuần tự: Đăng ký và xác thực OTP



Hình 3. 2 Biểu đồ tuần tự đăng ký và xác thực OTP

Biểu đồ tuần tự mô tả quá trình tạo tài khoản và xác thực email bằng OTP. Web App gửi thông tin đăng ký qua API Gateway đến User Service; hệ thống kiểm tra dữ liệu, mã hóa mật khẩu, lưu tài khoản ở trạng thái chưa xác thực và tạo OTP có thời hạn trong Redis. Notification Service gửi mã đến email người dùng. Khi người dùng nhập OTP, User Service đối chiếu mã và cập nhật trạng thái tài khoản thành đã xác thực nếu mã hợp lệ.

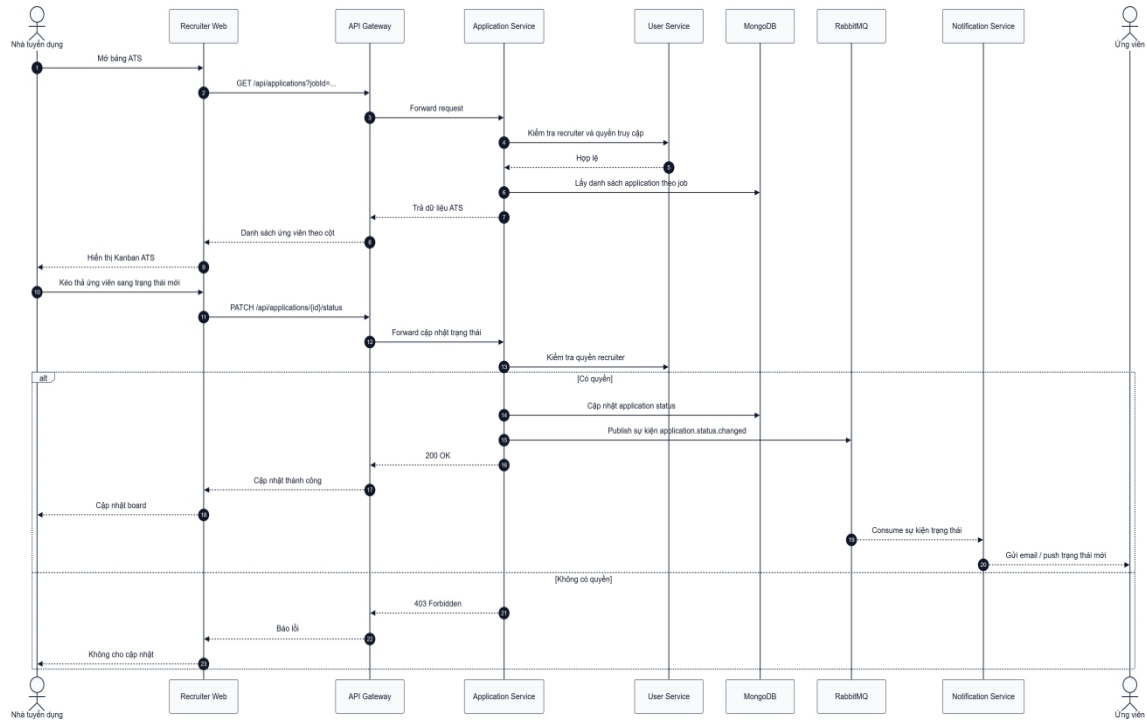
3.3.1.8. Biểu đồ tuần tự: Ứng tuyển và AI chấm điểm CV



Hình 3. 3 Biểu đồ tuần tự ứng viên ứng tuyển việc làm và AI chấm điểm CV

Biểu đồ tuần tự mô tả luồng xử lý từ lúc ứng viên gửi hồ sơ đến khi nhận kết quả đánh giá. Candidate Web gửi yêu cầu ứng tuyển qua API Gateway đến Application Service, nơi CV được lưu trữ, hồ sơ ứng tuyển được tạo trong MongoDB và sự kiện đánh giá CV được phát lên RabbitMQ. AI Engine Service nhận sự kiện, lấy nội dung công việc từ Job Service, gửi yêu cầu phân tích đến mô hình Llama 3 và lưu kết quả chấm điểm. Sau cùng, Notification Service gửi thông báo kết quả cho ứng viên; trong trường hợp điểm phù hợp thấp, hệ thống có thể tiếp tục gợi ý các việc làm phù hợp hơn.

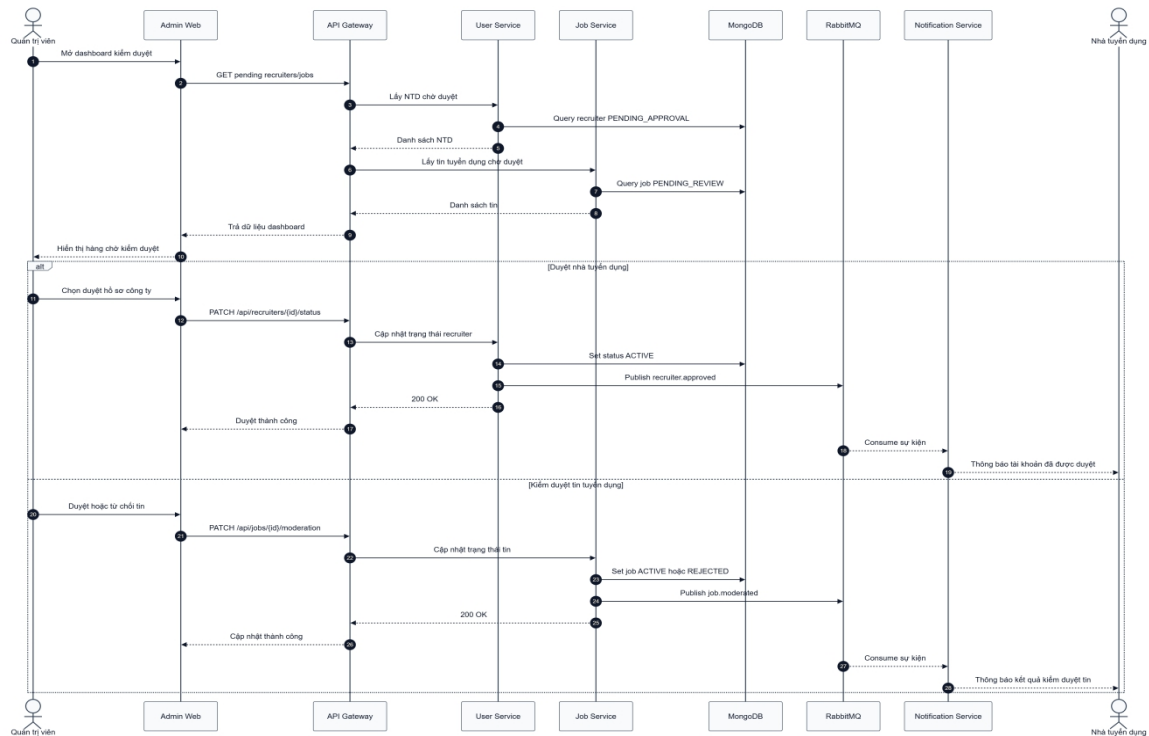
3.3.1.9. Biểu đồ tuần tự: Quản lý ứng viên và tuyển dụng



Hình 3. 4 Biểu đồ tuần tự quản lý ứng viên và tuyển dụng

Biểu đồ tuần tự mô tả cách Nhà tuyển dụng xem và cập nhật hồ sơ ứng viên. Recruiter Web yêu cầu danh sách hồ sơ theo tin tuyển dụng qua API Gateway; Application Service kiểm tra quyền sở hữu tin, truy vấn dữ liệu và trả danh sách để hiển thị trên bảng ATS. Khi Nhà tuyển dụng thay đổi trạng thái hồ sơ, hệ thống kiểm tra quyền, cập nhật dữ liệu trong MongoDB, phát sự kiện qua RabbitMQ và Notification Service gửi email hoặc thông báo đẩy đến ứng viên. Nếu người thao tác không có quyền, hệ thống trả lỗi 403 Forbidden.

3.3.1.10. Biểu đồ tuần tự: Quản trị hệ thống

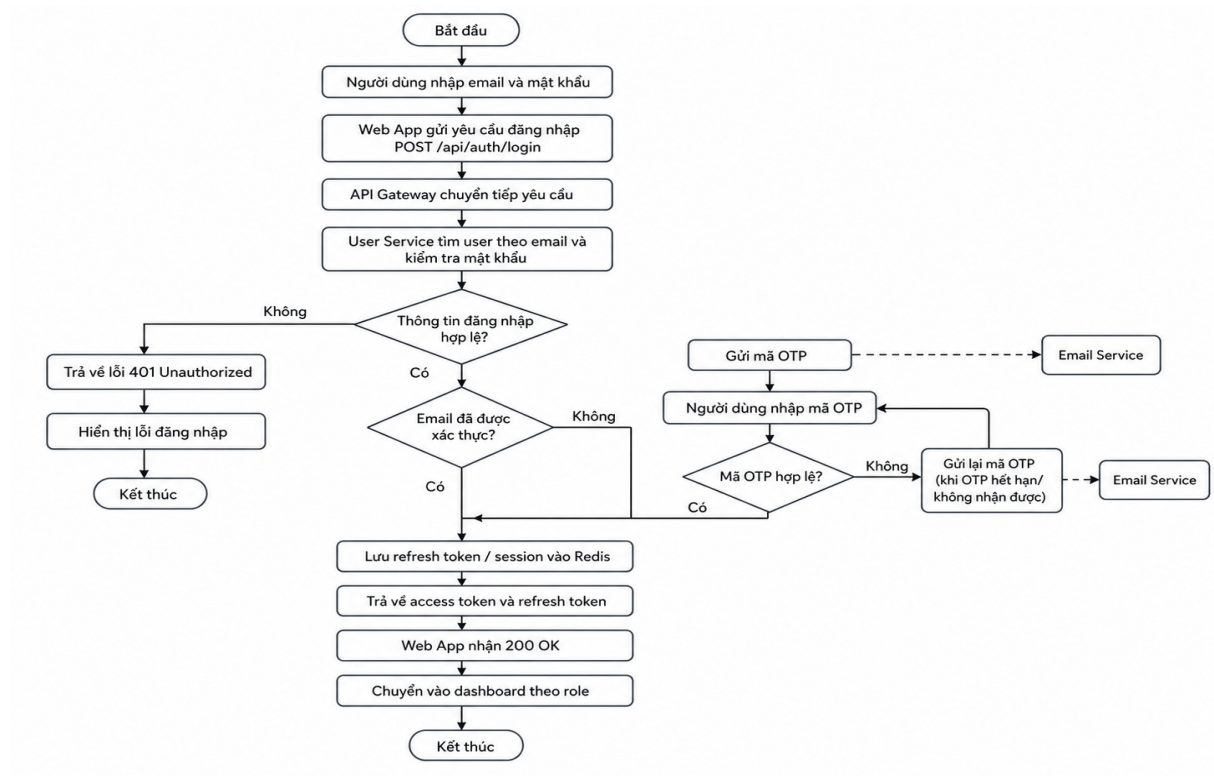


Hình 3. 5 Quản trị viên quản trị hệ thống

Biểu đồ tuần tự mô tả quá trình Quản trị viên kiểm duyệt Nhà tuyển dụng và tin tuyển dụng. Admin Web lấy danh sách đối tượng đang chờ duyệt qua API Gateway, sau đó User Service hoặc Job Service truy vấn dữ liệu tương ứng. Khi Quản trị viên phê duyệt hoặc từ chối, service phụ trách cập nhật trạng thái trong MongoDB, phát sự kiện lên RabbitMQ và Notification Service gửi kết quả kiểm duyệt đến Nhà tuyển dụng.

3.3.2. Biểu đồ luồng (DFD)

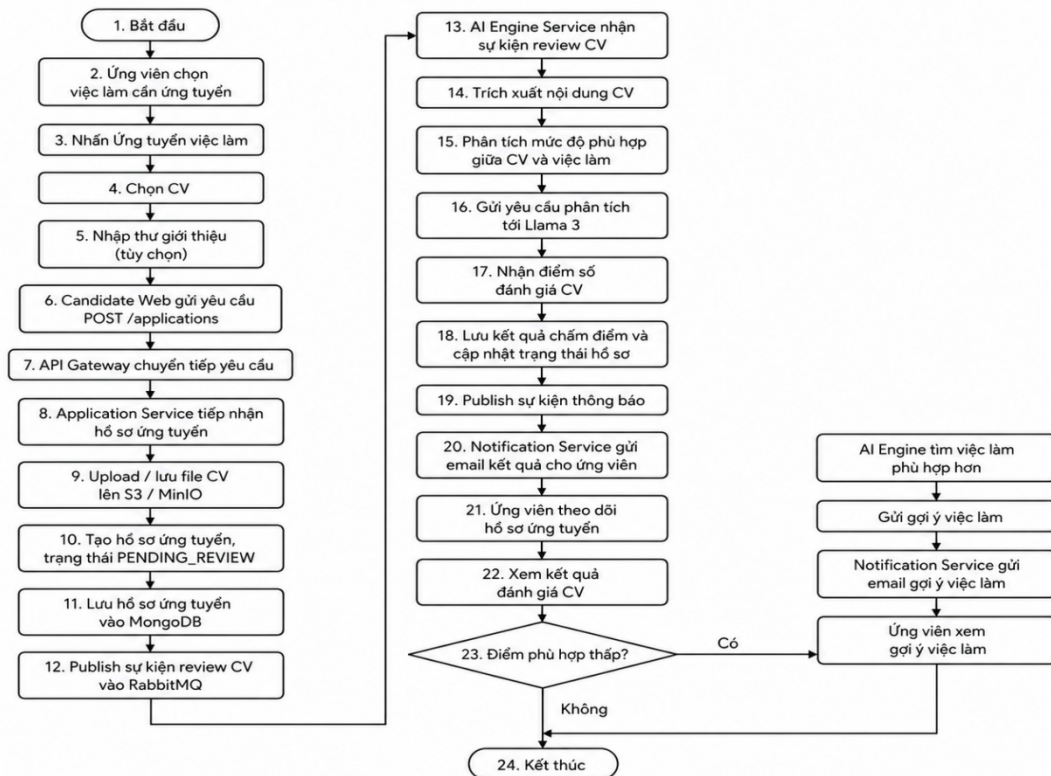
3.3.2.1. Biểu đồ hoạt động: Đăng nhập



Hình 3.6 Biểu đồ luồng đăng nhập

Biểu đồ luồng thể hiện toàn bộ quá trình xử lý đăng nhập. Sau khi người dùng nhập email và mật khẩu, Web App gửi yêu cầu qua API Gateway đến User Service để kiểm tra tài khoản. Nếu thông tin không hợp lệ, hệ thống trả lỗi và kết thúc luồng. Nếu tài khoản hợp lệ nhưng email chưa được xác thực, hệ thống gửi OTP và yêu cầu người dùng xác thực trước khi tiếp tục. Khi các điều kiện đều đạt, refresh token được lưu vào Redis, access token và refresh token được trả về để chuyển người dùng đến dashboard theo vai trò.

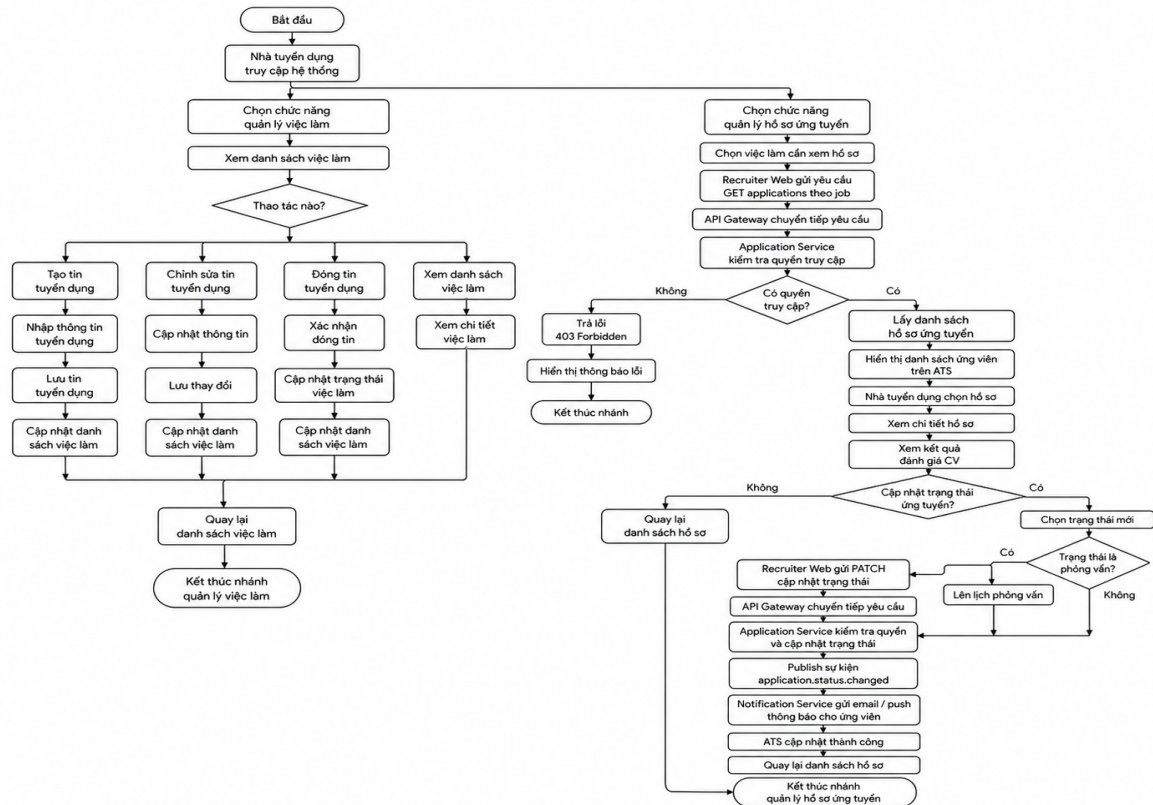
3.3.2.2. Biểu đồ hoạt động: Ứng viên ứng tuyển việc làm và AI chấm điểm CV



Hình 3.7 Biểu đồ luồng Ứng tuyển việc làm và AI chấm điểm CV

Biểu đồ luồng mô tả các bước từ lúc ứng viên lựa chọn việc làm đến khi nhận kết quả phân tích CV. Ứng viên chọn CV, nhập thư giới thiệu nếu cần và gửi yêu cầu ứng tuyển. Application Service lưu file, tạo hồ sơ ở trạng thái chờ đánh giá và phát sự kiện lên RabbitMQ. AI Engine trích xuất nội dung CV, so sánh với yêu cầu công việc, gọi mô hình ngôn ngữ để nhận điểm số và lưu kết quả. Ứng viên có thể theo dõi hồ sơ, xem đánh giá và nhận gợi ý việc làm khác khi mức độ phù hợp thấp.

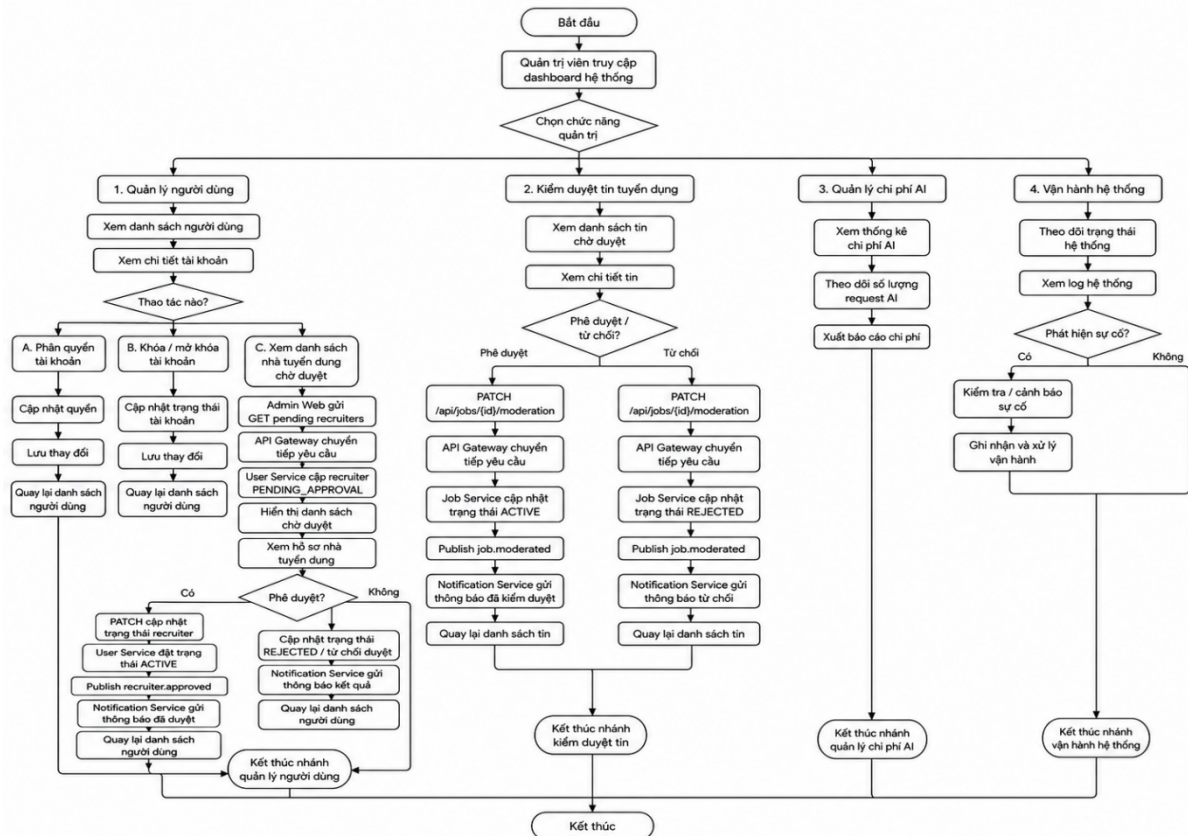
3.3.2.3. Biểu đồ hoạt động: Nhà tuyển dụng quản lý việc làm và hồ sơ ứng tuyển



Hình 3.8 Biểu đồ luồng Quản lý việc làm và hồ sơ ứng tuyển

Biểu đồ luồng trình bày hai nhánh nghiệp vụ của Nhà tuyển dụng. Với quản lý việc làm, người dùng có thể tạo, chỉnh sửa, đóng hoặc xem chi tiết tin tuyển dụng và danh sách được cập nhật sau mỗi thao tác. Với quản lý hồ sơ, Nhà tuyển dụng chọn tin cần xem, hệ thống kiểm tra quyền truy cập rồi hiển thị danh sách ứng viên trên ATS. Nhà tuyển dụng có thể xem chi tiết, xem điểm AI, cập nhật trạng thái và lên lịch phỏng vấn; mỗi thay đổi được phát thành sự kiện để gửi thông báo cho ứng viên.

3.3.2.4. Biểu đồ hoạt động: Quản trị viên quản lý hệ thống

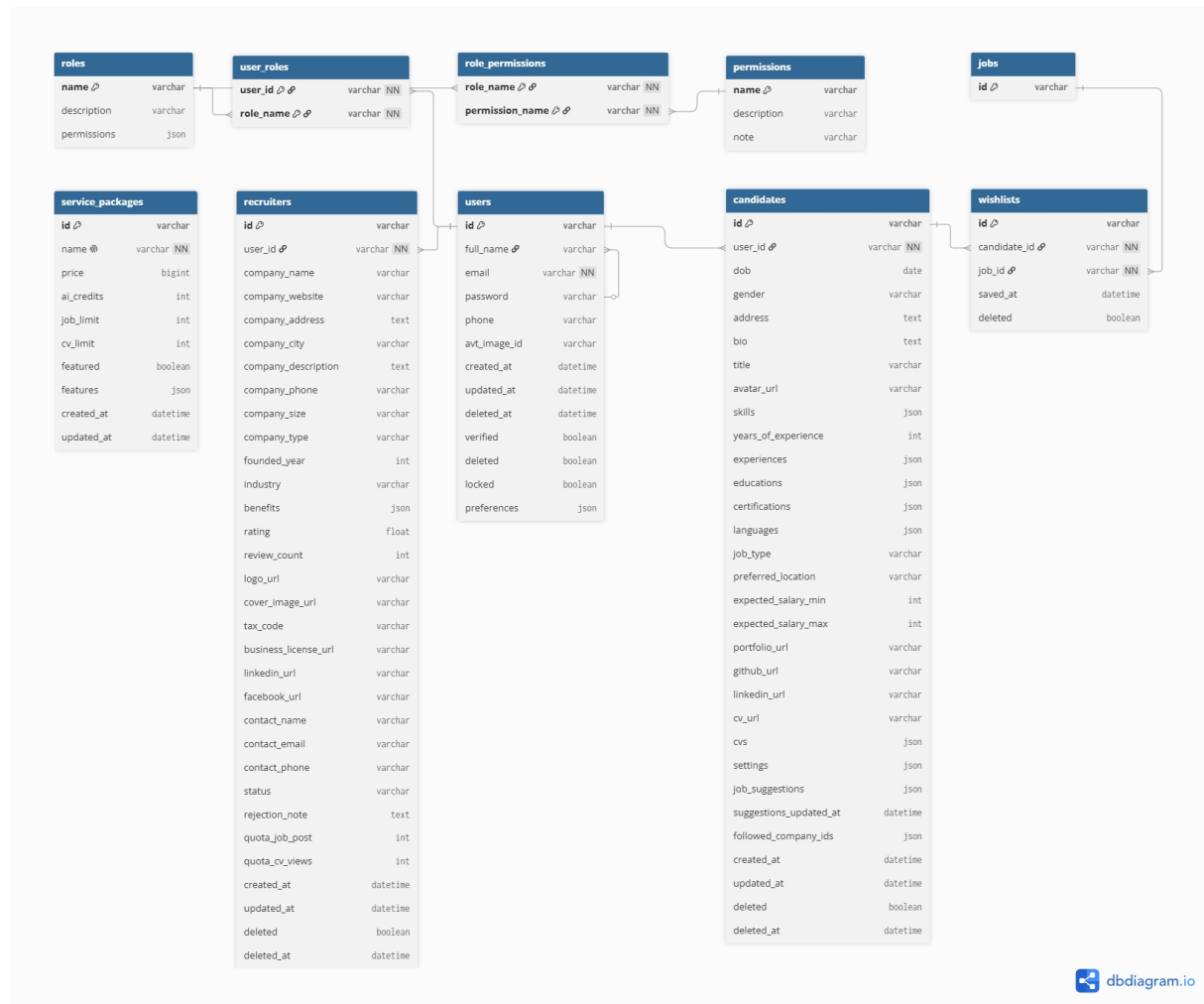


Hình 3.9 Biểu đồ luồng Quản lý hệ thống

Biểu đồ luồng mô tả bốn nhóm chức năng quản trị gồm quản lý người dùng, kiểm duyệt tin tuyển dụng, quản lý chi phí AI và vận hành hệ thống. Tùy chức năng được chọn, Quản trị viên có thể phân quyền, khóa tài khoản, duyệt hồ sơ Nhà tuyển dụng, phê duyệt hoặc từ chối tin tuyển dụng, theo dõi lượng yêu cầu AI, xuất báo cáo chi phí và xử lý sự cố vận hành. Các thao tác kiểm duyệt quan trọng đều cập nhật trạng thái dữ liệu và gửi thông báo đến đối tượng liên quan.

3.4. Biểu đồ lớp (Class Diagrams)

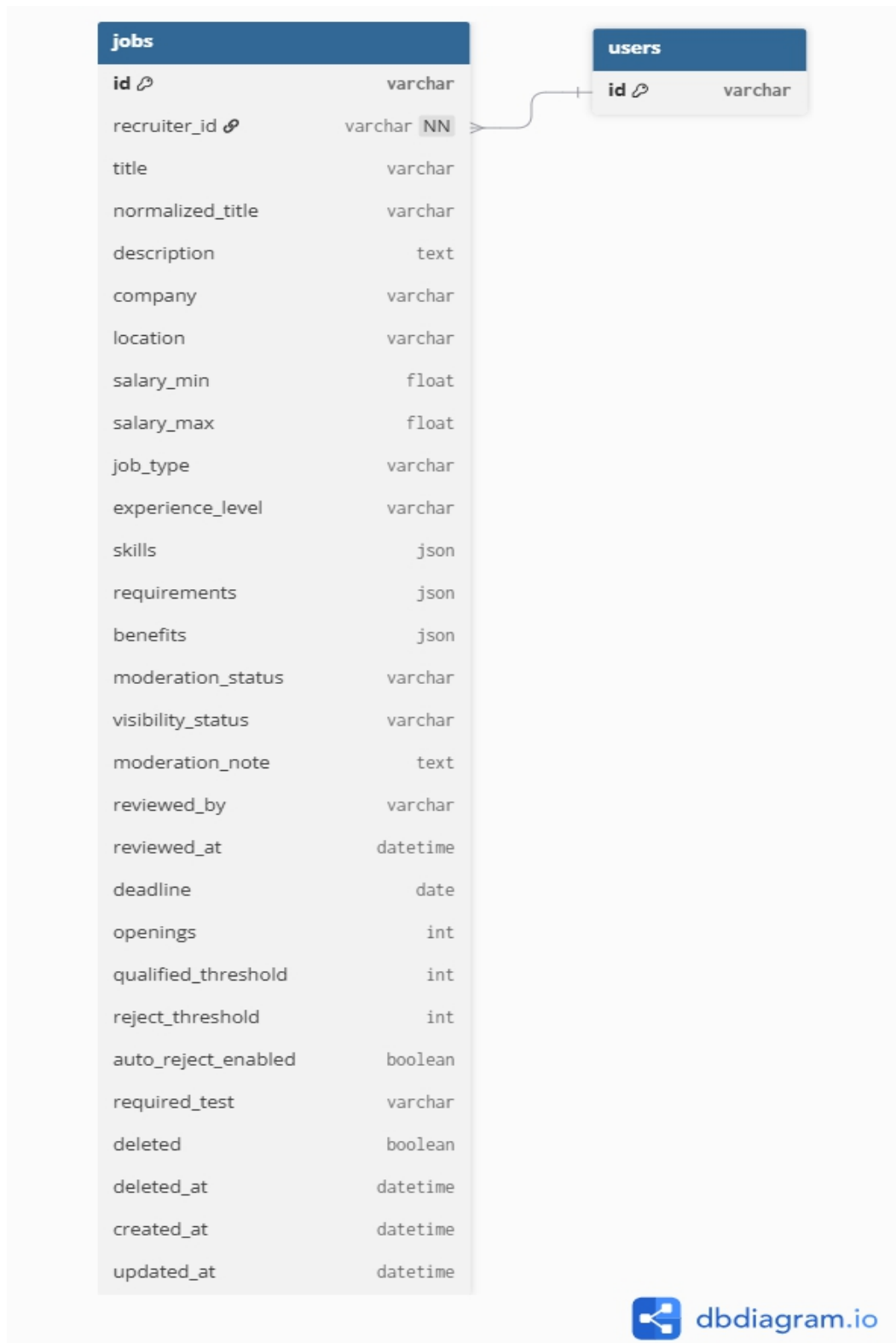
3.4.1. Biểu đồ lớp User Service Database



Hình 3.10 Biểu đồ lớp User Service Database

Biểu đồ mô tả cấu trúc dữ liệu thuộc User Service và quan hệ giữa các collection chính. Collection users lưu tài khoản và trạng thái xác thực; roles, permissions cùng các bảng liên kết hỗ trợ phân quyền RBAC. Candidates và recruiters mở rộng thông tin hồ sơ theo từng vai trò, trong khi wishlists lưu các việc làm được ứng viên quan tâm. Service packages lưu thông tin gói dịch vụ dành cho Nhà tuyển dụng, còn jobs được thể hiện như một thực thể tham chiếu thuộc Job Service.

3.4.2. Biểu đồ lớp Job Service Database

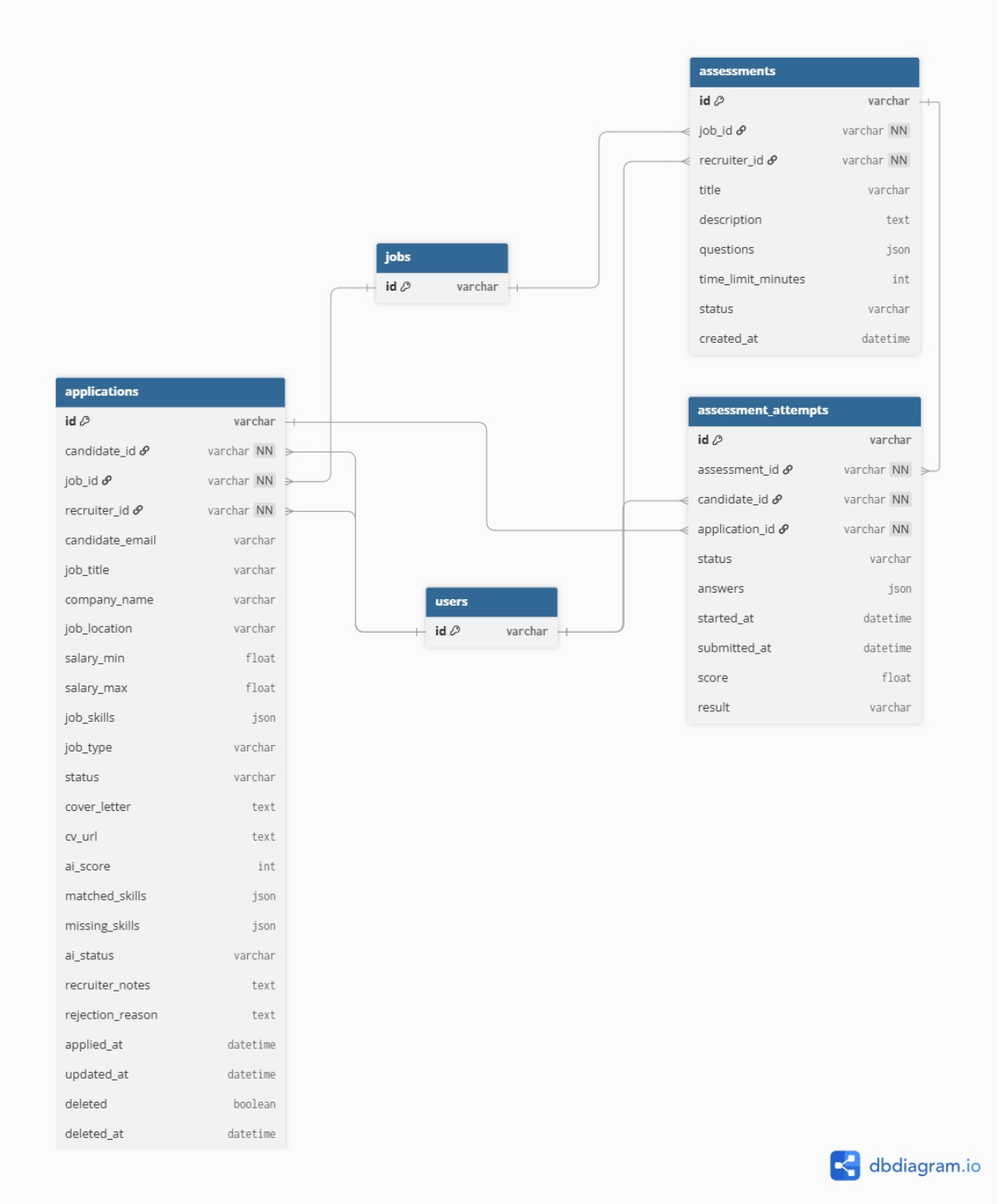


Hình 3.11 Biểu đồ lớp Job Service Database

Biểu đồ mô tả các collection phục vụ quản lý tin tuyển dụng. Thực thể jobs lưu thông tin công việc, yêu cầu kỹ năng, mức lương, trạng thái kiểm duyệt và thông tin Nhà tuyển dụng. Các cấu trúc dữ liệu liên quan hỗ trợ phân loại ngành nghề, địa điểm,

hình thức làm việc và đồng bộ dữ liệu tìm kiếm. Quan hệ giữa các thành phần giúp Job Service quản lý vòng đời tin tuyển dụng từ lúc tạo mới, chờ duyệt, hoạt động đến khi đóng hoặc bị từ chối.

3.4.3. Biểu đồ lớp Application Service Database

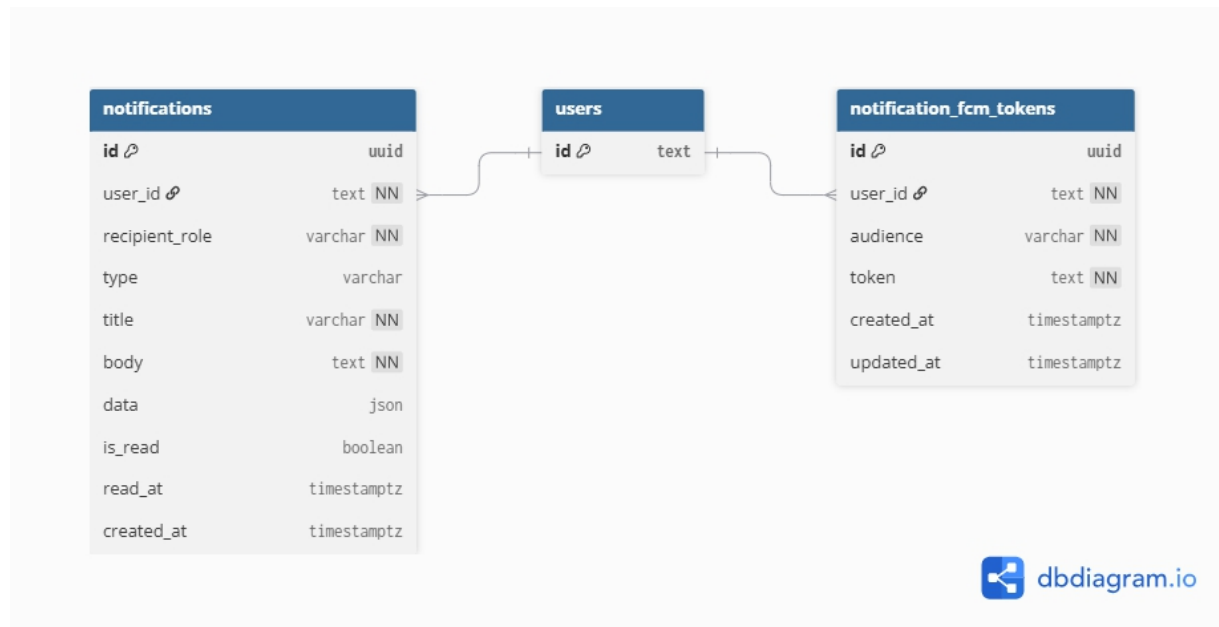


Hình 3.12 Biểu đồ lớp Application Service Database

Biểu đồ mô tả dữ liệu nghiệp vụ của Application Service. Collection applications lưu đơn ứng tuyển, liên kết ứng viên, tin tuyển dụng, CV được sử dụng và

trạng thái xử lý trên ATS. Kết quả phân tích AI, lịch sử thay đổi trạng thái và các lần đánh giá có thể được lưu dưới dạng document nhúng hoặc collection riêng tùy yêu cầu truy vấn. Cấu trúc này hỗ trợ theo dõi xuyên suốt quá trình từ lúc nộp hồ sơ đến khi phỏng vấn, chấp nhận hoặc từ chối.

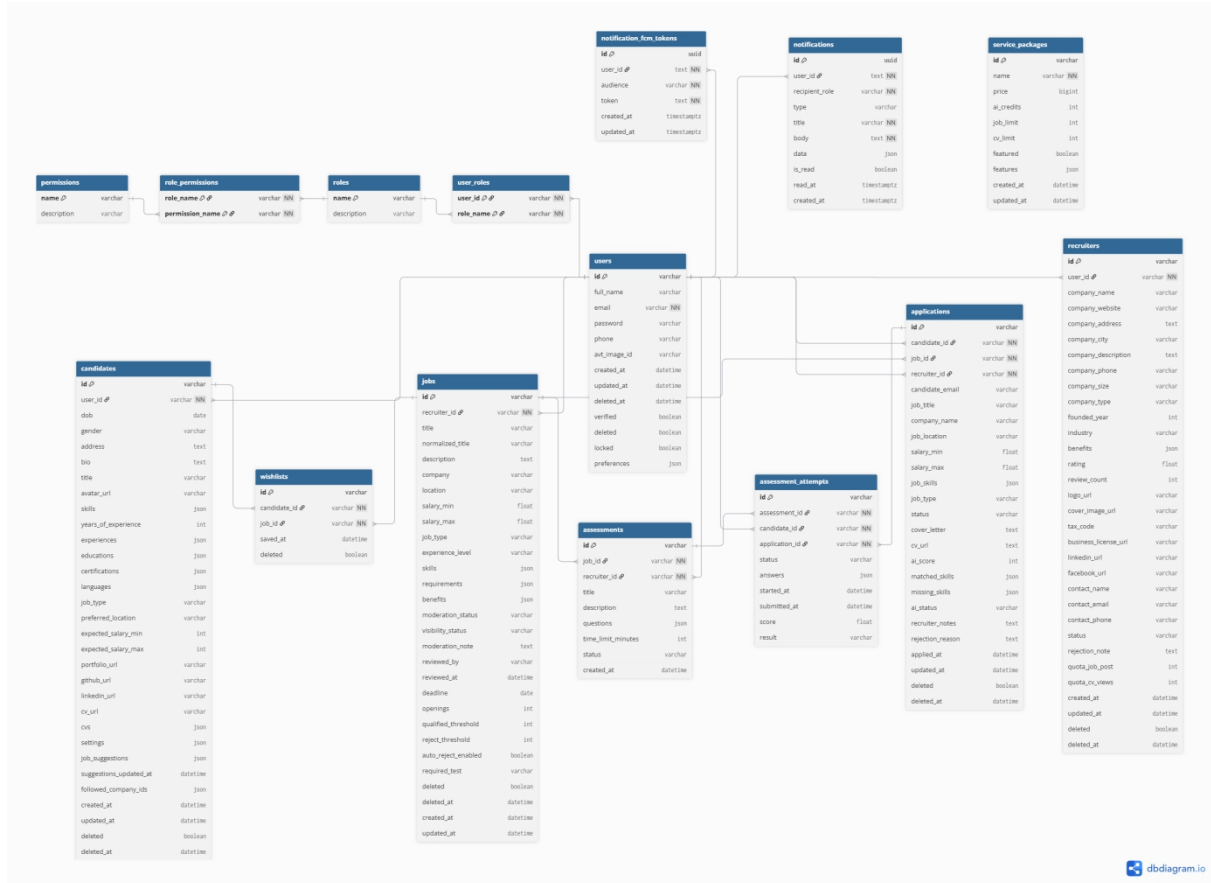
3.4.4. Biểu đồ lớp Notification Service Database



Hình 3.13 Biểu đồ lớp Notification Service Database

Biểu đồ mô tả cấu trúc dữ liệu của Notification Service. Bảng `notifications` lưu nội dung, loại thông báo, người nhận, trạng thái đọc, kênh gửi và thời điểm tạo. Bảng `notification_fcm_tokens` quản lý token thiết bị để gửi thông báo đẩy và liên kết token với người dùng. Thiết kế này cho phép service lưu lịch sử thông báo, gửi lại khi cần và hỗ trợ nhiều kênh như email, FCM hoặc thông báo trong ứng dụng.

3.4.5. Biểu đồ lớp Tổng quát



Hình 3.14 Biểu đồ lớp Tổng quát

Biểu đồ lớp tổng quát thể hiện mối liên hệ dữ liệu giữa các service trong toàn hệ thống SmartCV. User Service quản lý người dùng, hồ sơ ứng viên và Nhà tuyển dụng; Job Service quản lý tin tuyển dụng; Application Service quản lý đơn ứng tuyển và kết quả AI; Notification Service lưu thông báo và token thiết bị. Các quan hệ giữa service được biểu diễn ở mức tham chiếu định danh, phù hợp với nguyên tắc mỗi microservice sở hữu cơ sở dữ liệu riêng và trao đổi dữ liệu thông qua API hoặc sự kiện.

3.5. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được thiết kế theo hướng phục vụ kiến trúc microservices. Dữ liệu chính được lưu trong MongoDB vì các thực thể như hồ sơ ứng viên, tin tuyển dụng và kết quả AI có cấu trúc linh hoạt. Bên cạnh đó, Redis và Elasticsearch được dùng cho các nhu cầu chuyên biệt.

Thiết kế dữ liệu cần đảm bảo ba điểm. Truy vấn phải nhanh. Dữ liệu phải dễ mở rộng. Các quan hệ nghiệp vụ quan trọng như người dùng, hồ sơ, tin tuyển dụng, đơn ứng tuyển và giao dịch phải rõ ràng để service xử lý đúng.

3.5.1. Các Collections trong User Service

3.5.1.1. Collection users lưu thông tin tài khoản và xác thực

Collection users lưu thông tin tài khoản chung của toàn hệ thống. Đây là dữ liệu nền để xác thực và phân quyền người dùng.

Các trường chính gồm email, password_hash, role, is_verified, status, created_at và updated_at. Mật khẩu không lưu trực tiếp mà được mã hóa bằng BCrypt trước khi ghi vào cơ sở dữ liệu.

Role xác định người dùng thuộc nhóm CANDIDATE, RECRUITER hoặc ADMIN. Trạng thái is_verified cho biết tài khoản đã xác thực OTP hay chưa. Trường status hỗ trợ khóa tài khoản hoặc vô hiệu hóa khi cần.

Bảng 3.5 Bảng mô tả cấu trúc Collection Users

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id
full_name	varchar	Họ và tên đầy đủ
email	varchar	Email đăng nhập
password	varchar	Mật khẩu đã mã hóa
phone	varchar	Số điện thoại
avt_image_id	varchar	Id ảnh đại diện
created_at	datetime	Thời điểm tạo
updated_at	datetime	Thời điểm cập nhật
deleted_at	datetime	Thời điểm xóa mềm
verified	boolean	Trạng thái xác thực
deleted	boolean	Cờ xóa mềm
locked	boolean	Cờ khóa tài khoản
preferences	json	PreferencesSettings nhúng

3.5.1.2. Collection role - Vai trò và quyền hạn

Bảng 3.6 Bảng mô tả cấu trúc Collection role

Tên cột	Kiểu dữ liệu	Mô tả
name	varchar	Khóa chính của role.
description	varchar	Mô tả role.

permissions	json	Tập Permission nhúng trong role.
-------------	------	----------------------------------

3.5.1.3. Collection user_roles - Liên kết người dùng và vai trò

Bảng 3.7 Bảng mô tả cấu trúc Collection user_roles

Tên cột	Kiểu dữ liệu	Mô tả
user_id	varchar	Tham chiếu đến users.id.
role_name	varchar	Tham chiếu đến role.name.

3.5.1.4. Collection candidates - Hồ sơ mở rộng ứng viên

Collection candidates lưu hồ sơ chi tiết của ứng viên. Dữ liệu này tách khỏi users để tránh làm tài khoản chung trở nên quá lớn và dễ mở rộng thông tin nghề nghiệp.

Các nhóm dữ liệu chính gồm user_id, full_name, phone, location, skills, education, experiences và danh sách CV. Mỗi CV có thể lưu file_url, file_name, is_default, uploaded_at và raw_text_parsed.

Trường raw_text_parsed rất quan trọng. Sau khi CV được tải lên, hệ thống cần trích xuất văn bản để AI Engine dùng lại khi chấm điểm. Nếu đã có raw text, quá trình nộp đơn không phải parse lại file nhiều lần.

Bảng 3.8 Bảng mô tả cấu trúc Collection Candidates

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
user_id	varchar	Tham chiếu đến users.id.
dob	date	Ngày sinh.
gender	varchar	Giới tính.
address	text	Địa chỉ.
bio	text	Giới thiệu bản thân.
title	varchar	Tiêu đề nghề nghiệp.
avatar_url	varchar	URL ảnh đại diện.
skills	json	Danh sách kỹ năng.
years_of_experience	int	Số năm kinh nghiệm.
experiences	json	Danh sách

		WorkExperience.
educations	json	Danh sách Education.
certifications	json	Danh sách Certification.
languages	json	Danh sách Language.
job_type	varchar	Loại công việc mong muốn.
preferred_location	varchar	Địa điểm ưu tiên.
expected_salary_min	int	Mức lương mong muốn tối thiểu.
expected_salary_max	int	Mức lương mong muốn tối đa.
portfolio_url	varchar	URL portfolio.
github_url	varchar	URL GitHub.
linkedin_url	varchar	URL LinkedIn.
cv_url	varchar	URL CV hiện tại.
cvs	json	Danh sách CvItem.
settings	json	CandidateSettings nhúng trong document.
job_suggestions	json	Cache gợi ý việc làm.
suggestions_updated_at	datetime	Thời điểm cập nhật gợi ý.
followed_company_ids	json	Danh sách company id đã theo dõi.
created_at	datetime	Thời điểm tạo.
updated_at	datetime	Thời điểm cập nhật.
deleted	boolean	Cờ xóa mềm.
deleted_at	datetime	Thời điểm xóa mềm.

3.5.1.5. Collection recruiters - Hồ sơ công ty nhà tuyển dụng

Collection recruiters lưu thông tin mở rộng của nhà tuyển dụng và công ty. Một tài khoản nhà tuyển dụng chỉ được đăng tin khi hồ sơ công ty đã được quản trị viên phê duyệt.

Các trường chính gồm `user_id`, `company_name`, `tax_code`, `address`, `website`, `logo_url`, `business_license_url`, `approval_status`, `quota_remaining` và `reject_reason`.
Trạng thái phê duyệt có thể là `PENDING`, `APPROVED` hoặc `REJECTED`.

Quota đăng tin được lưu để kiểm soát quyền đăng tuyển. Khi nhà tuyển dụng thanh toán gói mới, `quota_remaining` được cộng thêm theo giao dịch thành công.

Bảng 3.9 Bảng mô tả cấu trúc Collection Recruiters

Tên cột	Kiểu dữ liệu	Mô tả
<code>id</code>	<code>varchar</code>	MongoDB document id.
<code>user_id</code>	<code>varchar</code>	Tham chiếu đến <code>users.id</code> .
<code>company_name</code>	<code>varchar</code>	Tên công ty.
<code>company_website</code>	<code>varchar</code>	Website công ty.
<code>company_address</code>	<code>text</code>	Địa chỉ công ty.
<code>company_city</code>	<code>varchar</code>	Thành phố.
<code>company_description</code>	<code>text</code>	Mô tả công ty.
<code>company_phone</code>	<code>varchar</code>	Số điện thoại công ty.
<code>company_size</code>	<code>varchar</code>	Quy mô công ty.
<code>company_type</code>	<code>varchar</code>	Loại hình công ty.
<code>founded_year</code>	<code>int</code>	Năm thành lập.
<code>industry</code>	<code>varchar</code>	Ngành nghề.
<code>benefits</code>	<code>json</code>	Danh sách quyền lợi.
<code>rating</code>	<code>float</code>	Điểm đánh giá.
<code>review_count</code>	<code>int</code>	Số lượng đánh giá.
<code>logo_url</code>	<code>varchar</code>	URL logo.
<code>cover_image_url</code>	<code>varchar</code>	URL ảnh bìa.
<code>tax_code</code>	<code>varchar</code>	Mã số thuế.
<code>business_license_url</code>	<code>varchar</code>	URL giấy phép kinh doanh.
<code>linkedin_url</code>	<code>varchar</code>	URL LinkedIn.
<code>facebook_url</code>	<code>varchar</code>	URL Facebook.
<code>contact_name</code>	<code>varchar</code>	Tên người liên hệ.
<code>contact_email</code>	<code>varchar</code>	Email liên hệ.

contact_phone	varchar	Số điện thoại liên hệ.
status	varchar	Trạng thái recruiter.
rejection_note	text	Ghi chú từ chối.
quota_job_post	int	Hạn mức đăng job.
quota_cv_views	int	Hạn mức xem CV.
created_at	datetime	Thời điểm tạo.
updated_at	datetime	Thời điểm cập nhật.
deleted	boolean	Cờ xóa mềm.
deleted_at	datetime	Thời điểm xóa mềm.

3.5.1.6. Collection service_packages - Gói dịch vụ

Bảng 3.10 Bảng mô tả cấu trúc Collection Service Packages

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
name	varchar	Tên gói dịch vụ.
price	bigint	Giá gói.
ai_credits	int	Số credit AI.
job_limit	int	Hạn mức đăng job.
cv_limit	int	Hạn mức xem CV.
featured	boolean	Gói nổi bật.
features	json	Danh sách tính năng.
created_at	datetime	Thời điểm tạo.
updated_at	datetime	Thời điểm cập nhật.

3.5.1.7. Collection wishlists - Danh sách việc làm đã lưu

Bảng 3.11 Bảng mô tả cấu trúc Collection Wishlists

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
candidate_id	varchar	Tham chiếu đến candidates.id.
job_id	varchar	Tham chiếu đến jobs.id ở job service.
saved_at	datetime	Thời điểm lưu.

deleted	boolean	Cờ xóa mềm.
---------	---------	-------------

3.5.2. Các Collections trong Job Service

3.5.2.1. Collection jobs - Tin tuyển dụng

Collection jobs lưu dữ liệu tin tuyển dụng do nhà tuyển dụng tạo. Đây là một trong các collection được truy vấn nhiều nhất vì ứng viên thường tìm kiếm và lọc việc làm liên tục.

Các trường chính gồm recruiter_id, title, description_html, requirements, skills_required, salary_min, salary_max, location, job_type, status, expired_at, created_at và updated_at. Trạng thái tin có thể là OPEN, CLOSED hoặc EXPIRED.

Sau khi tin tuyển dụng được tạo hoặc cập nhật, một phần dữ liệu sẽ được đồng bộ sang Elasticsearch. MongoDB vẫn giữ vai trò dữ liệu gốc, còn Elasticsearch phục vụ tìm kiếm toàn văn và xếp hạng kết quả.

Việc tách tìm kiếm ra Elasticsearch giúp giảm tải cho MongoDB. Các bộ lọc như địa điểm, mức lương và kỹ năng cũng dễ tối ưu hơn khi số lượng tin tuyển dụng tăng.

Bảng 3.12 Bảng mô tả cấu trúc Collection Jobs

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
recruiter_id	varchar	Id recruiter user từ gateway.
title	varchar	Tiêu đề job.
normalized_title	varchar	Tiêu đề đã chuẩn hóa.
description	text	Mô tả công việc.
company	varchar	Tên công ty.
location	varchar	Địa điểm làm việc.
salary_min	float	Lương tối thiểu.
salary_max	float	Lương tối đa.
job_type	varchar	Loại công việc.
experience_level	varchar	Cấp độ kinh nghiệm.
skills	json	Danh sách kỹ năng.

requirements	json	Danh sách yêu cầu.
benefits	json	Danh sách quyền lợi.
moderation_status	varchar	Trạng thái duyệt bài.
visibility_status	varchar	Trạng thái hiển thị.
moderation_note	text	Ghi chú kiểm duyệt.
reviewed_by	varchar	Người duyệt.
reviewed_at	datetime	Thời điểm duyệt.
deadline	date	Hạn ứng tuyển.
openings	int	Số lượng tuyển.
qualified_threshold	int	Ngưỡng đạt.
reject_threshold	int	Ngưỡng loại.
auto_reject_enabled	boolean	Bật tự động loại.
required_test	varchar	Bài test bắt buộc.
deleted	boolean	Cờ xóa mềm.
deleted_at	datetime	Thời điểm xóa mềm.
created_at	datetime	Thời điểm tạo.
updated_at	datetime	Thời điểm cập nhật.

3.5.3. Các Collection trong Application Service

3.5.3.1. Collection applications - Đơn ứng tuyển và kết quả AI

Collection applications lưu thông tin mỗi lần ứng viên nộp đơn vào một tin tuyển dụng. Đây là collection liên kết trực tiếp giữa ứng viên, CV, tin tuyển dụng và quy trình ATS của nhà tuyển dụng.

Các trường chính gồm candidate_user_id, job_id, recruiter_id, cv_id, status, cover_letter, applied_at, updated_at và ai_analysis. Trạng thái có thể đi qua các bước PENDING_REVIEW, QUALIFIED, NOT_QUALIFIED, UNDER_REVIEW, INTERVIEW_SCHEDULED, OFFER_SENT, ACCEPTED hoặc REJECTED.

Trường ai_analysis được thiết kế dạng embedded document. Nội dung gồm match_score, matched_skills, missing_skills, advice, summary và mock_questions_generated. Cách lưu này giúp màn hình chi tiết đơn ứng tuyển lấy được kết quả AI cùng với trạng thái ATS trong một truy vấn.

Khi AI Engine xử lý xong, Application Service cập nhật ai_analysis và trạng thái tương ứng. Nếu điểm phù hợp cao, đơn có thể được đưa vào nhóm QUALIFIED. Nếu điểm thấp, hệ thống có thể đánh dấu NOT_QUALIFIED để nhà tuyển dụng kiểm tra lại.

Bảng 3.13 Bảng mô tả cấu trúc Collection Applications

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
candidate_id	varchar	Tham chiếu đến candidate user id.
job_id	varchar	Tham chiếu đến jobs.id.
recruiter_id	varchar	Tham chiếu đến recruiter user id.
candidate_email	varchar	Email ứng tuyển.
job_title	varchar	Tiêu đề job snapshot.
company_name	varchar	Tên công ty snapshot.
job_location	varchar	Địa điểm job snapshot.
salary_min	float	Lương tối thiểu snapshot.
salary_max	float	Lương tối đa snapshot.
job_skills	json	Kỹ năng job snapshot.
job_type	varchar	Loại job snapshot.
status	varchar	Trạng thái application.
cover_letter	text	Cover letter.
cv_url	text	URL CV.
ai_score	int	Điểm AI đánh giá.
matched_skills	json	Kỹ năng khớp.
missing_skills	json	Kỹ năng thiếu.
ai_status	varchar	Trạng thái chấm AI.
recruiter_notes	text	Ghi chú recruiter.
rejection_reason	text	Lý do từ chối.
applied_at	datetime	Thời điểm ứng tuyển.
updated_at	datetime	Thời điểm cập nhật.

deleted	boolean	Cờ xóa mềm.
deleted_at	datetime	Thời điểm xóa mềm.

3.5.3.2. Collection assessments - Bài đánh giá tuyển dụng

Bảng 3.14 Bảng mô tả cấu trúc Collection Assessments

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
job_id	varchar	Tham chiếu đến jobs.id.
recruiter_id	varchar	Tham chiếu đến recruiter user id.
title	varchar	Tiêu đề assessment.
description	text	Mô tả assessment.
questions	json	Danh sách Question.
time_limit_minutes	int	Giới hạn thời gian làm bài.
status	varchar	Trạng thái assessment.
created_at	datetime	Thời điểm tạo.

3.5.3.3. Collection assessment_attempts - Lượt làm bài đánh giá

Bảng 3.15 Bảng mô tả cấu trúc Collection Assessment Attempts

Tên cột	Kiểu dữ liệu	Mô tả
id	varchar	MongoDB document id.
assessment_id	varchar	Tham chiếu đến assessments.id.
candidate_id	varchar	Tham chiếu đến candidate user id.
application_id	varchar	Tham chiếu đến applications.id.
status	varchar	Trạng thái lượt làm bài.
answers	json	Danh sách AttemptAnswer.
started_at	datetime	Thời điểm bắt đầu.
submitted_at	datetime	Thời điểm nộp bài.
score	float	Điểm số.

result	varchar	Kết quả bài làm.
--------	---------	------------------

3.5.4. Các bảng trong Notification Service

3.5.4.1. Table notifications - Thông báo người dùng

Bảng 3.16 Bảng mô tả cấu trúc Table Notifications

Tên cột	Kiểu dữ liệu	Mô tả
id	uuid	Khóa chính của notification.
user_id	text	Id người nhận.
recipient_role	varchar	Vai trò người nhận.
type	varchar	Loại thông báo.
title	varchar	Tiêu đề thông báo.
body	text	Nội dung thông báo.
data	json	Payload bổ sung.
is_read	boolean	Đã đọc hay chưa.
read_at	timestampz	Thời điểm đọc.
created_at	timestampz	Thời điểm tạo.

3.5.4.2. Table notification_fcm_tokens - Token thông báo FCM

Bảng 3.17 Bảng mô tả cấu trúc Table Notification FCM Tokens

Tên cột	Kiểu dữ liệu	Mô tả
id	uuid	Khóa chính của token.
user_id	text	Id người dùng sở hữu token.
audience	varchar	Nhóm client, ví dụ web-user.
token	text	FCM registration token.
created_at	timestampz	Thời điểm tạo.
updated_at	timestampz	Thời điểm cập nhật.

3.5.5. Collection transactions_log - Lịch sử giao dịch thanh toán

Collection transactions_log lưu lịch sử mua gói đăng tin của nhà tuyển dụng. Dữ liệu giao dịch cần rõ ràng để đối chiếu khi có lỗi thanh toán hoặc sai lệch quota.

Các trường chính gồm `recruiter_id`, `package_code`, `amount`, `payment_gateway`, `transaction_code`, `status`, `quota_added`, `created_at` và `paid_at`. Trạng thái giao dịch có thể là `PENDING`, `SUCCESS` hoặc `FAILED`.

Khi thanh toán thành công, hệ thống cần vừa cập nhật `transaction log`, vừa cộng `quota` cho hồ sơ nhà tuyển dụng. Hai thao tác này nên được xử lý trong `transaction` để tránh trường hợp tiền đã ghi nhận nhưng `quota` chưa được cộng, hoặc ngược lại.

3.5.6. Redis Key Schema - Cache và OTP

Redis không lưu dữ liệu nghiệp vụ dài hạn. Nó được dùng cho dữ liệu ngắn hạn, truy cập nhanh và có thời gian sống rõ ràng.

Nhóm key đầu tiên là OTP. Key có dạng `otp_email` và lưu mã xác thực trong thời gian ngắn, thường là 5 phút. Khi người dùng nhập OTP, User Service kiểm tra key này rồi xóa sau khi xác thực thành công.

Nhóm key thứ hai là `cache` tìm kiếm việc làm. Key có dạng `job_cache_filter_hash`, trong đó `filter_hash` được tạo từ từ khóa và bộ lọc tìm kiếm. Cache này giúp giảm số lần truy vấn Elasticsearch với các bộ lọc phổ biến.

Một số key phụ có thể dùng cho giới hạn tần suất gửi OTP hoặc chống thao tác lặp. Ví dụ `rate_limit_email` dùng để hạn chế người dùng yêu cầu quá nhiều mã OTP trong thời gian ngắn.

Redis cần được dùng có kiểm soát. Dữ liệu cache phải có TTL, còn dữ liệu nghiệp vụ quan trọng vẫn phải lưu ở MongoDB để đảm bảo không mất khi Redis bị xóa hoặc khởi động lại.

CHƯƠNG 4. THỰC NGHIỆM VÀ KẾT QUẢ

4.1. Môi trường phát triển và công nghệ sử dụng

4.1.1. Cấu hình phần cứng

Quá trình triển khai và kiểm thử hệ thống end-to-end được thực hiện trên máy tính cá nhân với cấu hình: CPU Intel Core i5-12450HX (8 nhân, 12 luồng, 4.4 GHz Boost), RAM 16 GB DDR4 3200 MHz, ổ cứng SSD NVMe 512 GB, hệ điều hành Ubuntu 24.04 LTS. Cấu hình này đủ để kiểm thử chức năng đầy đủ của hệ thống trong môi trường development.

4.1.2. Môi trường phần mềm

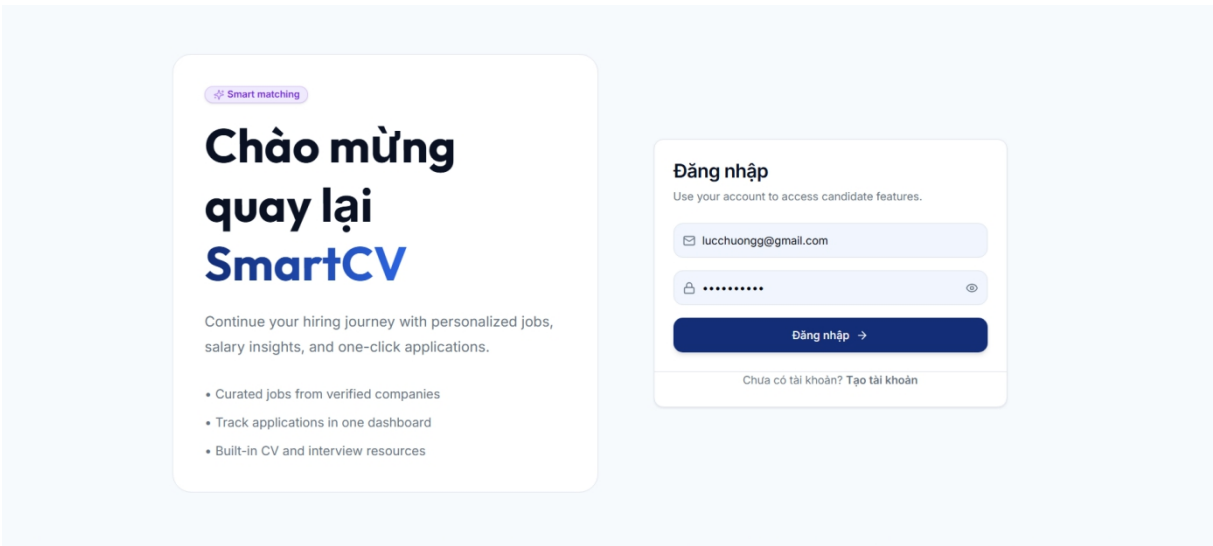
Bảng 4.1 Bảng môi trường phát triển (phần mềm)

Thành phần	Phiên bản	Ghi chú
Hệ điều hành	Ubuntu 24.04 LTS	Máy development
Docker Engine	24.0.7	Container runtime
Docker Compose	2.21.0	Orchestration
Java Development Kit	21	Phát triển các services backend
Go	1.24.0	SDK cho Notification Service
Node.js	20.11 LTS	Frontend build
Mongodb	8.3.4	Non-Relational DB
PostgreSQL	15.4	Relational DB
React	18.2.0	Frontend framework

4.2. Thiết kế và cài đặt giao diện

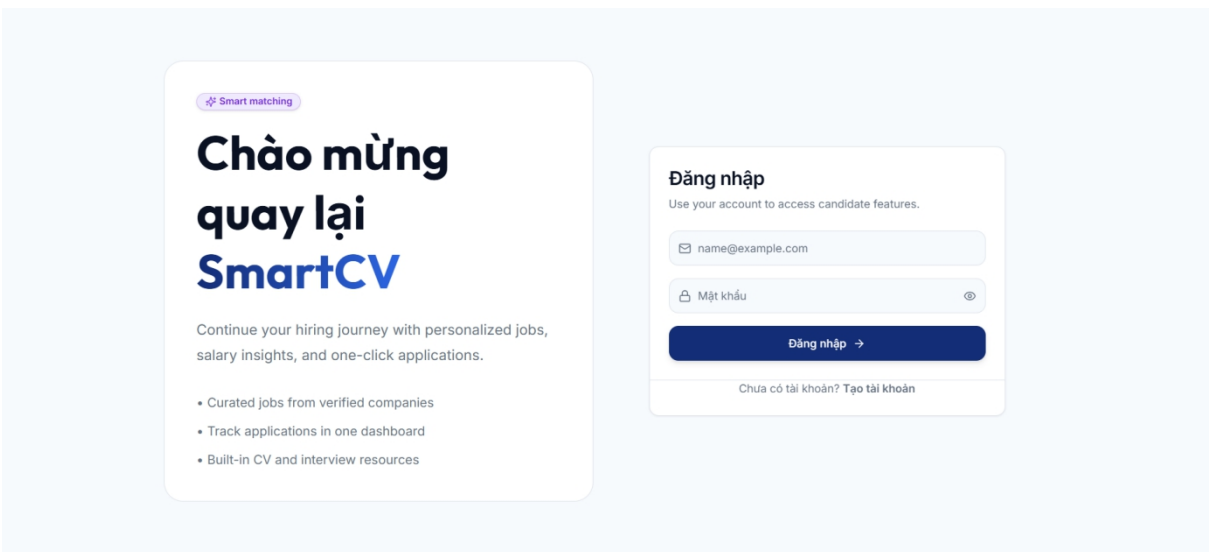
4.2.1. Giao diện đăng ký, đăng nhập

4.2.1.1. Giao diện đăng ký



Hình 4.1 Giao diện đăng ký tài khoản

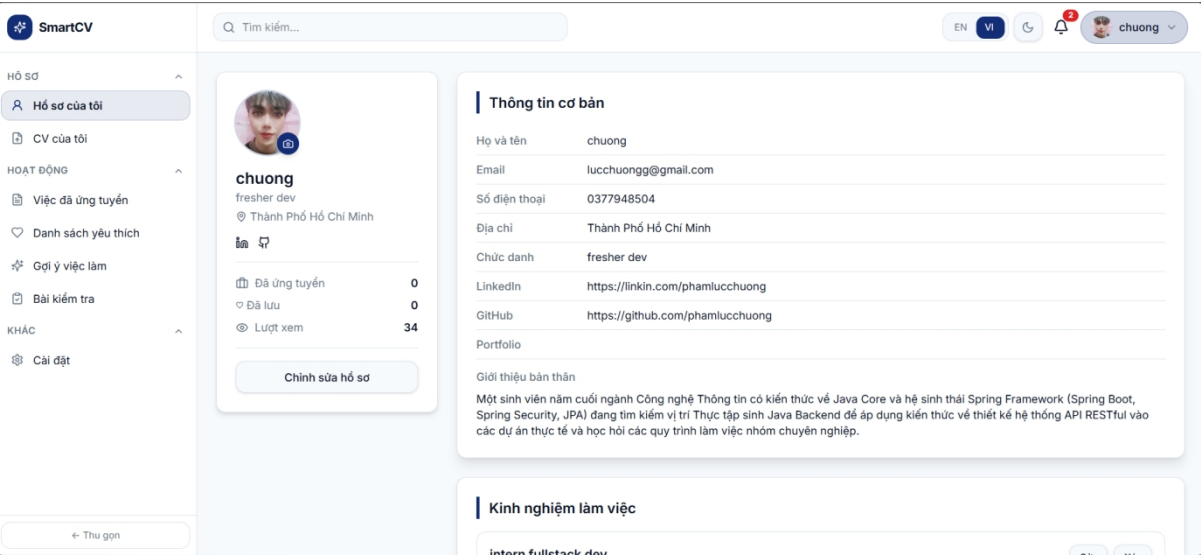
4.2.1.2. Giao diện đăng nhập



Hình 4.2 Giao diện đăng nhập

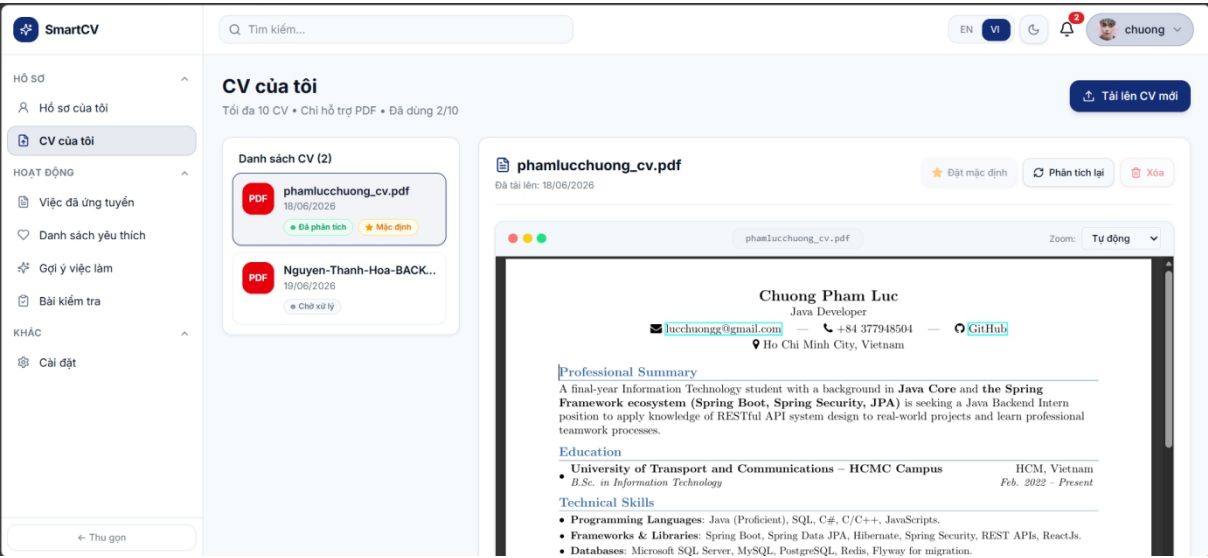
4.2.2. Giao diện màn hình trang chủ

4.2.2.1. Giao diện Quản lý hồ sơ ứng viên

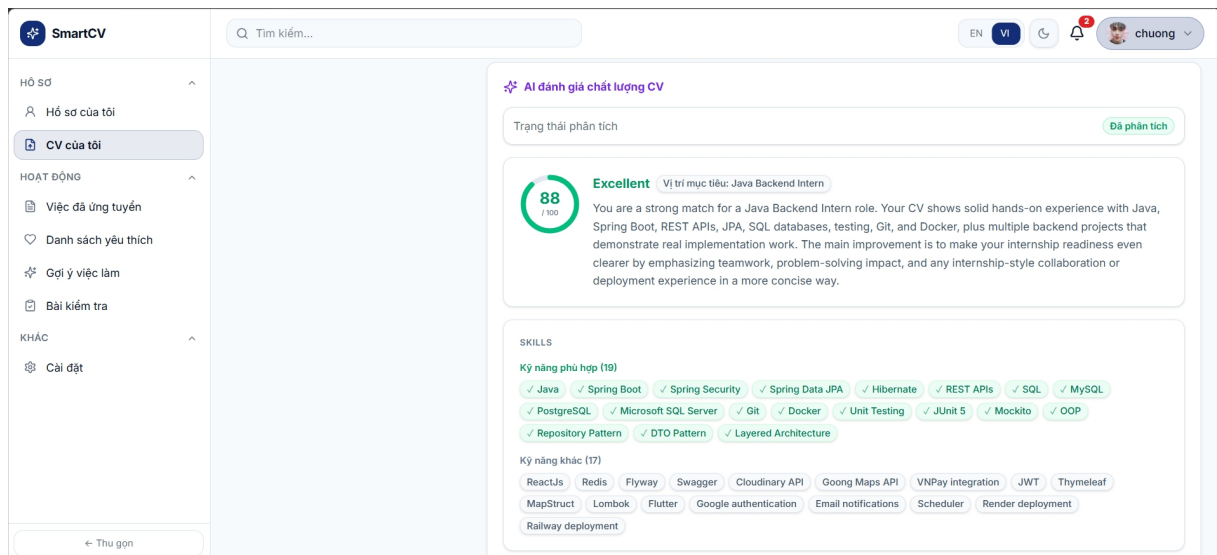


Hình 4.3 Giao diện quản lý hồ sơ ứng viên

4.2.2.2. Giao diện quản lý CV và xem kết quả AI phân tích CV



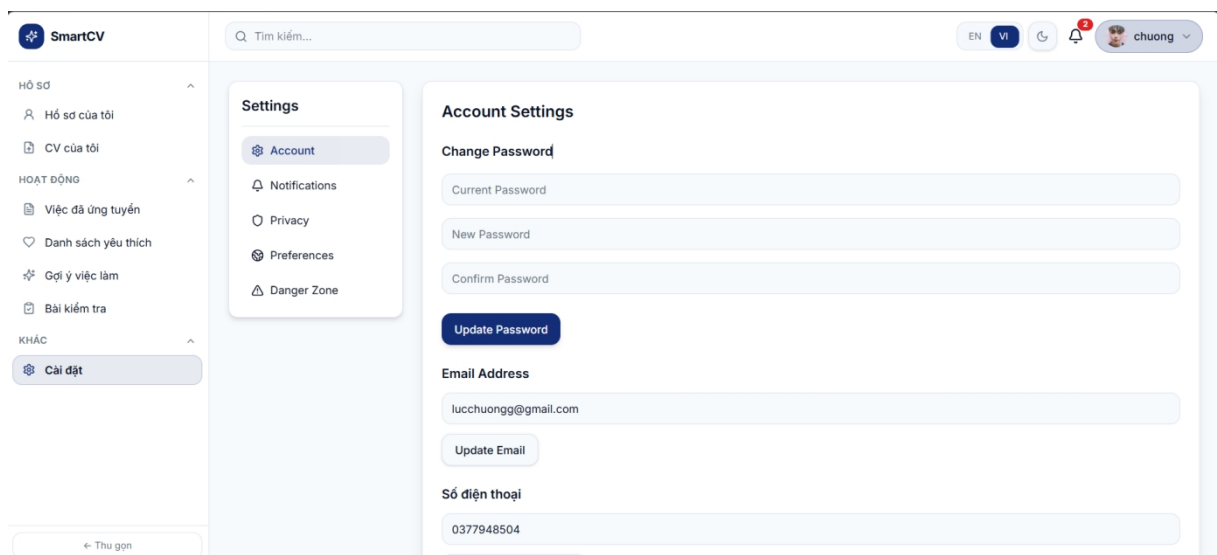
Hình 4.4 Giao diện quản lý và xem nội dung CV



Hình 4.5 Giao diện kết quả AI phân tích CV

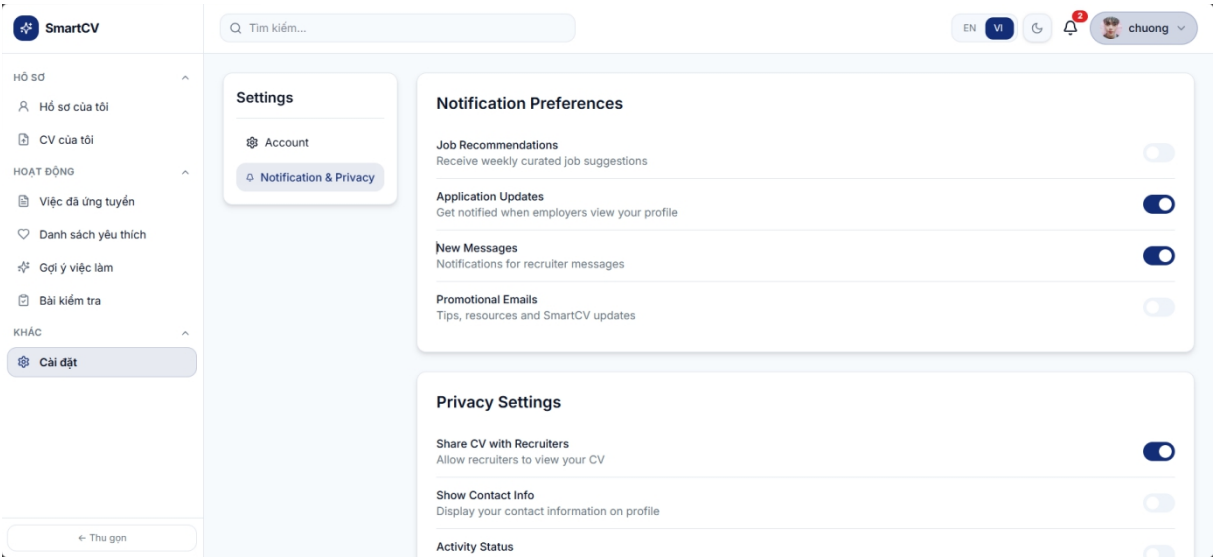
4.2.3. Giao diện màn hình cài đặt

4.2.3.1. Giao diện Cài đặt tài khoản



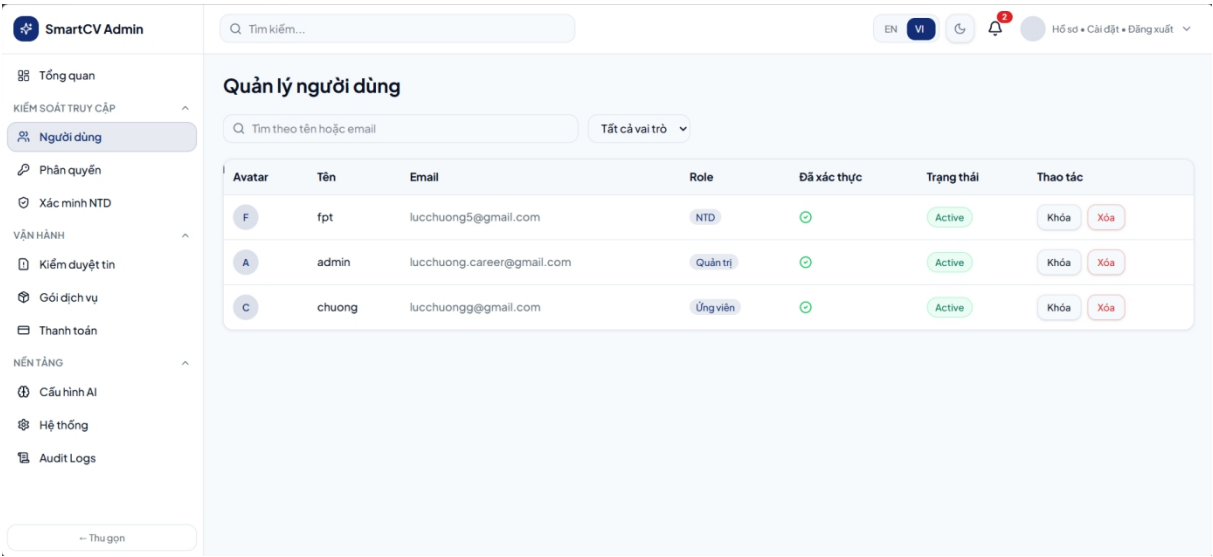
Hình 4.6 Giao diện Cài đặt tài khoản

4.2.3.2. Giao diện cài đặt thông báo và quyền cá nhân

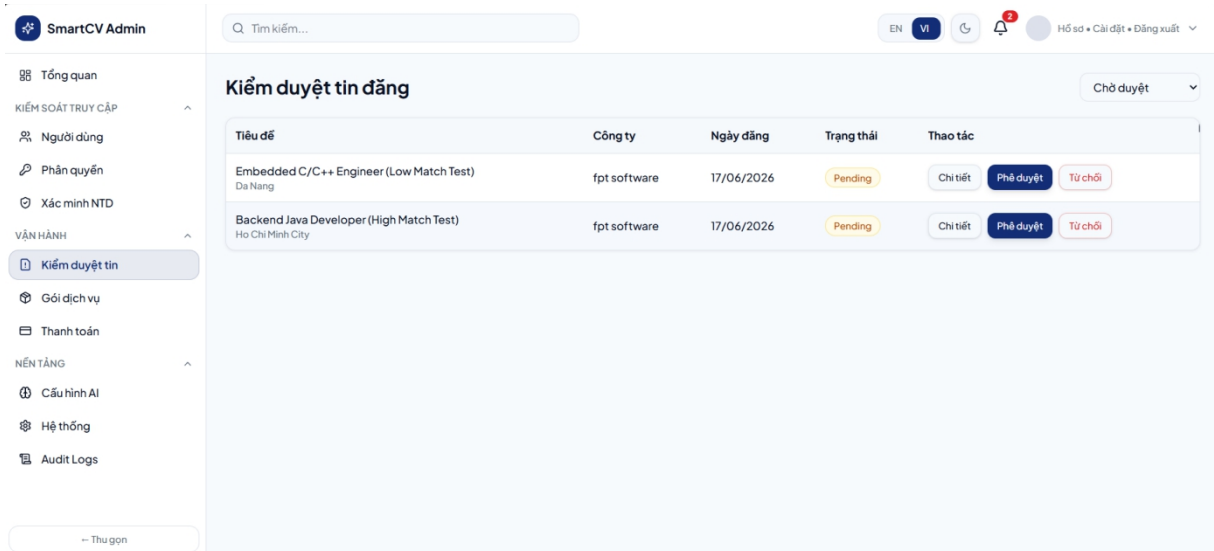


Hình 4.7 Giao diện cài đặt thông báo và quyền cá nhân

4.2.4. Thiết kế và cài đặt giao diện quản trị người dùng



Hình 4.8 Giao diện Quản trị người dùng



Hình 4.9 Giao diện kiểm duyệt tin tuyển dụng

CHƯƠNG 5. KẾT LUẬN VÀ KIẾN NGHỊ

5.1. Kết quả đạt được

Sau thời gian tìm hiểu và phát triển đề tài Xây dựng hệ thống hỗ trợ đánh giá CV và gợi ý việc làm cho ứng viên dựa trên kiến trúc Microservices và trí tuệ nhân tạo, nhóm đã hoàn thành được các chức năng cốt lõi đã đặt ra ở phần mục tiêu. Hệ thống có thể vận hành theo luồng tuyển dụng cơ bản, từ tạo tài khoản, đăng tin, tìm kiếm việc làm, nộp hồ sơ đến phân tích CV bằng AI.

Hệ thống chạy được từ đầu đến cuối. Đây là kết quả quan trọng.

Ở phía người dùng, em đã xây dựng ba giao diện riêng cho ứng viên, nhà tuyển dụng và quản trị viên. Cách tách giao diện như vậy giúp mỗi nhóm người dùng chỉ nhìn thấy những chức năng cần cho công việc của mình, hạn chế thao tác dư thừa và giảm nhầm lẫn khi sử dụng.

Các chức năng chính đã hoàn thành gồm:

- Quản lý tài khoản, đăng ký, đăng nhập và xác thực bằng JWT.
- Xác thực email bằng OTP, lưu OTP có thời hạn trong Redis.
- Quản lý hồ sơ ứng viên, kỹ năng, kinh nghiệm và CV tải lên.
- Quản lý hồ sơ nhà tuyển dụng, thông tin công ty và trạng thái phê duyệt.
- Đăng tin tuyển dụng, chỉnh sửa tin, đóng tin và tìm kiếm việc làm.
- Nộp đơn ứng tuyển bằng CV đã chọn.
- Phân tích CV so với mô tả công việc bằng LLM thông qua Spring AI và Azure Openai API.
- Trả về điểm phù hợp, kỹ năng phù hợp, kỹ năng còn thiếu và gợi ý cải thiện CV.
- Quản lý ứng viên theo dạng Kanban ATS cho nhà tuyển dụng.
- Gửi thông báo email tự động cho các luồng chính như OTP và thay đổi trạng thái hồ sơ.

Ở phía kiến trúc, hệ thống đã được tổ chức theo hướng microservices. Các service chính gồm User Service, Job Service, Application Service, AI Engine Service và Notification Service. API Gateway đóng vai trò cửa vào chung, xử lý định tuyến và kiểm soát xác thực. RabbitMQ được dùng để truyền sự kiện bất đồng bộ, nhất là khi kích hoạt chấm điểm CV và gửi email thông báo.

Phần dữ liệu cũng được tách theo đúng nhu cầu. MongoDB lưu dữ liệu nghiệp vụ linh hoạt. Redis phục vụ OTP và cache. Elasticsearch hỗ trợ tìm kiếm tin tuyển dụng theo từ khóa, địa điểm, mức lương và bộ lọc. Cách kết hợp này giúp hệ thống không bị phụ thuộc vào một loại cơ sở dữ liệu duy nhất.

Phần AI đã tạo được giá trị rõ ràng. Khi ứng viên nộp CV, hệ thống không chỉ lưu đơn ứng tuyển mà còn phân tích mức độ phù hợp với công việc. Kết quả AI giúp nhà tuyển dụng có thêm một góc nhìn khi sàng lọc hồ sơ, đồng thời giúp ứng viên biết CV của mình còn thiếu kỹ năng nào để cải thiện.

Một số điểm đạt được có thể xem là nền tảng cho các bước phát triển tiếp theo:

- Hệ thống có kiến trúc tách lớp rõ, dễ mở rộng thêm service mới.
- Luồng xử lý AI chạy bất đồng bộ nên không làm nghẽn luồng nộp hồ sơ.
- Frontend dùng React và TypeScript, dễ bảo trì khi số lượng màn hình tăng.
- API được thiết kế theo REST, có thể tích hợp với web, mobile hoặc hệ thống bên ngoài.

Kết quả AI được lưu lại trong đơn ứng tuyển, tiện cho việc tra cứu và đánh giá sau này.

5.2. Hạn chế của hệ thống

Bên cạnh các phần đã hoàn thành, hệ thống vẫn còn một số hạn chế. Một phần đến từ thời gian thực hiện đồ án. Một phần khác đến từ việc hệ thống dùng nhiều công nghệ cùng lúc, nên chưa thể tối ưu sâu ở mọi thành phần.

Các hạn chế chính gồm:

- Phụ thuộc vào Azure Openai API: Chất lượng phân tích CV khá tốt, nhưng hệ thống vẫn phụ thuộc vào nhà cung cấp bên thứ ba. Nếu API chậm, lỗi hoặc thay đổi chi phí, luồng phân tích AI sẽ bị ảnh hưởng.
- Chi phí token khi xử lý hàng loạt: Khi số lượng CV lớn, chi phí gọi LLM có thể tăng nhanh. Đây là vấn đề cần tính toán kỹ nếu triển khai thật cho doanh nghiệp.
- Chưa có thông báo real-time: Một số màn hình vẫn cần tải lại hoặc gọi API định kỳ để xem kết quả mới. Cách này dùng được trong phạm vi thử nghiệm, nhưng chưa tạo trải nghiệm tốt bằng WebSocket hoặc SSE.

- Tìm kiếm tiếng Việt chưa tối ưu hoàn toàn: Elasticsearch đã hỗ trợ tìm kiếm nhanh, nhưng việc tách từ tiếng Việt, từ đồng nghĩa và các biến thể kỹ năng vẫn cần xử lý thêm.
- Kết quả AI cần được kiểm chứng bởi con người: Điểm phù hợp và gợi ý kỹ năng chỉ nên dùng như công cụ hỗ trợ. Nhà tuyển dụng vẫn cần đọc CV, xem kinh nghiệm thực tế và đánh giá ứng viên theo tiêu chí riêng.
- Chưa triển khai production cloud: Hệ thống chủ yếu được kiểm thử trên localhost và Docker Compose. Các vấn đề như autoscaling, giám sát log, backup dữ liệu và bảo mật hạ tầng chưa được triển khai đầy đủ.
- Chưa có dữ liệu sử dụng thật ở quy mô lớn: Các kiểm thử hiện tại chủ yếu dựa trên mẫu CV và tin tuyển dụng thử nghiệm. Vì vậy, độ ổn định khi có nhiều người dùng đồng thời vẫn cần được đánh giá thêm.

Những hạn chế này không làm mất ý nghĩa của đề tài. Ngược lại, chúng cho thấy rõ các phần cần ưu tiên nếu SmartCV được phát triển thành sản phẩm thực tế.

5.3. Hướng phát triển

Từ các kết quả và hạn chế trên, nhóm đề xuất một số hướng phát triển để hệ thống hoàn thiện hơn.

Bổ sung WebSocket hoặc Server-Sent Events: Kết quả phân tích AI, thay đổi trạng thái hồ sơ và thông báo mới có thể được đẩy trực tiếp lên giao diện. Người dùng sẽ không cần tải lại trang.

Tối ưu LLM cho CV tiếng Việt: Có thể xây dựng bộ dữ liệu CV và mô tả công việc tiếng Việt để đánh giá prompt tốt hơn. Về sau, có thể fine-tune hoặc dùng mô hình cục bộ cho một số tác vụ lặp lại.

Xây dựng Recommendation Engine: Hệ thống có thể gợi ý việc làm phù hợp dựa trên kỹ năng, kinh nghiệm, lịch sử nộp đơn và mức lương mong muốn của ứng viên.

Cải thiện tìm kiếm tiếng Việt: Bổ sung bộ phân tích từ tiếng Việt, chuẩn hóa tên kỹ năng và ánh xạ các từ tương đương như JavaScript, JS, ReactJS và React.

Mở rộng thanh toán thực tế: Tích hợp VNPAY, Momo hoặc cổng thanh toán ngân hàng để nhà tuyển dụng mua gói đăng tin và quản lý quota tự động.

Bổ sung tính năng chat: Ứng viên và nhà tuyển dụng có thể trao đổi trực tiếp sau khi hồ sơ được quan tâm. Tính năng này giúp giảm phụ thuộc vào email.

Triển khai lên cloud: Có thể dùng AWS, GCP hoặc máy chủ riêng với CI/CD tự động, giám sát log, backup dữ liệu, quản lý secrets và tách môi trường dev, staging, production.

Hoàn thiện bảo mật: Bổ sung rate limiting chi tiết, audit log, phân quyền theo từng hành động và cơ chế khóa tài khoản khi phát hiện đăng nhập bất thường.

Đánh giá hệ thống với người dùng thật: Thu thập phản hồi từ ứng viên và nhà tuyển dụng để điều chỉnh giao diện, tiêu chí AI scoring và luồng ATS.

Nếu tiếp tục được phát triển, hệ thống có thể trở thành nền tảng hỗ trợ tuyển dụng có giá trị thực tế. Hệ thống không thay thế hoàn toàn con người. Nó giúp giảm phần việc lặp lại, lọc thông tin nhanh hơn và tạo dữ liệu rõ ràng hơn cho quyết định tuyển dụng.

TÀI LIỆU THAM KHẢO

- [1] [A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser và I. Polosukhin, Attention Is All You Need, Advances in Neural Information Processing Systems, 2017 \[20/04/2026\]](#)
- [2] [Spring, Spring AI Reference Documentation, 2026. \[22/04/2026\].](#)
- [3] [Spring, Spring Boot Reference Documentation, 2026. \[22/04/2026\].](#)
- [4] [C. Richardson, Microservices Patterns: With examples in Java. Manning Publications, 2018.\[26/04/2026\]](#)
- [5] [S. Newman, Building Microservices, 2nd ed. O Reilly Media, 2021.\[26/04/2026\]](#)
- [6] [RabbitMQ, RabbitMQ Documentation, 2026. \[01/05/2026\].](#)
- [7] [MongoDB, MongoDB Manual, 2026. .\[01/05/2026\]](#)
- [8] [Redis, Redis Documentation, 2026. \[02/05/2026\]](#)
- [9] [Elastic, Elasticsearch Guide, 2026. \[02/05/2026\]](#)
- [10] [Meta Open Source, React Documentation, 2026. \[02/05/2026\].](#)
- [11] [TanStack, TanStack Query Documentation, 2026. \[06/05/2026\].](#)
- [12] [Docker Inc., Docker Documentation, 2026. \[10/05/2026\]](#)
- [13] [M. Jones, J. Bradley và N. Sakimura, JSON Web Token, RFC 7519, Internet Engineering Task Force, \[20/04/2026\]](#)
- [14] [D. F. Ferraiolo và D. R. Kuhn, Role-Based Access Controls, 15th National Computer Security Conference, 1992 \[10/06/2026\]](#)
- [15] [E. Gamma, R. Helm, R. Johnson và J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994 \[01/06/2026\]](#)
- [16] [OpenAPI Initiative, OpenAPI Specification, 2026. \[01/06/2026\]](#)

PHỤ LỤC

- [1] Link github repository: <https://github.com/phamlucchuong/smartCy>